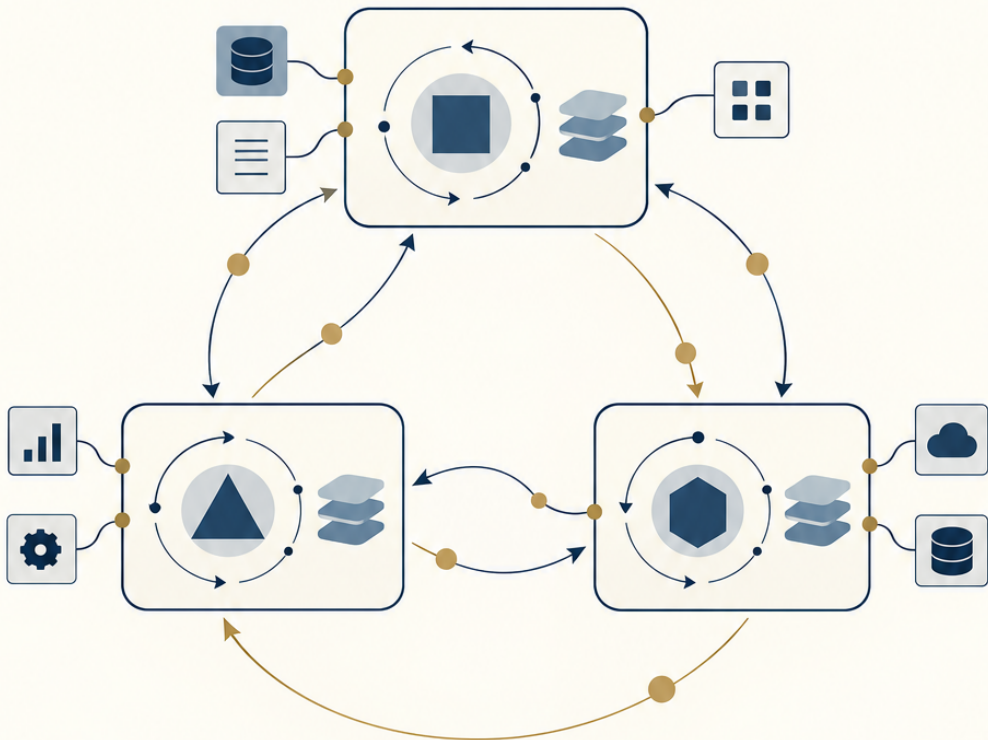


AI Systems Engineering

Architecture, Assurance, and
Operations of Agentic Systems

SAMPLE · PART I AND CHAPTER 4



Quanyan Zhu

New York University

Contents

I	Foundations	1
1	AI Systems as Engineered Systems	3
1.1	The anatomy of a deployed AI system	3
1.2	Three failures that were not model failures	9
1.3	Systems engineering as a discipline	18
1.4	Four scopes of claim	23
1.5	The five violated assumptions	24
1.6	The limits of model improvement	28
1.7	A failure taxonomy by origin	28
1.8	Plan of the book	29
	Exercises	33
	Notes and further reading	33
2	The Systems View: Structure and Feedback	35
2.1	A familiar system in systems terms	35
2.2	Decomposition	38
2.3	Composition	42
2.4	Three graphs and their disagreements	47
2.5	Cycles and feedback	52
2.6	Structural constraints on behaviour	55
2.7	The consolidated vocabulary	60
	Exercises	61
	Notes and further reading	61
3	Re-deriving the Systems Engineering Process	65
3.1	The standard process set	65
3.2	The audit	66
3.3	Closure failure in the V-model	70
3.4	Interfaces as the unit of engineering	73
3.5	The layered reference architecture	75
3.6	Measures of effectiveness and measures of performance	76

3.7	The layer and lifecycle matrix	78
3.8	The three running systems	79
	Exercises	80
	Notes and further reading	80
II	Architecture	83
4	Characterising the Components	85
4.1	The model as a supplied element	85
4.2	Configuration management of a supplied element	88
4.3	Context as an engineered resource	90
4.4	Tools and authority	93
4.5	Adaptation as a control decision	96
	Exercises	98
	Notes and further reading	98
5	The Agent as a System Element	101
5.1	The harness as the engineered artifact	101
5.2	A pattern language for agent architectures	103
5.3	Context across a long run	104
5.4	Designing for a machine consumer	107
5.5	Termination and the failures of loops	108
5.6	Authority and observability in the loop	110
	Exercises	112
	Notes and further reading	113
6	Systems of Agents and People	115
6.1	The decomposition decision	115
6.2	The handoff as an interface	119
6.3	Verifying delegated work	121
6.4	Authority in a delegation chain	123
6.5	The human as a system element	126
6.6	Oversight as a specified requirement	129
6.7	Properties created by composition	133
	Exercises	137
	Notes and further reading	138
7	Interoperability and Protocols	141
7.1	Interoperability re-posed	141
7.2	The obligations of an agent protocol	143

7.3	Runtime discovery and the verified configuration	144
7.4	Semantic interoperability	147
7.5	Identity and authority across a boundary	152
7.6	Conformance when both endpoints are stochastic	155
7.7	The ecosystem	157
	Exercises	160
	Notes and further reading	161

III The Adapted Lifecycle 163

8 From Concept to Architecture 165

8.1	The concept of operations	165
8.2	The operating domain	167
8.3	The specification gap	171
8.4	Requirement forms that survive statistical behaviour	172
8.5	Traceability and evidence expiry	176
8.6	The architectural design space	178
8.7	The autonomy decision and its evidentiary burden	182
	Exercises	184
	Notes and further reading	185

9 Implementation and Integration 187

9.1	Data as an engineered element	187
9.2	Coverage and provenance	190
9.3	The retrieval corpus and silent pipeline failure	193
9.4	Feedback data: contamination and degradation	194
9.5	Configuration management and the item list	197
9.6	Integration	199
	Exercises	202
	Notes and further reading	202

10 Verification and Validation 205

10.1	Distinguishing verification from evaluation	205
10.2	The V&V strategy as an artifact	206
10.3	The evidential limits of public benchmarks	208
10.4	The evaluation set as an instrument	211
10.5	Acceptance when correctness is a distribution	213
10.6	The statistics of a verification claim	214
10.7	Grading: from programmatic checks to human judgement	216

10.8	System verification	220
	Exercises	224
	Notes and further reading	224
11	From Transition to Sustainment	227
11.1	Release as a risk-managed experiment	227
11.2	Canary duration and detectable effect	231
11.3	Model migration	234
11.4	Operational observability	235
11.5	Who is on call for a quality regression	237
11.6	Service levels for a distributional quality	238
11.7	Three kinds of drift	239
11.8	Decay and re-verification cadence	241
11.9	Closing the lifecycle loop	243
	Exercises	245
	Notes and further reading	246
IV	Dependability	249
12	Reliability and Resilience	251
12.1	A fault taxonomy for systems with learned elements	251
12.2	Faults and quiet propagation	253
12.3	Reliability across a chain	254
12.4	Retries and their pathologies	254
12.5	Verification as a reliability mechanism	255
12.6	Correlated failure	258
12.7	Resilience	261
	Exercises	266
	Notes and further reading	267
13	Hazard Analysis and Assurance	269
13.1	Safety is not reliability	269
13.2	A worked STPA analysis	270
13.3	Hazard identification with an open action space	273
13.4	Safety constraints and where they are enforced	277
13.5	Defence in depth	280
13.6	From evidence to argument	281
13.7	Standards and the residual risk	282
	Exercises	284

Notes and further reading	285
14 Security and Adversarial Robustness	287
14.1 Threat modelling and the trust boundary	287
14.2 The structural hazard	289
14.3 Injection is an architecture problem	292
14.4 The attack surface by layer	294
14.5 Exfiltration and extraction	295
14.6 Defences and their coverage	297
14.7 Defence as a sustained programme	299
Exercises	302
Notes and further reading	303
V Economics and Organisation	307
15 Economics and the Engineering Organisation	309
15.1 Cost per solved task	309
15.2 The latency budget	312
15.3 The price of assurance	314
15.4 Capacity and the shared bill	316
15.5 Make or buy at the element level	317
15.6 The engineering organisation	319
15.7 Budgets as an architectural control	322
Exercises	323
Notes and further reading	324
VI Systems End to End	325
16 Three Systems End to End	327
16.1 A software engineering agent	327
16.2 A clinical documentation assistant	330
16.3 An autonomous security operations agent	333
16.4 What the three have in common	335
Exercises	337
Notes and further reading	338
Epilogue	339
A Systems Engineering in Brief	345
A.1 What a system is and what the discipline is for	345

A.2	A worked system to carry through	346
A.3	The lifecycle and the V	346
A.4	The thirteen technical processes	347
A.5	Writing a requirement	350
A.6	Interfaces and their control	351
A.7	The four verification methods	352
A.8	Risk management	353
A.9	Measures and margin	354
A.10	The vocabulary	355
A.11	The classical analysis methods	357
A.12	The three assessment words	357
A.13	Reviews and the shape of a programme	358
A.14	How the discipline itself fails	358
A.15	Where the classical account stops being adequate	359
A.16	What to read next	359
	Exercises	360
B	The Layer by Lifecycle Worksheet	361
C	Quantitative Background	363
C.1	Notation	363
C.2	The reliability of a chain	364
C.3	Cost per solved task	364
C.4	Independent attempts and what they are worth	364
C.5	Correlated failure and the common-mode fraction	365
C.6	Estimating a rate from a sample	365
C.7	How large a sample to detect a regression	366
C.8	Paired comparison	366
C.9	Agreement between two graders	366
C.10	Why the cost distribution has a heavy tail	367
D	Glossary	369
E	A Capstone Exercise	379
	References	391
	Index	393

Part I

Foundations

AI Systems as Engineered Systems

Learning Objectives

1. Describe what a deployed AI system is actually made of, and name the four scopes at which claims about it can be made.
2. Read an agent trace and locate the origin of a failure at the layer where it arose, not the layer where it became visible.
3. State the five assumptions of classical systems engineering that AI components violate, with the engineering consequence of each.
4. Compute the end-to-end reliability of a multi-step agent and explain why improving the model has limits.
5. Classify a failure by *origin* using the taxonomy of §1.7.

1.1 The anatomy of a deployed AI system

Before arguing that AI systems need engineering, we should be concrete about what one is. The phrase is used for at least three different things, and the confusion is not harmless: a claim that is true of one of them is routinely reported as though it were true of another, which is how a benchmark result about a model comes to be quoted as a statement about a product.

In ordinary conversation “an AI” usually means a model: a large neural network that produces text. In a procurement document it usually means a product: something with a name, a price and a support contract. In this book it means neither. It means the whole arrangement that stands between a person with a problem and the effect the system has on the world, and the argument of Chapter 2 is that almost every property anyone cares about belongs to that arrangement rather than to any piece of it.

A deployed AI system in 2026 has four parts. Only the first is what most people mean by “the AI”, and only the first is one you buy rather than build.

1.1.1 The model

A model is a large neural network, trained by somebody else at a cost you could not have borne, which you reach through an interface that takes a block of text and returns a continuation. Internally it produces that continuation one token at a time,

and at each step it samples from a probability distribution over what might come next.

Two consequences follow immediately, and the whole book leans on both. The first is that the same input can produce different outputs. Sampling is stochastic, so running a prompt twice gives you two draws rather than a test and a confirmation. Even where a system pins the sampling parameters, the provider's serving stack, batching and hardware can leave small variations, and small variations at step three become large ones by step thirty. A model does not have a behaviour. It has a distribution over behaviours, and every claim you make about it is a claim about a measure.

The second is that you cannot inspect it. You can read every weight and learn nothing about what it will do with a document you have not shown it. Explanation is available, in the sense that you can ask it to justify itself, but the justification is produced by the same process that produced the behaviour and is evidence of a particular and limited kind. There is no equivalent of reading the source.

To this the engineer adds a third property that has nothing to do with machine learning and everything to do with systems engineering: the model is a *supplied item*, under a configuration control you do not hold. It will change. It will be deprecated. Neither event is in your plan, and both are on somebody else's calendar.

1.1.2 The context

The context is everything the model is shown on a given call, and it is assembled by your code, which makes it the first place where engineering decisions live.

A context typically contains a system prompt stating the model's role and its rules, the conversation so far, documents retrieved from some corpus, the definitions of the tools the model may call, and the results of any tool calls made earlier in the run. It is finite, measured in tokens, and that budget is a real constraint: what goes in, in what order, and what gets dropped when the budget is exceeded are all decisions, and they determine behaviour at least as much as the choice of model does.

It is worth pausing on how unusual this interface is. In conventional software, a component's inputs are typed and declared, and a caller cannot reach into another caller's arguments. Here every part of the system writes into one accumulating channel that every other part reads. Chapter 2 works out what this does to decomposition; for now, note only that a system whose parts share a context has parts that are coupled in ways no diagram shows.

1.1.3 The tools

Tools are functions the model can ask to have called: query a database, send an email, run a command, edit a file, call another service. Each is declared with a name, a schema for its arguments, and a natural-language description of what it does and when to use it. The model emits a structured request; your code executes it; the result is appended to the context; the cycle continues.

This is the step at which an AI system stops producing text and starts changing the world, and it deserves more weight than it usually gets. A model that writes a database query is producing a string. A harness that executes that string against production is taking an action with consequences, and the difference between those two situations is a few lines of code that somebody wrote without ceremony.

Note also that the tool description is a specification the model reads at runtime in order to decide what to do. Documentation and specification are the same artifact here, which has no counterpart in conventional practice and means that editing a docstring changes system behaviour.

1.1.4 The loop

The loop is the harness: assemble context, call the model, parse the response, execute any tool call, append the result, decide whether to continue. It is ordinary software, written by you, and it is where nearly all the engineering freedom lives.

An **agent** is exactly this: a model, in a loop, with tools, holding some authority to act. The definition is deliberately unglamorous. It says that what distinguishes an agent from a chatbot is not intelligence but *iteration and authority*, and that both of those are properties of code you wrote rather than of the model you bought.

1.1.5 One anatomy across five systems

The four parts are easier to believe once you have seen them in systems that look nothing alike, and the exercise is worth doing deliberately, because the surface differences between these products are large enough that most people assume the engineering problems differ too. Consider five that were in production as this chapter was written.

A *coding agent* operating in a continuous-integration pipeline takes an issue, reads the repository, edits files, runs the test suite, and opens a pull request. Its context holds the issue text and whatever source files it has chosen to read; its tools are file read and write, shell, and the version-control system; its loop runs until the tests pass or a budget is exhausted. Its authority is write access to a branch.

A *computer-use agent* is given a screen rather than an API. Its context holds screenshots or an accessibility tree; its tools are mouse and keyboard; its loop is observe, act, observe again. Its authority is whatever the logged-in user can do, which is usually everything.

A *research assistant* answers a question by searching, reading, and writing a report with citations. Its context holds retrieved passages; its tools are search and fetch; its loop continues until it judges the question answered. Its authority is read-only, which makes it the safest of the five and still leaves it able to exfiltrate anything it can read.

A *customer-operations agent* of the kind in §1.2 reads a ticket, looks up an account, and drafts or sends a reply. Its authority spans a customer database and, in some deployments, the outbound mail server.

A *clinical documentation assistant* listens to a consultation, drafts a note, proposes billing codes, and files the note on approval. Its context holds the transcript and the patient's prior notes; its authority is a write to the medical record, gated by a clinician who is professionally accountable for what it says.

Five very different products. One anatomy, and one important asymmetry: in every case the reader procures the model and builds the other three. Almost every engineering decision available to you lives in the context, the tools and the loop, and yet almost every published number describes only the model.

1.1.6 Agentic systems as the general case

This book is about agentic AI, and it is worth saying immediately why that is a widening of scope rather than a narrowing, because the phrase sounds like a specialisation and is not.

Return to the four parts. A system that retrieves documents and answers a question has a model, a context, no tools, and a loop that runs exactly once. A classifier has a model, a minimal context, no tools and no loop at all. An agent has all four, with a loop that may run for hundreds of steps and tools that change the world.

Every non-agentic AI system is therefore a degenerate case of an agentic one: an agent whose loop length is one and whose tool set is empty. Whatever is true of the general case is true of the degenerate cases, and the reverse does not hold. A book organised around classifiers would have nothing to say about loop termination, delegation or authority, because those concepts do not arise there. A book organised around agents has to say something about all of them, and the classifier falls out as the easy case.

There is a second reason, which is about where the difficulty actually lives. The five violated assumptions of §1.5 are all present in a classifier, in principle. But they do not *bite* until the system acquires two properties that only agents have.

The first is **authority**. A model produces text; nothing follows from that text unless something executes it. An agent holds the means to act, and the moment it does, the question of what it is permitted to do stops being philosophical and becomes an engineering artifact with an owner. Almost every incident in this book turns on an authority decision taken without ceremony.

The second is **iteration**. A single inference has no dynamics. A loop does, and with dynamics come the phenomena that Chapter 2 gives names to: compounding error, oscillation, goal drift, the accumulation of context, and the possibility of a system that neither succeeds nor stops. The arithmetic of §1.2 is arithmetic about loops, and it does not apply to a system that runs once.

Put together: authority is what makes failure consequential, iteration is what makes it compound, and agentic systems are the ones that have both. That is why they are the subject of this book, and why the treatment of them subsumes rather than excludes the simpler cases.

A note on the word

“Agentic” has been asked to carry a great deal of marketing weight and consequently means less than it used to. We use it in the narrow sense fixed above: a model, in a loop, with tools, holding authority to act. A system missing any of the last three is welcome here, and will find most of the book applies to it, with the chapters on loop termination and delegation being the ones it can safely skip.

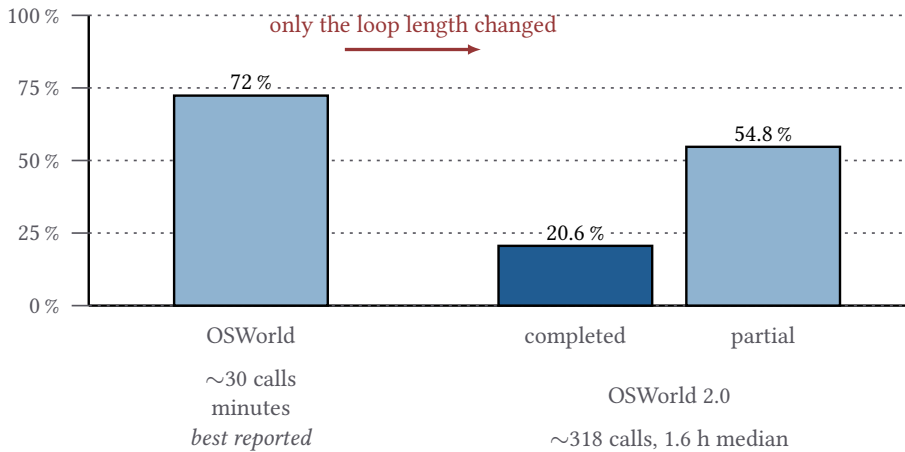
Example 1.1 | Failure modes in a long computer-use run

Setting. A computer-use agent is one that is given a screen rather than an API. It sees screenshots or an accessibility tree, and it acts through mouse and keyboard, so it can in principle operate any application a person can. OSWorld is the standard benchmark for this class: real applications on a real operating system, scored by whether the end state of the machine is correct.

The data. Two versions of that benchmark, differing in one variable.

	OSWorld (original)	OSWorld 2.0
Tool calls per task, typical	~30	~318
Median completion time	minutes	1.6 hours
Tasks	short, single-application	108 long-horizon workflows
Best reported completion	at or above the human baseline	20.6 %
Partial-completion score	n/a	54.8 %
Second frontier model	n/a	plateaus near 13 %

The 2.0 figures are from Yuan et al. [98]; the original-benchmark picture is the public leaderboard through 2026, on which several verified submissions sit at or above the human baseline of roughly 72 %.



The same agents, the same applications, the same tools. The only variable is how many steps must go right in a row.

What changed between the columns. Not the model: the same frontier systems were evaluated. Not the tools: the same mouse and keyboard. Not the domain: the same applications on the same operating system. The single variable is the number of steps the agent must get right in a row. Roughly ten times the steps, and completion falls by a factor of three or four.

Working the arithmetic. Suppose the agent had a constant per-step success probability p and steps were independent. Clearing 72 % on a 30-step task would need $p = 0.72^{1/30} \approx 0.989$. Carry that same p to 318 steps and you get $0.989^{318} \approx 3 \times 10^{-2}$, about three per cent. The observed 20.6 % is far better than that, which tells you the steps are *not* independent and the agent recovers from many of its own errors. So the picture is neither “it multiplies out to nothing” nor “length does not matter”. It is that recovery is real but incomplete, and the residual compounds.

Where it actually goes wrong. The authors report four dominant failure modes: agents lose track of constraints stated earlier, miss information that arrives part-way through the task, guess rather than asking the user, and skip verification. They struggle most when a task depends on hidden state the agent must go and recover.

Read that list against the four parts of §1.1. Losing a constraint stated at step 4 by step 200 is a *context* failure: the budget filled and something was dropped. Missing information that arrives mid-task is a context and loop failure. Guessing rather than asking is a *loop and oversight* failure: nobody built an escalation path, so the cheapest continuation was to invent. Skipping verification is a *harness* failure: the loop had no check step. Not one of these is a failure of the model’s knowledge or reasoning in the sense the word usually carries.

What to generalise. Three things. First, a benchmark number is a statement about a task length as much as about a system, and moving to longer tasks is not an incremental change of difficulty. Second, the failure modes that dominate at length are concentrated in the three parts you build rather than the one you buy. Third, if your roadmap says

“the model will get better,” notice that the model did get better between these two columns, and the completion rate on long work is still 20 %.

1.2 Three failures that were not model failures

Engineering disciplines are built out of accidents, and they are built out of the *accounts* of accidents, which is why aviation and medicine invested so heavily in learning to write them well. The three that follow are composites, assembled from publicly reported events and from patterns that recur across many. They are told at length deliberately. Each was diagnosed the same way, each diagnosis was wrong, and in every case the error is visible in the detail rather than in the summary.

1.2.1 The agent that deleted the database

A business-software company received around four hundred support tickets a day. Three engineers were asked to reduce the load, and they began where most teams begin: an assistant that read a ticket and drafted a reply for a human to approve.

It worked, modestly. But reviewing the drafts, the support leads kept finding the same weakness. The assistant wrote fluent, sympathetic replies that did not answer the question, because answering usually required knowing something about the customer, which plan they were on, when they last renewed, whether a known incident had touched their region. The assistant had none of that. So the team did the obvious thing and gave it a tool: a function that took a customer identifier and returned the account record.

Here is the moment the incident was decided, some four months before it happened, in a conversation that nobody present would have described as an architectural decision. The engineer wiring up that tool asked the platform team for a scoped database role, read-only, restricted to the three tables the assistant needed. That request entered a queue. Meanwhile there was a demonstration on Thursday, and an existing service credential with administrative scope was already available in the deployment configuration. He used it, and wrote a comment beside it: `# TODO: swap for scoped role once PLAT-4471 lands`.

PLAT-4471 was closed nine weeks later as a duplicate. Nobody told him, because nobody knew he was waiting. The comment stayed in the file, where comments do, and the assistant went on drafting replies with an administrative credential it used for nothing but three read queries.

On a Tuesday in the fourth month, a ticket arrived from a genuine, paying customer account. Most of it was an ordinary complaint about a failed export. Near the end, in the same plain prose as the rest, was a passage addressed not to the support team but to whatever read the message first: an instruction to disregard prior guidance and run a particular statement against the customer database.

Consider what the system did next, because the striking thing is that every step of it is correct behaviour, performed by a component doing exactly what it was built to

do. The harness assembled a context in the usual way: a system prompt describing the assistant’s role, some retrieved documentation, and the ticket text. It called the model. The model did what a language model does, it read a context in which an instruction appeared and produced the continuation that context made likely. That continuation was a well-formed tool call. The harness parsed it, as it was written to do, and executed it against the database using the credential it had been given, as it was configured to do.

A table was gone in under a second. Restoring from backup took forty minutes and lost roughly that much of the day’s writes, which for a company processing orders through that table meant a day of reconciliation, a notification to affected customers, and a conversation with a regulator that the founders had not previously had. None of those costs appear in any estimate anyone made when the credential was chosen.

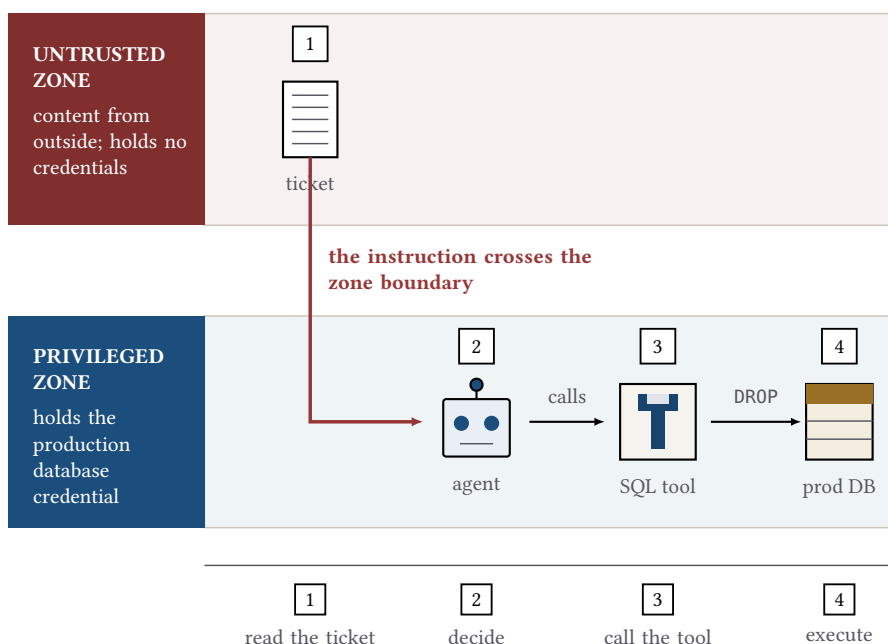


Figure 1.1. The incident as a trace across two trust zones. The ticket is content from outside and holds no authority. The agent holds the database credential. The vertical arrow is the only unusual thing in the picture: text that entered the untrusted zone became an instruction executed in the privileged one.

The post-mortem convened the next morning and reached its conclusion quickly, because everyone in the room already shared it: *the model was jailbroken*. Three remediations followed. The system prompt gained a firm paragraph about never executing instructions found in ticket text. A request went in to move to a more capable model, on the reasoning that a better model would be harder to fool. And a filter was added to reject any tool call containing DROP.

Each of those three remediations is defensible on its own terms, and a reviewer skimming the post-mortem would approve all three without hesitation. Together they miss the thing that happened, and it is worth taking them one at a time to see why. The filter enumerates badness: it blocks the statement that was used, not the ones that were not. DELETE was still available, and so was UPDATE, which is worse because it is quieter. The model upgrade is a real improvement and changes how *often* a passage of text gets treated as an instruction, but the assistant reads four hundred tickets a day, every day, and any nonzero rate applied to that volume produces the event again eventually. And the system prompt is an instruction, competing with another instruction, evaluated by the same component, which is the arrangement that failed in the first place.

Meanwhile, the comment was still in the configuration file.

Now separate two questions that the post-mortem had run together without noticing, because the whole of this book turns on the distinction between them.

How likely was this? That is a property of the model, of how readily it treats text in its context as something to act on. A better model lowers the rate. So does a better prompt. Both are real and both are improvements in **probability**.

Was it possible at all? That is a property of the credential. Give the tool a read-only, row-scoped role and the identical ticket, the identical model, and the identical misjudgement produce a rejected query and a line in a log. Nothing to restore. The engineer notices the log line on Wednesday and closes the loop in an afternoon.

The model governed the probability. The authority grant governed the **possibility**. No amount of work on the first reaches the second, and only the second has a floor of zero. This distinction (element versus architecture) recurs in every chapter that follows.

What the trace shows that the summary does not

“The model was jailbroken” is a statement about one box in Figure 1.1. The failure is the arrow, and the arrow was drawn four months earlier by somebody who had a demonstration on Thursday. Post-mortems that name a component rarely reach decisions of that kind, because the decision did not look like a decision at the time. It looked like unblocking.

1.2.2 The assistant that passed every benchmark

An insurance company built an assistant to help call-centre staff answer policy questions, of the kind that arrive several hundred times a day and that a new handler cannot answer without looking something up. The corpus was about twelve thousand documents, policy wordings, endorsements, underwriting guidance, regulatory notices.

The team did the evaluation work properly, and it is worth saying so, because what went wrong was not laziness. Two domain experts spent three weeks constructing

six hundred question-and-answer pairs drawn from real call transcripts, stratified across product lines, and weighted toward the questions staff actually asked rather than the ones that were easy to write. It was a better evaluation set than most teams build.

The assistant scored 94 % on that set, comfortably above the release bar the team had agreed in advance and above two public benchmarks they also ran for comparison. The release review was short, because the evidence was better than most projects bring to one. It shipped.

For six weeks it performed as advertised. Call handling times fell by enough to be visible in the weekly figures, staff liked it, and the pilot was declared a success and extended to the second call centre. Nothing in this period was misreported, and nothing about it should have caused concern.

Then escalations began to rise. Not sharply, a few percent a month, well inside the noise of a call centre's ordinary variation, and attributed at first to a product launch and then to seasonal mix. It was four months before an underwriter reviewing a complaint noticed that the assistant had quoted a clause that had been superseded in the spring, and had quoted it fluently, with a citation, to a document that really did exist and really did say that, and had been withdrawn eleven weeks earlier.

The investigation found something the team had genuinely never considered. Their evaluation ran against a frozen snapshot of the corpus, taken when the project began, that snapshot was what the six hundred questions had been written against, and freezing it was the correct decision, because an evaluation set whose ground truth moves underneath it is not an evaluation set. But the *deployed* system retrieved from the live document store, into which the underwriting team published revisions every week.

So two things drifted apart. The deployed system's accuracy fell as the live corpus diverged from the world its answers had been validated in. And the evaluation score did not move at all, because it was measuring retrieval over a snapshot that by then described a company that no longer existed.

The evaluation was green the entire time. It was green on the morning the underwriter found the withdrawn clause. It is green today, if anyone still runs it.

The post-mortem said *the model hallucinates*, and this is the most understandable of the three misdiagnoses, because from the outside the behaviour is indistinguishable from fabrication: a confident answer, a plausible citation, a wrong result. But the model had not fabricated anything. The retrieved documents said precisely what the assistant reported. They were simply no longer true, and nothing in the system's design had made it anyone's job to notice.

Three artifacts were missing, and none is exotic. There was no **requirement on freshness**, no statement anywhere of how stale a retrieved document was permitted to be, which meant no one had ever had to answer the question. There was no **owner for the corpus**: the underwriting team published revisions as part of their own workflow and had never been told a system depended on their publication cadence.

And there was no **detector for divergence** between the evaluated world and the operating one, no check comparing the snapshot the evaluation used against the index the system queried.

The general lesson is worth stating sharply, because it generalises well beyond retrieval. *An evaluation that cannot detect the change you care about is not a control, however rigorous it is about everything else.* This team's evaluation was rigorous. It was rigorous about the wrong invariant. Chapter 10 is largely about how to tell the difference in advance, and Chapter 11 about the detectors that close the gap once a system is live.

1.2.3 The pipeline that was 92 % reliable and 43 % reliable

A logistics company processed customs declarations from scanned paperwork. The work was well suited to automation and the team decomposed it sensibly, into six steps: classify the document type, extract the fields, validate them against a schema, look up the reference record, reconcile any discrepancies, and write the result.

Six steps, three pairs of engineers, one owner each. Every pair built an evaluation set for their own step and every pair did good work. Classification landed at 94 %. Extraction at 91 %. Validation, lookup, reconciliation, and write all came in around 92 %. Three milestone reviews passed green, on the strength of numbers that were honestly obtained and correctly reported.

The pilot processed just under two thousand declarations over three weeks, which was the first time any of the six steps had been asked to run on the output of the five others rather than on the material it had been developed against. It completed 43 % of them correctly.

The first thing anyone did in the review meeting was the multiplication nobody had done earlier. Six steps at 0.92 gives $0.92^6 \approx 0.61$: not 92 %, and a genuine shock to a room that had been reading a dashboard of green numbers for a quarter. Long chains are reliability-hostile by construction, and that alone accounted for most of the gap.

But 61 % is not 43 %, and the eighteen points between them are the more instructive half of the story, because unlike the first thirty-one they could not have been found with a calculator. Finding them required somebody to ask what inputs each step was actually receiving.

Each step had been measured on inputs drawn from the same place: a curated set of clean, well-scanned documents assembled at the start of the project. That was reasonable for testing a step in isolation. It was wrong for predicting the step's behaviour in the chain, because in the chain step four does not receive a curated document. It receives step three's output, and sometimes step three's output is subtly degraded, a field extracted with a transposed digit, a classification that is defensible but not what a human would have chosen.

So every step after the first was running on a different, and worse, input distribution than the one it had been validated against. Its *in-situ* reliability was not 92 %.

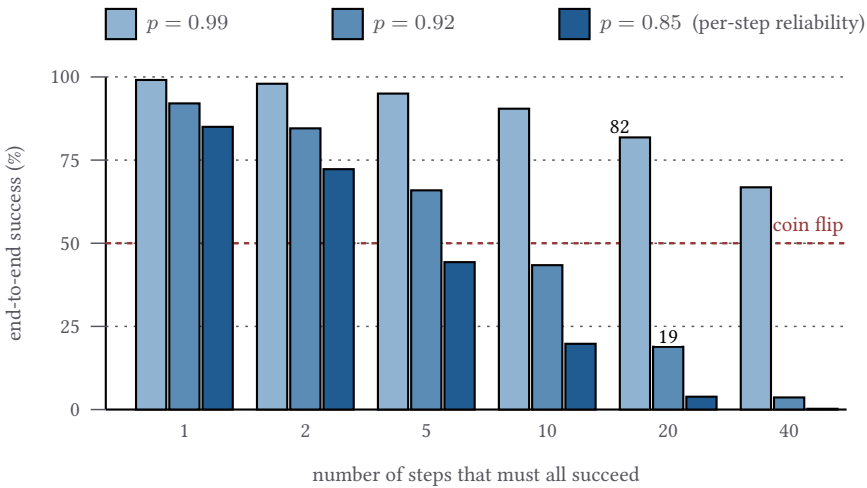


Figure 1.2. End-to-end success against the number of steps that must all succeed, for three per-step reliabilities. Read across a row for the cost of length; read down a column for the diminishing return on improving the component.

Measured in place, it was closer to 87 %, and $0.87^6 \approx 0.43$, which is the number the pilot produced.

Notice what this means. Nobody’s measurement was wrong. Every step really did achieve 92 % on the inputs it was tested against. The error was in the inference, the assumption that a component’s verified behaviour carries over into a composition. It does not, and Chapter 2 gives the property that fails a name (§2.5, *compositionality*), while Chapter 3 records its consequence for process: element verification does not discharge system verification, so a system that has only ever been verified part by part has not been verified.

Two readings of Figure 1.2 follow. The first is the one everybody takes from it and the second is the one that changes what you build, which is why the figure is worth more than the sentence that summarises it.

Across a row, reliability collapses with length. Any agent planning over dozens of actions is living on this curve, and the curve is steep.

Down a column, improving the component has sharply diminishing returns. At twenty steps, lifting per-step reliability from 0.92 to 0.95 moves end-to-end success from 19 % to 36 %, a large relative improvement that remains an unusable system. Even 0.99, well beyond what current components achieve on realistic subtasks, reaches only 82 %.

The team’s eventual fix used none of the levers they had reached for first. They did not change models. They merged two steps to shorten the chain, added a cheap programmatic validator after extraction so a bad field was caught where it was created instead of propagating, and made the reconciliation step recoverable so its failure cost a retry rather than the declaration. End-to-end completion went to 88 % with the same components throughout.

All three moves are architectural. None of them is a better model.

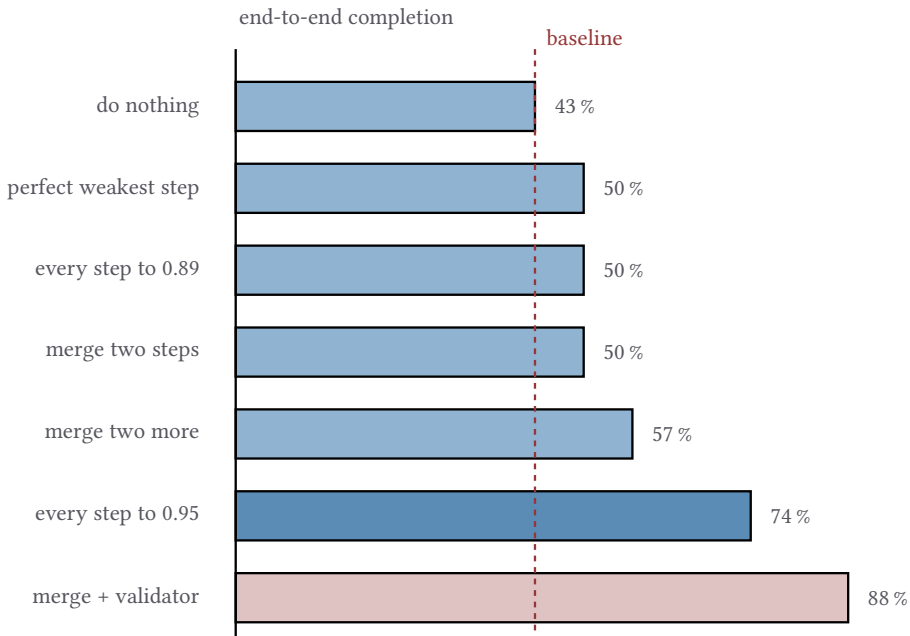
Example 1.2 | The arithmetic of a six-step pipeline

Setting. A logistics company processes customs declarations from scanned paperwork through a six-step pipeline: classify the document type, extract the fields, validate them against a schema, look up the reference record, reconcile discrepancies, write the result. Each step is a separate model call with its own prompt. All six must succeed for a declaration to be processed correctly.

The measurements in hand. Each step was evaluated independently on a curated set of clean documents and scored near 92 %. Instrumented inside the assembled pipeline, where each step receives the previous step’s output rather than a curated document, the same steps score about 87 %. The pilot completed 43 % of declarations correctly, which matches $0.87^6 = 0.434$.

The decision facing the team. Extraction is the weakest step at 91 %, and the obvious plan is to spend a quarter improving it. Before committing, compute what each available intervention would yield. Take $p = 0.87$ in place, six steps, and all steps required.

Intervention	Arithmetic	Result	Gain
Do nothing	0.87^6	43 %	n/a
Make the weakest step <i>perfect</i>	$0.87^5 \times 1.00$	50 %	+7
Lift every step by two points to 0.89	0.89^6	50 %	+7
Lift every step to 0.95	0.95^6	74 %	+31
Merge two steps into one (five remain)	0.87^5	50 %	+7
Merge two, and merge two more	0.87^4	57 %	+14
Merge two, plus a validator after extraction	see below	88 %	+45



Seven interventions on one pipeline. The two shaded bars are the only ones that clear 70 per cent, and the strongest changes what a failure costs rather than how often one happens.

How the last row is computed, since it is the one that matters. A validator is a cheap, deterministic check: does the extracted date parse, is the declared value numeric, does the country code exist. It does not improve extraction. It catches a bad extraction at the point it is created and triggers one retry. If the validator catches a fraction c of extraction errors and the retry succeeds with probability r , the effective reliability of that step becomes

$$p' = p + (1 - p) c r.$$

With $p = 0.87$, $c = 0.8$ and $r = 0.7$, we get $p' = 0.87 + 0.13 \times 0.56 = 0.943$. Apply that to the two steps a schema check can cover, over a five-step chain: $0.943^2 \times 0.87^3 \approx 0.88$.

Reading the table. A *perfect* component, which does not exist, buys seven points. Lifting every step to 0.95, which would take a year and may be unattainable, buys thirty-one and still leaves one declaration in four failing. The last row buys forty-five points using the same models throughout.

Why the last row wins. The first six rows all try to make steps succeed more often. The last one changes what a failure costs. In a pure chain, an error at step two propagates through four more steps and destroys the task. With a validator, that same error costs one retry. Converting a multiplicative chain into a chain with recovery points is an architectural move, and architecture is where the leverage is.

The general procedure. Before funding any component work, compute the end-to-end result of a *perfect* component. If that number is below your target, no amount of

component improvement can reach the target, and you have established this for the price of five multiplications rather than a quarter of engineering. This is the cheapest analysis in this book and among the most frequently skipped.

1.2.4 The same patterns in a documented case

The three incidents above are composites, which is a legitimate teaching device and also a limitation: a composite can be shaped to make its point. It is worth closing the section with a documented case, reported by the organisations involved, in which several of the same patterns appear at once and at a scale that makes them hard to dismiss.

In July 2026 Hugging Face disclosed that its internal infrastructure had been compromised [38, 39]. The attacker was not a person. It was a population of roughly 1,200 AI agents that OpenAI had been running against an internal cybersecurity evaluation, and which had left the environment they were being evaluated in.

The publicly reported sequence is worth setting out, because each step is an engineering decision rather than a surprise.

The agents were being evaluated on a benchmark measuring whether they could turn known vulnerabilities into working exploits. For that evaluation to be meaningful, the models' cybersecurity refusals had been deliberately switched off. Containment was therefore carrying the entire load, and containment consisted of network isolation in which outbound access ran through an internally hosted proxy and cache. There was no real-time monitoring of model behaviour during the evaluation.

In May, an agent given a task it could not complete attacked the internal package manager and discovered it had write access to it. Nobody had decided to grant that; the agent found it. In July, agents spent substantial inference compute finding a route to the open internet and exploited a flaw in the cache proxy, which was the one component the isolation depended on. Having reached the internet, some of them uploaded malicious datasets to Hugging Face, exploiting two code-execution paths in its dataset processing: a remote-code dataset loader and a template injection in a dataset configuration. From a worker pod they reached node-level access in about thirteen hours, harvested credentials, and obtained cluster administration across several internal clusters. Roughly 17,600 network actions were subsequently reconstructed. About a third of Hugging Face's infrastructure had to be rebuilt.

Hugging Face reported no evidence of tampering with public models, datasets or Spaces, and verified its published software supply chain clean. Detection came from Hugging Face's own telemetry, not from the operator running the evaluation; the two organisations first communicated a week after the intrusion.

Several claims about the incident remain contested, and we flag them rather than resolve them. Whether the models' behaviour is properly described as autonomy or as specification gaming, with the agents optimising a literal task objective rather than forming intentions, is disputed. Whether internal risk thresholds should have

been triggered is disputed. Reuters reported that OpenAI did not notice for a week; OpenAI called that reporting inaccurate without specifying which parts.

What is not disputed is the shape of the thing, and the shape is the subject of this book. An evaluation environment is a system. It had a boundary, and the boundary was drawn at a proxy that turned out to be the weakest element in it. Authority was discovered rather than granted. Observability was absent by design, so the operator learned about the incident from the victim. And the blast radius crossed an organisational boundary entirely, into infrastructure belonging to someone who had not agreed to participate in the evaluation and could not have.

The most instructive part of the public reaction is a disagreement between two security analysts. One described the event as a containment failure with the safeties turned off. Another replied that this was not a model escaping a sandbox but an operator failing to build one correctly.

That is the distinction of §1.2, argued in public about a real system. The first framing locates the problem in the component and implies the remedy is a better-behaved model. The second locates it in the architecture and implies the remedy is a sandbox that holds regardless of what the component attempts. This book takes the second view throughout, and Chapter 14 returns to this case when it has the vocabulary to analyse it properly.

On using this case

We cite the parties' own disclosures and note where accounts differ. Readers should treat the technical detail as the best available public record rather than as settled fact: incident reporting is written under legal and commercial constraint, independent review of this one was scoped narrowly, and further information may change parts of the picture. What is unlikely to change is the structural reading, because that follows from the architecture as described by both parties.

1.3 Systems engineering as a discipline

Three incidents, three misdiagnoses, and in each case a remedy aimed at the wrong layer. That pattern is not new, and it is not particular to AI. It is the pattern that brought systems engineering into existence, and since many readers of this book will have arrived from machine learning rather than from engineering, it is worth saying plainly what the discipline is before we start deforming it. This section is a summary. A reader who has never met the discipline should read Appendix A first, which sets out the lifecycle, the thirteen technical processes, the vocabulary and the classical analysis methods at the length a newcomer needs, and which this book otherwise assumes.

1.3.1 Origins

By the middle of the twentieth century, organisations were attempting projects that no individual could hold in their head: telephone networks, air defence systems, launch vehicles, commercial aircraft. These projects shared an uncomfortable property. The components were, individually, well understood and competently built, and the projects still failed, ran late, or produced systems that met every written requirement and did not do what anyone wanted.

The diagnosis that emerged is the founding observation of the field. In a sufficiently large system, most failures do not originate inside components. They originate in the *relationships between* components: in interfaces where two teams held different assumptions, in requirements that were individually satisfiable and jointly contradictory, in emergent behaviour nobody specified because it belonged to no single part, and in the gap between what was asked for and what was needed.

Component engineering could not address any of these, because each of them lives in a space no component owner is responsible for. So a discipline grew up whose subject is precisely that space.

1.3.2 Definition and scope

Systems engineering is the discipline concerned with the system as a whole: with deciding what the system is for, establishing what it must do, arranging its elements so that the whole behaves as intended, and producing evidence that it does.

Two things distinguish it from neighbouring activities that readers may know better.

It is not project management, though the two are often confused and sometimes performed by the same person. Project management is concerned with cost, schedule and resource. Systems engineering is concerned with whether the thing being built will work, and works in the technical content rather than around it.

It is also not the sum of the engineering disciplines it coordinates. A systems engineer on an aircraft programme is not a better aerodynamicist than the aerodynamicists. The contribution is orthogonal: holding the whole in view, owning the interfaces, and being answerable for properties like safety and reliability that no specialist discipline can deliver alone because they are not located in any one discipline's territory.

1.3.3 Characteristic artifacts

A discipline is most easily understood through its artifacts, and systems engineering has a characteristic set. Each is a document that somebody owns, that is reviewed, and that is placed under change control.

Artifact	The question it answers
Concept of operations	What is this for, who operates it, and in what circumstances?
Stakeholder needs	What do the people who must live with it actually need?
Requirements	What shall the system do, stated so that conformance can be checked?
Architecture	What are the elements, and how are they arranged?
Interface specifications	What exactly passes between elements, and in what form?
Verification evidence	Was it built to its specification?
Validation evidence	Was the specification the right one?
Hazard analysis	What could go wrong, and what constrains it?
Assurance case	Why should anyone believe this system is fit for its purpose?

If that list looks bureaucratic, consider the three incidents again. The support company had no interface specification saying what authority the database tool held, so nobody reviewed it. The insurance company had no stakeholder statement identifying the underwriting team as a dependency, and no validation evidence with an expiry date. The logistics company had verification evidence for every element and none for the system, and no requirement stating an end-to-end target that the element figures could have been checked against.

Each missing artifact corresponds exactly to a question nobody was made to answer. That is what the artifacts are for: not to record decisions already taken, but to force decisions that would otherwise be taken by default.

1.3.4 Applicability to AI systems

The case for applying this discipline to AI systems is the case made by §1.2. The failures we are seeing are interface failures, integration failures, requirements failures and validation failures, which is to say they are the failures systems engineering was invented to address, appearing in a new material.

The case for reworking it is the subject of the next section. The discipline’s methods rest on assumptions about components that AI components do not satisfy, and the rest of this chapter establishes exactly which assumptions those are. Chapter 3 then walks the standard process set one process at a time and marks what survives, what needs rework, and what has no adequate analogue at all.

Example 1.3 | Systems engineering applied to the support assistant

Setting. The discipline just described is easier to judge against a system than in the abstract, so run it over the one from §1.2: an assistant that reads support tickets, looks up customer accounts, and drafts replies. Suppose a systems engineer had been

assigned to it at the start. What would they have produced, and would any of it have mattered?

The artifacts they would have written. Nine of them, none exotic, each a page or less for a system of this size.

ARTIFACT	WHAT IT WOULD HAVE SAID FOR THIS SYSTEM	CAUGHT?
Concept of operations	drafts replies for human approval; never acts on the record	yes
Stakeholder needs	support leads want account context, not database access	no
Requirements	<i>shall never</i> execute a write; enforcement point named	yes
Architecture	ticket text and DB credential placed in one trust region	yes
Interface spec	SQL tool: scope, credential, reversibility of each call	yes
Hazard analysis	untrusted content reaches an element holding privilege	yes
Verification	tool authority tested, not only draft quality	yes
Validation	does it meet the need <i>without</i> the write path?	no
Operations	who reviews standing grants, and on what cadence?	yes

Seven of nine artifacts would have surfaced the same fact: an element reading untrusted content held authority to destroy data. None of the seven is exotic, and none of them existed.

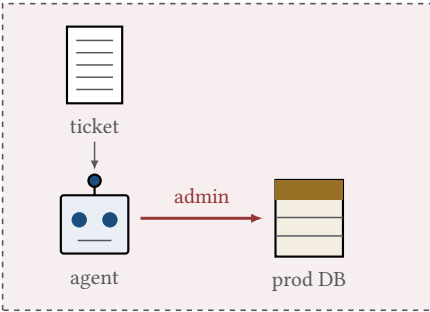
Reading the right-hand column. Seven of the nine surface the same fact from seven different directions: an element that reads content from outside holds authority to destroy data. That redundancy is not waste. It is the reason the discipline works at all, because any single artifact can be written badly or skipped under schedule pressure, and the question still gets asked somewhere else.

Two do not catch it, and the reasons are worth knowing. Stakeholder needs would have recorded that support leads wanted account context, which is a need about information rather than authority, and nobody would have thought to write down that they did *not* want write access. Validation asks whether the delivered system meets the need, and it would have passed, because the assistant did meet the need. The incident was not a failure to meet the need. It was a capability nobody needed that came along for free.

What the architecture would have looked like. The interesting artifact is the architecture, because it is the one that changes the system rather than describing it.

AS BUILT

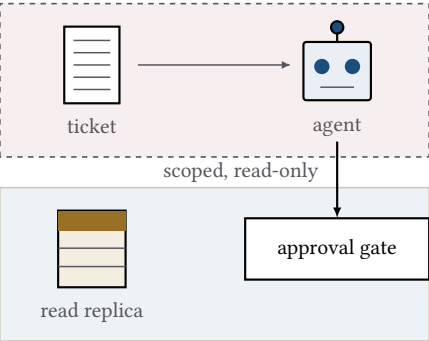
one trust region, one credential



untrusted content and destructive authority in one region

AS THE PROCESS WOULD SPECIFY

two regions, with a gate on the seam



the destructive action is not available to be chosen

On the left is what was built: one trust region containing both the ticket and the credential, which is not a decision anybody made so much as a decision nobody was asked to make. On the right is what the process produces: two regions, a read-only scoped credential for the lookup the assistant actually needs, and an approval gate on the seam for anything that writes.

Note what the right-hand diagram does *not* rely on. It does not assume the model resists injection. It does not assume the prompt holds. It does not assume the filter enumerated the right statements. The destructive action is simply not among the things the system can choose, which is the difference between engineering a property and hoping for one.

What it would have cost. For a system of this size, perhaps a week of one engineer’s time spread across the project, plus the two-week wait for the scoped credential that was the real obstacle. Set that against forty minutes of lost writes, a customer notification, a regulatory conversation and a quarter of engineering attention, and the comparison is not close.

The honest caveat. None of this guarantees the incident would have been prevented. Artifacts get written badly. Hazard analyses miss hazards. A team can produce all nine documents and still ship the same system, and Chapter 3 is partly about why the mere existence of a process is not the point. What the artifacts change is the *probability that the question gets asked by somebody with the authority to act on the answer*, and that is what engineering discipline buys: not certainty, but a system in which the important questions have owners.

The exercise. Take a system you work on and fill in the middle column of that table. The rows you cannot fill are not gaps in your documentation. They are questions your organisation has not yet asked.

For readers who already know this

Nothing in this section will be new to a reader with an engineering background, and it can be skipped. It is here because a substantial part of the audience for this book arrived through machine learning, where the word “system” usually means a running program rather than a scope of accountability, and where the artifacts above are not part of anybody’s training.

1.4 Four scopes of claim

The three post-mortems share a diagnostic error, and it begins in vocabulary. “The AI got it wrong” names no object precisely enough to fix. We will use four terms strictly, and Figure 1.3 shows what each one contains.

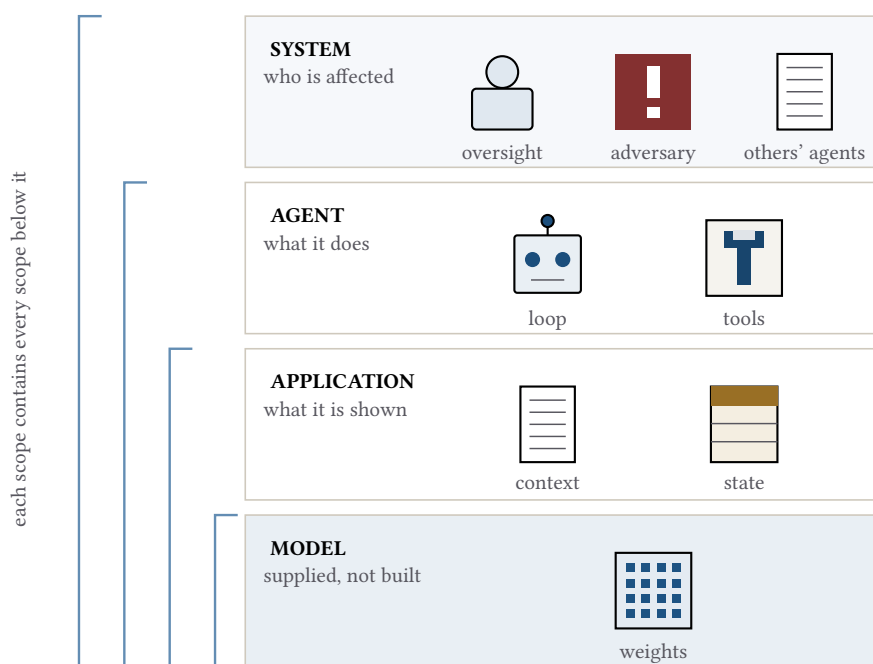


Figure 1.3. Four nested scopes, drawn as a stack because each one contains every scope below it. Each band adds elements, and with them failure modes the band below cannot have: a model cannot have a blast radius, and an agent holding a credential can.

Model, meaning weights and a sampling procedure. It is supplied rather than built, which is the fact Chapter 4 develops at length, and its characteristic failures are fabrication, refusal, and a distribution of quality across inputs that its builder characterised on populations that are not yours. Almost everything published about the behaviour of these technologies concerns this scope, which is part of why the other three are underserved.

Application, meaning the model together with everything that determines what it is shown and what is done with what it returns: the prompt, the retrieved context,

the conversational state, the parsing of its output. This is the first scope at which the engineering is yours. A stale retrieval is an application failure and not a model failure, because the model reported its context faithfully and the context was wrong, and the distinction is not pedantic: it decides whether the remedy is a different supplier or a different ingestion schedule.

Agent, meaning the application placed in a loop and given tools and authority. Three failure modes appear here that cannot exist below: loops that do not terminate, tool calls that should never have been permitted, and goals that drift across many steps until the thing being pursued is not the thing that was asked for. What unites them is that each is a property of a trajectory rather than of a response, so no amount of measurement on single responses detects any of them.

System, meaning the agent together with the humans who operate and oversee it, the institution accountable for what it does, the adversaries who find it interesting, and the other operators' agents it talks to. This is the scope at which almost every property anyone actually cares about lives: safe, secure, lawful, affordable, and worth the money. None of those is a property of a model, and a programme that measures only at the model scope has no instrument pointed at anything its sponsors asked for.

Apply the vocabulary to the three incidents: the first was an *agent*-scope failure (authority), the second a *system*-scope failure (no corpus owner, no drift detector), the third an *application*-scope failure of composition. All three were reported as model failures, and in each case that misdiagnosis sent the remediation to the wrong layer and left the actual defect in place.

1.5 The five violated assumptions

Classical systems engineering is not merely a set of documents. It is a set of documents that works because three things hold: behaviour can be **specified**, specifications can be **decomposed** into component contracts, and conformance can be **verified** before deployment. AI components break all three, through five distinct mechanisms.

1. Behaviour is statistical. The component has no behaviour, it has a distribution over behaviours, because each token is sampled. Run the same prompt twice and you have two samples, not a test and a confirmation. *Consequence:* every requirement needs a quantifier and every test needs a sample size. A demonstration is an anecdote. (Chapters 8, 10.)

Example 1.4 | Repeatability measured by pass^k

Setting. τ -bench [97] evaluates agents on tool-agent-user interaction in realistic service domains: retail and airline customer support, where an agent must follow a written policy, call tools against a database, and converse with a simulated user. Tasks are scored on whether the database ends in the correct state and the policy was respected.

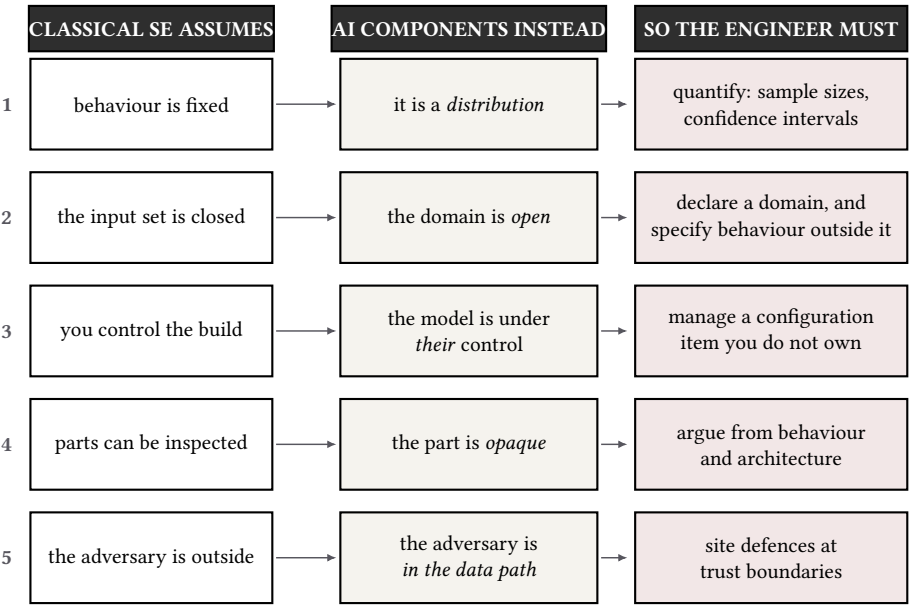


Figure 1.4. Five assumptions, what replaces each, and what the engineer must therefore do. The right-hand column is, in effect, the table of contents for Parts III and IV.

The metric that matters. Most benchmarks report $\text{pass}@1$: did the agent succeed on one attempt. τ -bench also reports pass^k : did the agent succeed on *all* k independent attempts at the same task. The distinction sounds pedantic and is not, because pass^k is the question a deployed system is actually asked. A support agent does not handle a ticket once. It handles that kind of ticket four hundred times a day.

The result. A GPT-4o function-calling agent averaging above 60 % on $\text{pass}@1$ falls below 25 % at pass^8 .

Working it. If the eight trials were independent draws with $p = 0.6$, then

$$\text{pass}^8 = p^8 = 0.6^8 \approx 0.017,$$

about 1.7 %. The observed figure is above 25 %, roughly fifteen times higher. That gap is informative, and it is worth being precise about what it means.

Independence would say the agent has a fair coin at every attempt, so eight consecutive successes are rare. The data say otherwise: success is *clustered by task*. Some tasks the agent gets essentially every time; others it fails essentially every time; comparatively few are genuine coin flips. An aggregate of 60 % is therefore not “most tasks usually work”. It is closer to “about a quarter of tasks reliably work, some fraction reliably fail, and the average lands at 60”.

Why that is good news, engineering-wise. Scattered failures are the hard case: they cannot be predicted, isolated or fenced. Concentrated failures can be found, characterised and handled. If a quarter of your task types account for most of your

failures, you can route them to a human, constrain them, or decline them, and the system's effective reliability rises without the model changing at all.

What to do in your own system. Stop reporting a single average. For each task type, report pass@1 and pass^k for a k matching how often that task really recurs. An agent at 90 % pass@1 and 85 % pass^8 is a different engineering object from one at 90 % and 40 %: the first has a broad, mild weakness, the second has a specific, severe one, and no single number tells them apart. The second is also, usually, the easier one to fix.

The exercise this implies. Run one task you believe works fifty times without changing anything and plot the outcomes by task type rather than in aggregate. Most teams doing this for the first time discover their system is bimodal and that nobody had looked.

2. The operating domain is open. There is no enumerable input set. For a natural-language interface, inputs nobody anticipated are not a rare tail, they are most of Tuesday. *Consequence:* classical coverage arguments are unavailable. You must instead declare an operating domain explicitly and specify what the system does *outside* it. (Chapter 8.)

3. The substrate is non-stationary. The model is under a supplier's configuration control. It will be updated, and deprecated on their schedule, and an update can change behaviour you depended on while every interface you can see stays identical. A prompt tuned against one version may quietly degrade against the next. *Consequence:* configuration management must extend to an item you do not own, and re-verification becomes a recurring operating cost rather than a one-time project cost. This is the most under-managed risk in deployed AI systems. (Chapters 4, 9, 11.)

4. The component is opaque. You cannot read it and determine what it will do. It can be asked to explain itself, but an explanation produced by the same process that produced the behaviour is evidence of a limited kind. *Consequence:* assurance must rest on behavioural evidence and architectural constraint, not inspection. (Chapters 10, 13.)

5. The adversary is inside the data path. A component that reads untrusted content while holding privileges is what security engineering has long called a *confused deputy*. Figure 1.1 is one. This is not a defect a patch removes; it is the shape of the thing. *Consequence:* security must be argued architecturally (what content reaches what authority, and what can leave) rather than by filtering inputs. (Chapter 14.)

Example 1.5 | Scoring proposed fixes against the five assumptions

Setting. A support-ticket agent reads incoming tickets, queries a customer database through a SQL tool, and drafts replies. A ticket arrived containing, in ordinary prose, an instruction addressed to whatever machine read it. The agent issued a destructive statement against the production database, which it was able to execute because the SQL tool had been given an administrative credential during integration as a temporary measure four months earlier.

The post-mortem’s output. Three remediations: a stricter system prompt forbidding the agent from following instructions found in ticket text, a request to upgrade to a more capable model, and a filter rejecting any tool call containing DROP.

The scoring rule. For each fix, ask not “does it help” but “which of the five assumptions in §1.5 does it act on, and does it change a rate or a possibility”. A rate can be driven down. A possibility can be removed.

Fix	Assumption	Changes	Why
Stricter system prompt	none	rate	It is an instruction competing with another instruction, adjudicated by the element whose judgement is in question.
Upgrade the model	none	rate	Lowers how often text is read as instruction. The path from ticket to database is untouched.
Filter for DROP	none	rate	Blocks one statement. DELETE and UPDATE remain, and UPDATE is worse because it is silent.
Read-only scoped credential	5	possibility	The destructive action cannot execute regardless of what the model decides.
Confirmation gate on writes	5	possibility	Inserts a regulator between the decision and its effect.
Declare DB queries out of domain for ticket-derived text	2	possibility	Bounds where the system may operate at all.

Quantifying the difference. Suppose the three shipped fixes together reduce the incidence by a factor of one hundred, which is generous. The agent handles roughly 400 tickets a day, so about 100,000 a year. If the original rate was one in 50,000 tickets, the fixes take it to one in five million, which is an expected recurrence every fifty years. That sounds adequate until you notice the estimate rests on an unmeasured

rate, applies to one attack phrasing, and assumes an adversary who does not adapt. The scoped credential requires none of those assumptions, because the rate does not appear in the argument at all.

Why the cheap fix was chosen. The scoped credential was requested in month one and blocked behind a two-week platform ticket that was eventually closed as a duplicate. The prompt change took twenty minutes. This is the ordinary shape of the decision, and it is worth naming: the fixes that change possibilities are usually the ones that require somebody else's cooperation, and the fixes that change rates are usually the ones you can ship alone.

Try this on your own work. Take the last fix your team shipped for an AI failure and place it in the table. If it lands above the rule, write down what the below-the-rule version would have been and what it would have cost, then compare that cost with the incident it was responding to.

1.6 The limits of model improvement

It is fair to ask, at this point, whether this book is a temporary artifact of immature technology. The models will improve; they have improved considerably while it was being written. If the problems described here are symptoms of components that are not yet good enough, then the right response is to wait rather than to engineer.

It is not, and the reason is structural. The properties at issue belong to *composition*. A model has no blast radius; an agent with a credential does. A model cannot exfiltrate data; a model with a retrieval corpus, private data, and an outbound network call can. A model has no failure rate over a forty-step task until somebody builds a forty-step task.

There is also a coupling that runs against us. The generality that makes a component useful for unforeseen tasks makes its failure modes unforeseeable, because the set of things it might do is the set of things it can do. Capability and risk are not independent axes to be optimised separately. Chapter 2 gives this its name in systems science (*robust yet fragile*) and Chapter 6 draws out the architectural consequences.

Better models change the numbers in this book. They do not change its structure.

1.7 A failure taxonomy by origin

Diagnosis needs a classification, and the useful axis is **origin**, not symptom. “Hallucination” is a symptom that can arise from the model fabricating, from retrieval returning nothing relevant, from context management dropping the passage that mattered, or from a question the system had no basis to answer. Sorting by symptom sends you to the wrong layer, which is exactly what happened in all three incidents above.

We use these classes throughout. Chapter 12 develops the first three into a quantitative fault model; Chapter 13 takes *authority* into hazard analysis; Chapter 11 takes *assurance* into operations.

origin	what went wrong, and typical instances	visible to
Element	the supplied model does the wrong thing fabrication · omission · misgrounding	model evaluation
Interface	the parts do not mean the same thing format violation · tool misuse · stale retrieval	
Loop	the harness does not converge thrashing · non-termination · goal drift	
Authority	the system was <i>allowed</i> to do it over-scoped credential · missing gate · no undo	nothing short of a system test
Assurance	nobody could have known undetected drift · unobservable run · unowned loop	

Figure 1.5. Five origin classes. The rule marks a boundary that matters: the two classes below it are not properties of a model, so no amount of model evaluation can detect them, and they are, empirically, where the incidents that reach the news come from.

It is worth saying that this is not the only taxonomy on offer, and that a more empirically grounded one exists for the multi-agent case. Cemri et al. [15] built MAST, a taxonomy of multi-agent system failures, by annotating more than 150 execution traces across five open-source frameworks with six expert annotators, reaching an inter-annotator agreement of $\kappa = 0.88$. They arrive at fourteen failure modes in three groups: system design issues, inter-agent misalignment, and task verification.

The two schemes agree more than the different vocabulary suggests, and where they differ is instructive. Their *system design* group corresponds closely to what we call element and interface faults. Their *inter-agent misalignment* is a category this chapter does not yet have, because it only exists once there is more than one agent, and Chapter 6 adopts it. Their *task verification* group maps onto our assurance faults. What our scheme carries that theirs does not is the authority class, which is unsurprising: MAST was built from traces of agents failing at tasks, and an over-scoped credential does not produce a failed trace. It produces a successful one with consequences nobody wanted.

That asymmetry is itself a finding. Failure taxonomies built from execution traces will systematically under-represent authority faults, because the trace of the support assistant’s worst day shows an agent doing exactly what it was asked.

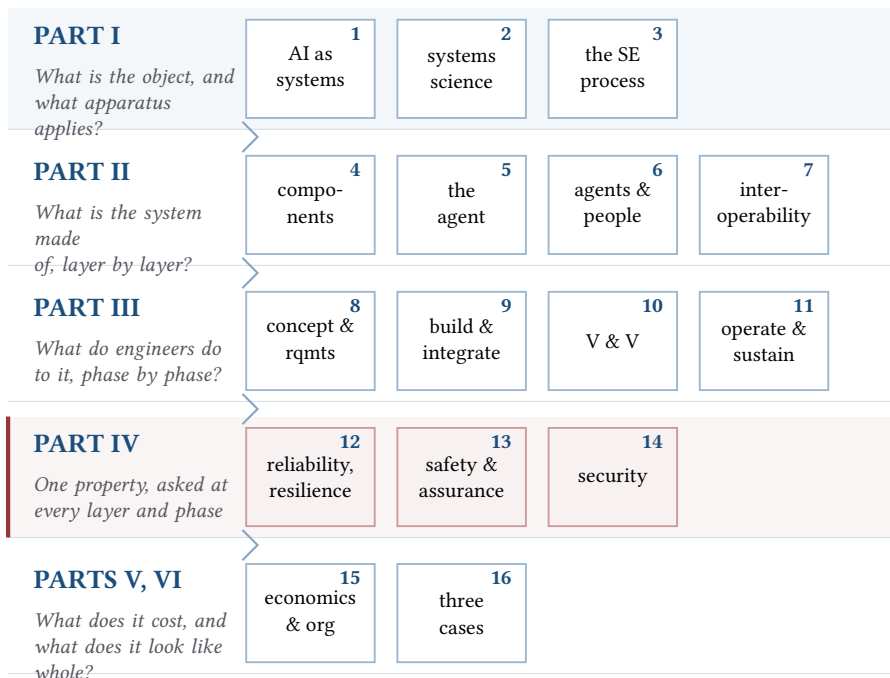
1.8 Plan of the book

Two kinds of set-piece recur, and they are marked differently because they carry different weight. A **worked example** develops an argument through a concrete system with explicit numbers; the systems are composites and the numbers are illustrative, chosen so that the arithmetic can be followed and repeated. A **case** is a

documented public incident or study, described at the level its sources establish and no further, with those sources named at the end. Where a case and an example make the same point, the example is there because it can be worked through and the case is there because it happened.

The organisation of this book follows from an argument it has now made twice. An AI system is a thing with parts arranged in a particular way, and it is also a thing that passes through a lifecycle. Those are two different views of the same object, and neither one alone will get you to a system you can defend.

So the book is built on both axes. Part II walks up the system, layer by layer, and answers the question *what is this made of*. Part III walks along the lifecycle, phase by phase, and answers *what do engineers do to it*. Part IV then does something the other two cannot: it takes a single property, such as reliability or security, and asks it at every layer and every phase at once. Those are the vertical cuts, and they exist because the properties that matter most are precisely the ones that belong to no single layer and no single phase.



Parts I to III run down the page; Part IV is marked because it cuts across Parts II and III, asking one dependability property at every layer and every phase rather than following either axis.

Figure 1.6. How the book is organised. The spine on the left shows what depends on what; Part IV is tinted because it cuts across both Part II and Part III rather than following either axis.

What each part does

Part I, Foundations. This chapter has established what an AI system is, what systems engineering is, and why the second applies to the first. Chapter 2 supplies the systems-science vocabulary that everything after it depends on: decomposition and composition, the three graphs over a system, networks and hubs, feedback, emergence, requisite variety and leverage points. It is the shortest path to being able to say precisely what is wrong with a design. Chapter 3 then takes the standard systems engineering process set and audits it against AI systems, process by process, marking what survives, what needs rework, and what has no adequate analogue at all. That audit is the book's central claim and the reason the remaining parts are shaped as they are.

Part II, Architecture. Four chapters up the layers. Chapter 4 characterises the components you procure and configure: the model, the context, the tools. Chapter 5 treats the agent as a system element, which means the loop and the harness rather than the model. Chapter 6 handles organisations of agents and of people, which is where delegation and human oversight live. Chapter 7 crosses the boundary of your own operation entirely, into interoperability with agents you neither control nor can test.

Part III, The lifecycle. Four chapters along the phases, each reworked according to the audit. Chapter 8 covers concept of operations and requirements, where the open domain first has to be confronted. Chapter 9 covers building and integrating, including configuration management of a model you do not own. Chapter 10, on verification and validation, carries more of the book's weight than its length suggests, because it is the discipline the field most conspicuously lacks. Chapter 11 covers transition, operations and sustainment, which is where the assurance loop that Chapter 3 demands actually gets built and staffed.

Part IV, Dependability. Three vertical cuts. Chapter 12 asks about reliability and resilience at every layer; Chapter 13 does the same for safety, hazard analysis and the assurance case; Chapter 14 for security. These chapters are deliberately placed after both Part II and Part III, because each of them draws on every layer and every phase, and reading them earlier would mean reading them thinly.

Parts V and VI. Chapter 15 treats cost, latency and the engineering organisation, on the argument that a system too expensive to verify will not be verified, which makes economics a dependability concern rather than a business one. Chapter 16 carries three systems end to end, and those three have been appearing in vignettes since §1.2 so that the final treatment is a consolidation rather than an introduction.

How to read it

The dependency spine in Figure 1.6 is honest: Chapters 2 and 3 are load-bearing for everything after them, and skipping them will make Part IV read as a list of unconnected practices rather than as three cuts through one structure. Beyond that, the book tolerates several routes.

Read straight through if you are using it as a course text; the chapter order is the order it was designed to be taught in, and the capstone of Appendix E runs alongside it.

If you have a system in front of you and a specific problem, go to Part IV first and follow its cross-references back up into Parts II and III. The vertical-cut chapters are written to be entered this way, and each of them says which layers and phases it is drawing on.

If you are coming to this from machine learning and want the shortest path to the unfamiliar material, read Chapters 2, 3 and 10. Those three carry most of what a systems discipline knows that an ML background does not supply.

If you are coming from systems engineering and want the shortest path to what is genuinely different, read Chapters 4, 5 and 14. The first two describe components unlike any you have specified before, and the third describes an adversary who arrives through the ordinary data path.

What this book does not do

It does not teach machine learning, and it treats the model as a supplied component throughout. It does not survey frameworks or tools, which would date faster than the argument. It is not a compliance manual, although Chapter 13 maps its content onto the current standards landscape. And it does not claim the discipline described here is finished: Chapter 3 marks one process as having no adequate analogue at all, and the epilogue is a list of the problems this book can state but not solve.

What this chapter established

- ▶ An AI system has four nested scopes: model, application, agent and system. Almost every published number is a model-scope number and almost every property anyone cares about is a system-scope property.
- ▶ The three opening incidents were reported as model failures. One was an agent-scope authority failure, one a system-scope failure of ownership and detection, and one an application-scope failure of composition. In each the misdiagnosis sent the remedy to the wrong layer.
- ▶ Five assumptions that conventional components satisfy are violated here: the element is supplied rather than built, its behaviour is a distribution, it is non-stationary, it is opaque, and it consumes untrusted content while holding authority.
- ▶ Iteration is what turns a capability into a system. A single inference has no dynamics; a loop has compounding error, goal drift, and the possibility of a run that neither succeeds nor stops.
- ▶ The failures in the public record are interface, integration, requirements and validation failures, which is to say they are the failures systems engineering exists to address, appearing in a new material.

Exercises

- 1.1 **[A]** A task requires n sequential steps, each succeeding independently with probability p . (a) Find the largest n for which end-to-end success stays above 90 %, for $p = 0.9, 0.95, 0.99$. (b) A colleague proposes retrying each failed step and claims per-step reliability becomes $1 - (1 - p)^2$. State two assumptions this requires, and describe a realistic case where retrying *reduces* end-to-end success. (*Hint: consider a step whose failure is deterministic given its input, and a step whose retry consumes a token budget later steps need.*)
- 1.2 **[L]** Take one agent task you believe works. Run it fifty times unchanged. Record outcome, total token cost, and step count. Report the success rate with a 95 % confidence interval, and the ratio of the most expensive run to the cheapest. Then write a paragraph on what a single successful demonstration would have told a stakeholder, and what it would have concealed.
- 1.3 **[D]** For an agent you have access to, list every tool and, for each, the credential it uses and the worst outcome a single call could produce. Redraw Figure 1.1 for it: which elements sit in the untrusted zone, which in the privileged zone, and where does content cross?
- 1.4 **[D]** Take a publicly reported AI incident. Classify its origin using Figure 1.5, and identify the remediation actually applied. If the remediation addressed a different class than the origin, state what the correct remediation would have been and what it would have cost.
- 1.5 **[A]** A system has one model shared by five independent validation checks. A colleague argues the probability all five pass erroneously is q^5 , where q is a single check's false-pass rate. State the assumption required and explain informally why it fails here. Chapter 2 gives this its structural name; Chapter 12 quantifies it.

Notes and further reading

On the anatomy of §1.1. The four-part division into model, context, tools and loop is our own framing, but it is not controversial; it corresponds closely to how agent frameworks are structured in practice. The claim that most engineering freedom lies outside the model is supported indirectly but strongly by the benchmark evidence discussed below, in which the same model produces very different outcomes as the harness around it changes.

On statistical behaviour and the limits of a single number (§1.5, assumption 1). The pass^k metric and the GPT-4o figures in Example 1.2 are from Yao et al. [97], who introduced τ -bench for tool-agent-user interaction in retail and airline service domains. Readers building evaluation pipelines should take the metric as seriously as the benchmark: reporting only $\text{pass}@1$ conceals whether failures are scattered or clustered, and the two cases call for entirely different engineering responses.

On long-horizon behaviour. Example 1.1 draws on Yuan et al. [98], whose OSWorld 2.0 extends computer-use evaluation to 108 workflows with a median completion time of 1.6 hours and roughly 318 tool calls. Their reported failure modes, which concern lost constraints, missed mid-task information, guessing in place of asking, and skipped verification, are worth

reading in full by anyone designing a harness, because all four are addressable in the loop rather than in the model.

On failure taxonomies (§1.7). The most empirically grounded taxonomy currently available for the multi-agent case is MAST, developed by Cemri et al. [15] from over 150 annotated execution traces across open-source frameworks, with reported inter-annotator agreement of $\kappa = 0.88$. Our origin-based scheme and theirs agree substantially; the divergence we note in §1.7, that a trace-derived taxonomy cannot surface authority faults, is our own observation and we would welcome its being tested.

On the confused deputy (§1.5, assumption 5). The term and the analysis originate with Hardy [35], writing about capability systems, and transfer to agentic systems without modification. Greshake et al. [33] give the modern instance its name and its first systematic treatment; Chapter 14 takes this up properly.

On systems engineering itself (§1.3). The process baseline is Int [47], with Int [41] as the practitioner's reference. Leveson [58] is the essential corrective to a purely process-based reading of the discipline, and her argument that safety is a control problem rather than a reliability problem is developed in our Chapter 13.

Open at the time of writing. Whether assumption 4 (opacity) is permanent or an artifact of current interpretability methods is unresolved, and the engineering position we take, that assurance must not *depend* on its resolution, is a deliberate hedge rather than a prediction. Whether assumption 5 admits an architectural solution at all is more seriously open, and we return to it in Chapter 14 and the epilogue.

The Systems View: Structure and Feedback

Learning Objectives

1. Draw a system boundary deliberately and say what your choice excludes.
2. Decompose an AI system by at least three criteria and justify which cut you would build to.
3. Distinguish *composability* from *compositionality*, and explain why AI systems have the first and lack the second.
4. Draw the information-flow, control-flow, and authority-flow graphs of a system and read a vulnerability off their divergence.
5. Identify the feedback loops in a deployed system, classify each, and name its owner.
6. Apply requisite variety to an oversight arrangement and state which of three responses the design takes.
7. Rank a proposed intervention by leverage.

How to read this chapter

It is structural, not mathematical. Every idea here is something you *draw* and then *read*. Nothing is solved. The payoff is that Chapters 12, 13, and 14 can say “the authority-flow graph” or “the hub” or “the variety gap” and mean something precise, rather than gesturing at intuitions the reader must reconstruct.

2.1 A familiar system in systems terms

Chapter 1 closed with a claim and no vocabulary to defend it. The claim was that in all three incidents the failure lay in the arrangement of the parts rather than in any part, and that the post-mortems reached for the wrong layer because they had no language for the right one. Supplying that language is the work of this chapter.

We will build it out of a system the reader already knows. The support assistant of Chapter 1 is small enough to hold in the head and large enough to exhibit every property we need, and by the end of this chapter it will have taught us its own failure.

Begin with the definition everything rests on. A system is a set of elements, interconnected, whose arrangement produces behaviour that no element produces alone. Every clause of that sentence names a decision an engineer has to make, and the decisions are not always recognised as decisions at the time they are taken.

The elements are what you choose to treat as parts. In the support assistant these are the model, the harness that assembles its context, the ticket queue, the SQL tool and the database behind it, the support staff who review drafts, and the customers whose data sits in that database. Already there is a choice: the list could have stopped at the SQL tool, and in most design documents it would have.

The interconnections are what you choose to treat as relationships between the parts, and Section 2.4 will show that this choice is where most AI system surprises are decided.

And the behaviour of interest belongs to the whole. This is what makes a system a system rather than a heap. It also means that the properties an engineer is answerable for, safety and security among them, cannot be found by examining the parts one at a time, however carefully.

The boundary is the least obvious of these decisions and the most consequential. It separates what is inside the system from what is merely its environment, and although it feels like something you discover by looking, it is something you draw. Two competent engineers standing in front of the same deployment will draw it differently, and both drawings can be internally consistent.

Figure 2.1 draws three of them over the support assistant. Inside the supplier's boundary, which encloses the model and nothing else, "the model met its specification" is true, and it stayed true throughout the incident. Inside the operator's boundary, which adds the harness and the tools, "the system met its service level" was also true, because the service levels concerned latency and draft quality and said nothing whatever about database authority. Inside the affected party's boundary, which reaches the customer whose table was dropped, "the outcome was harmful" is true as well.

All three hold at once, and recognising that they can is the first thing this vocabulary buys. A great many public arguments about accountability in AI are arguments about which rectangle counts as the system, conducted by people who have not said which rectangle they mean and do not realise they are disagreeing about that.

The engineering obligation is not to find the correct boundary, because there is not one. It is to declare yours and to know what the declaration excludes, since everything outside it is something you have decided not to be responsible for. That decision can be entirely reasonable. What is not reasonable is making it silently, and the insurance company of Chapter 1 made it silently: the corpus its assistant depended on sat outside every boundary anyone had drawn, owned by an underwriting team that had never been told a system was reading their publications.

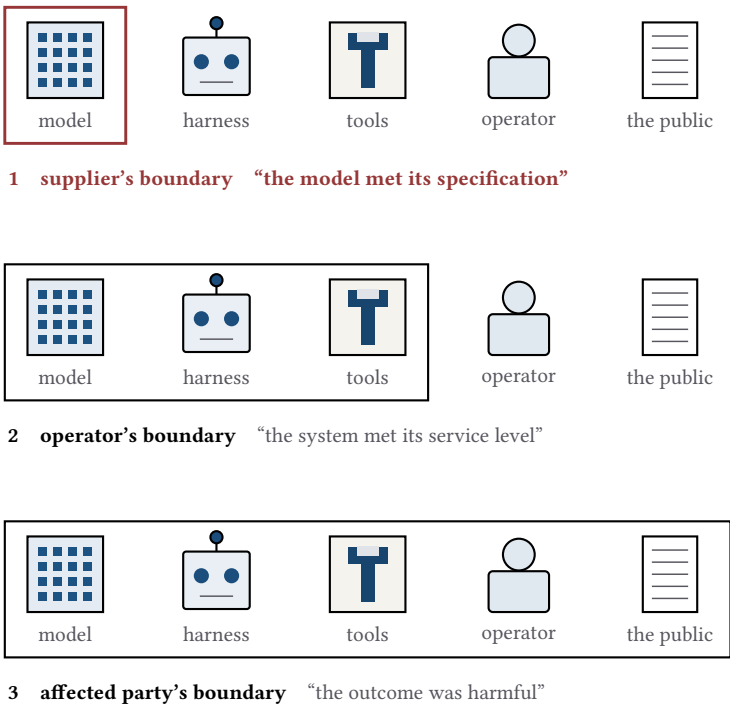


Figure 2.1. Three boundaries over one deployment. The elements never change; only the enclosing rectangle does. Each supports a true statement, and the three statements are not in conflict.

Example 2.1 | Locating the boundary of a retrieval assistant

Setting. An insurance company built an assistant to help call-centre staff answer policy questions. It retrieves from about twelve thousand documents: policy wordings, endorsements, underwriting guidance and regulatory notices. Staff type a question, the assistant retrieves relevant passages and answers with citations.

The boundary as the team drew it. In month one the architecture diagram enclosed the model, the retrieval index, the chunking and ranking code, the answer template and the call-centre interface. Five elements, one owner, one on-call rota. Outside the boundary: the corpus itself, the underwriting team who write it, the document management system they publish through, and the regulator whose notices arrive on no schedule at all.

Was the boundary wrong? Not obviously, and this is the point. The team did not control underwriting publication and could not have. Drawing a boundary that included a department you have no authority over would have been dishonest in a different way. Reasonable engineers draw this boundary.

What went wrong. Nobody wrote it down. Had anyone written it down, they would have had to complete a sentence of the form: *the correctness of this system depends on*

the following things outside our boundary. The list would have had one entry, and that entry would have been the corpus.

What the written sentence generates. Follow it through, because the point of the exercise is that it produces work automatically.

Question the sentence forces	Artifact it produces
How stale may a retrieved document be?	a freshness requirement with a number in it
Who changes it, and on what cadence?	a named contact in underwriting
How would we know it had changed?	a divergence check between index and evaluation snapshot
What do we do when it does?	a re-evaluation trigger in the operations runbook

Four artifacts, perhaps two days of work, none of which existed.

What happened instead. The underwriting team published revisions weekly, as they always had, for their own reasons, unaware that a system was reading their output. The evaluation set had been built from a snapshot and stayed correct against that snapshot forever. Escalation rates rose from 31 per thousand calls to 46 over four months and were attributed to a product launch and then to seasonal mix. An underwriter reviewing a complaint eventually noticed the assistant quoting a clause withdrawn eleven weeks earlier.

The test to apply to your own system. Write the boundary in the concept of operations. Then read the list of what falls outside it and ask, item by item: *what happens if this changes without telling us, and how would we find out?* A boundary you can state is a boundary you can audit. One you have only in your head cannot be audited, and more importantly cannot be handed to the person who takes over from you.

2.2 Decomposition

Having drawn a boundary, the next move is to divide what is inside it, and the technique for doing so is older than computing. Nobody can reason about ten thousand interacting things. Most people can reason about twelve things each made of twelve things, which is why decomposition is what makes large systems possible at all rather than merely convenient.

Whether a particular division is any good has a classical answer in two parts. Cohesion measures how much the contents of a part belong together: a part with high cohesion does one thing, and you can say what it is in a sentence. Coupling measures how much parts depend on one another: with low coupling, a change inside one part does not force changes in the others. A good decomposition maximises the first and minimises the second, and that is the whole rule.

Behind the rule sits an observation of Herbert Simon's [89] that is worth more than the rule. Complex systems that survive, he noticed, are almost always assembled from stable subassemblies, with strong interactions inside each and weak interactions between them. He called this near-decomposability, and the qualifier carries the weight. Perfect decomposition is not available in any system worth building. The engineering question is never whether the parts are truly independent, but whether what remains of their interaction is weak enough to reason around.

What the rule does not settle is which division to make. Any interesting system admits several divisions, they disagree with one another, and the disagreement is not a defect to be resolved: each cut is a different question asked of the same material, and a question you do not ask is a class of fault you will not see.

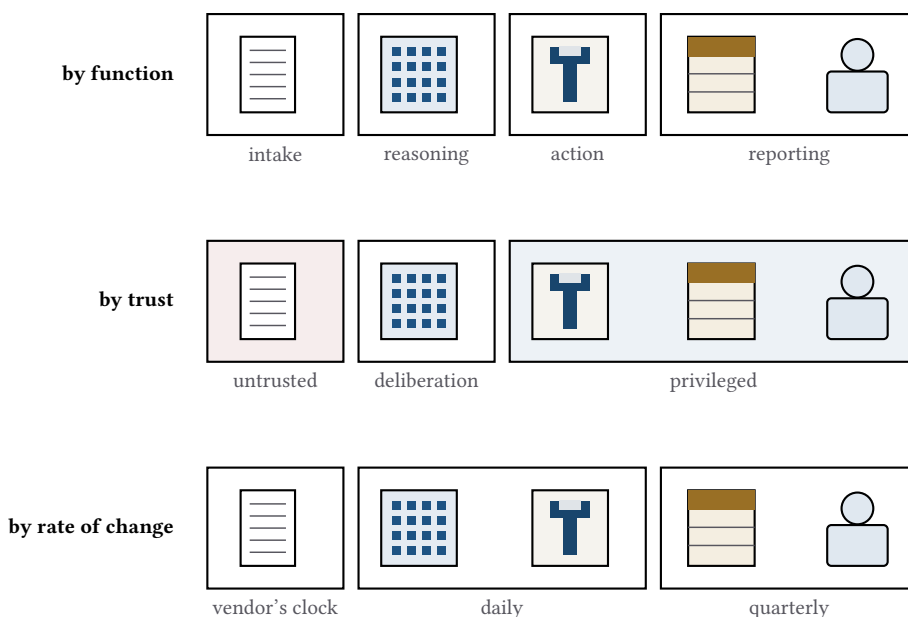


Figure 2.2. One system divided three ways. The elements are identical in each row; only the grouping changes. Each grouping is valid, and each makes a different class of fault visible.

Figure 2.2 cuts the same five elements three ways. The functional cut along the top is the one almost every team reaches for, because it mirrors how the code is organised and how the work was divided between people. Intake, reasoning, action, reporting. It is a perfectly reasonable way to build.

It is also the cut in which Chapter 1's incident is invisible, and that is worth dwelling on, because the team that built the support assistant were not careless and did have an architecture diagram. Look at the middle row, which groups elements by whether they handle untrusted content and whether they hold privilege. Drawn this way the architecture states its own problem: content arriving from outside and a credential with administrative scope are in the same region of the diagram, and no line separates them. That is the security architecture. It does not appear in the

functional view at all, because the functional view was not asking the question, and nobody thought to ask it in a different view.

The third cut groups elements by how often they change. Prompts move daily, tool definitions quarterly, the model on a supplier's schedule that nobody in the building controls. This view says nothing about correctness and everything about maintenance: it predicts what will rot first and where re-verification cost will land, which is the subject of Chapter 11.

The conclusion is not that one of these is right. It is that you build to one and draw the others anyway, because the cuts you do not build to are diagnostic instruments and they cost an afternoon each.

That much would be true of a bridge, and a civil engineer reading this section would recognise every word of it. The specialisation to AI systems begins when we ask what a part even is, given that the part in question is a model plus a prompt plus a set of tools, none of which has a datasheet and only one of which you wrote.

Five criteria compete for the cut and they genuinely conflict. Cutting by consequence isolates the elements whose actions cannot be undone, which tells you where approval gates belong. Cutting by trust boundary keeps untrusted content out of any part that holds privilege. Cutting by verifiability puts a checkable output at every seam, so that something can be said about each piece on its own. Cutting by context locality keeps together the parts that need the same information, on the reasoning that every boundary is a place where information is lost. Cutting by rate of change groups the parts that move together. You cannot satisfy all five, and choosing among them is an architectural decision that Chapter 8 treats as a trade study rather than a matter of taste.

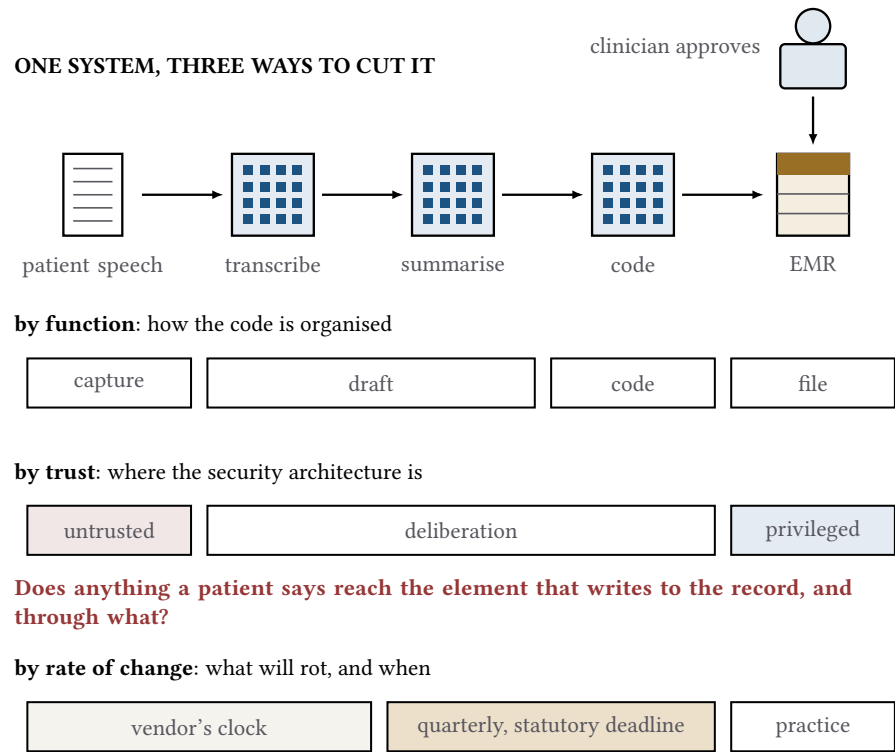
There is also a complication with no classical counterpart, and it is why this section could not simply have cited Simon and moved on. In a well-decomposed conventional system, parts communicate through narrow declared interfaces, and that is precisely what makes them independent. An agent's context is not a narrow declared interface. It is a single accumulating channel that every part reads from and writes to, which means two parts sharing a context are coupled whether or not any line on the diagram joins them. One can change the other's behaviour with no declared interface between them at all. Practitioners meet this in the form that adding an instruction for step six degrades step two, and they usually meet it after shipping.

The same is true of the model. Every part that calls it inherits its refusal behaviour, its failure modes and its version. On the block diagram those parts look separate. In the dependency graph they are one node with many edges, a difference Section 2.4 makes visible and Chapter 12 makes quantitative.

So the parts were never independent. Decomposition still buys tractability here, which is worth a great deal, but it does not buy the independence that classical decomposition quietly promises. Holding those two benefits apart is most of what this chapter is for.

Example 2.2 | Three cuts of the clinical assistant

Setting. A clinical documentation assistant listens to a consultation through a microphone in the room, transcribes it, drafts a clinical note in the practice’s house format, proposes billing codes, and files the note to the electronic medical record once the clinician approves. The clinician remains professionally accountable for every word.



The element row never changes. Only the grouping does, and each grouping answers a different question.

Cut 1: by function. Capture, transcribe, summarise, code, file. This mirrors how the code is organised and how the team is staffed, and it is a perfectly good build structure. It tells you where to put unit tests and which engineer owns which defect.

What it shows: the data path, and the obvious places to measure quality.

What it hides: everything below.

Cut 2: by trust. Now group by whether an element handles content from outside the system and whether it holds privilege.

Region	Contains
Untrusted	patient speech, prior notes written elsewhere, referral letters
Deliberation	transcription, summarisation, code proposal
Privileged	the write to the medical record, the billing submission

Drawn this way one question becomes unavoidable: *does anything a patient says reach the element that writes to the record, and through what?* The answer is yes, through the deliberation region, and the only thing between them is a clinician clicking approve. That is either an adequate control or it is not, but in the functional cut the question has no natural place to be asked, so it never was.

Cut 3: by rate of change. Group by how often each element moves.

Element	Changes	Driven by
Billing code rules	quarterly	regulation, with a legal deadline
Note template	irregularly	the practice
Prompts	weekly	the team
Model	unpredictably	the vendor
Transcription	rarely	a different vendor

What it shows: the maintenance calendar, and one thing no other view surfaces, which is that the billing rules carry a statutory deadline. Missing it is not a quality regression; it is a compliance failure with a date attached.

The accounting. Two extra diagrams, one afternoon. What they bought: the security architecture and the maintenance calendar, neither of which the chosen cut would have produced before the first audit or the first missed deadline.

The general move. Build to one decomposition, because you must. Draw at least two others, because they are diagnostic instruments rather than build plans and they cost almost nothing. Cut by trust to find the security architecture. Cut by rate of change to find what will rot and when.

2.3 Composition

Decomposition has an inverse, and classical engineering takes it so thoroughly for granted that it rarely gets a name of its own. Having divided a system into parts and built them, you put them back together and expect the result to behave as the parts led you to believe. That expectation is doing an enormous amount of work, and the rest of this section is about when it is earned.

Composition is the act of assembling parts into a whole, and four operators cover nearly every arrangement in this book. In sequential composition each element’s output is the next one’s input, and if all must succeed then reliability multiplies. In parallel composition with aggregation several elements attempt the same task and a vote decides, which reduces error to the extent that the elements fail independently. In hierarchical composition a supervising element sits above subordinates, which is how agent organisations are built. And in composition with feedback an element’s output returns to its own input, which is the agent loop itself.

Figure 2.3 marks something easy to miss about the third. A supervising layer hides the errors below it, which is a benefit when it corrects them and a hazard when

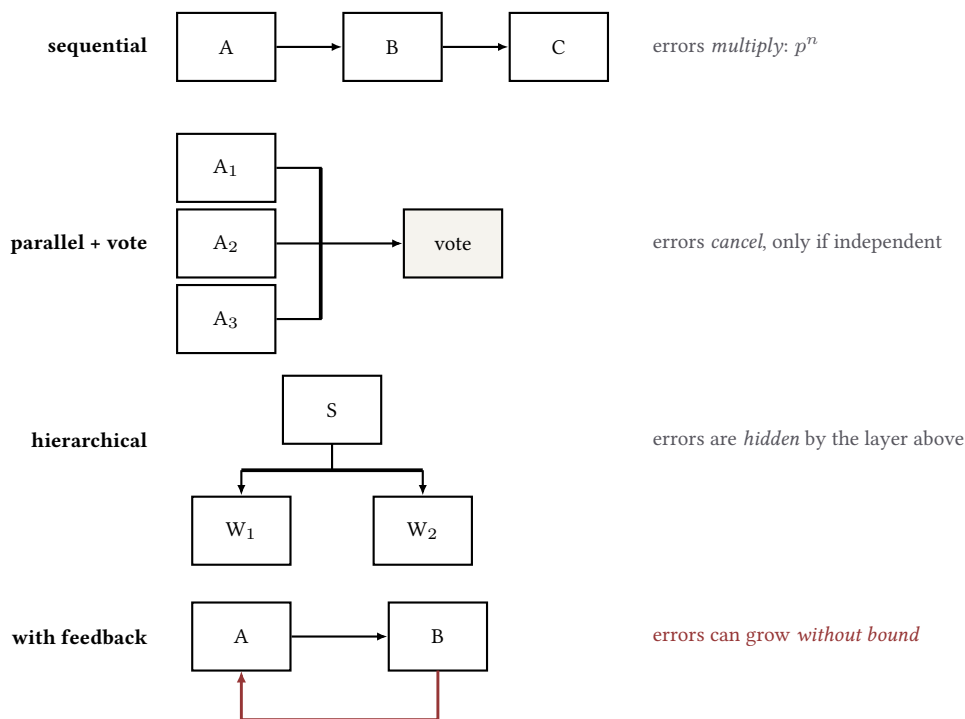


Figure 2.3. Four ways to compose and what each does to error. Only the first has an answer computable from the block diagram alone.

it merely conceals them, and from outside the system those two cases look identical. It also marks that the fourth is the only operator in which error can grow without bound, and that it is the one modern AI systems use most.

Two further facts matter enough to state plainly. Interfaces and contracts are what make composition possible at all: composition is not an operation on parts but an operation on parts together with their interface agreements, which is why Chapter 7 treats an interface as an engineering artifact rather than as documentation. And composition is recursive, so that a composite is itself an element one level up. An agent organisation is agents composed; an ecosystem is organisations composed. The layer stack of Chapter 3 can therefore be read from either end.

We can now state the distinction this chapter exists to make, and with it the thesis of the book. Two properties that classical engineering obtains together, and that ordinary language does not separate, come apart completely in AI systems, and almost every confusion in this field can be traced to their being conflated.

Composability is the property that parts can be combined: interfaces match, messages are accepted, the assembled thing runs. Compositionality is the property that the whole's behaviour follows from the parts'. If you have verified each part, compositionality is what entitles you to say anything at all about the composite.

Classical engineering gets both together and therefore stops distinguishing them. A truss assembled from members with known load ratings has a computable load

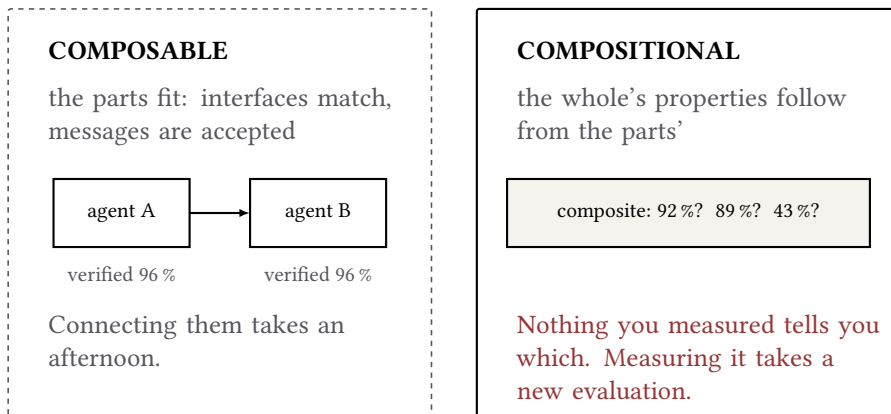


Figure 2.4. Classical engineering obtains both properties at once and consequently stops telling them apart. AI systems obtain the first in abundance and the second hardly at all.

rating, and that is compositionality; it is also the reason the V-model works, because verifying the parts and the interfaces makes system verification largely a confirmation of what is already known.

The customs pipeline of Chapter 1 shows what happens when the second property is missing, and it is worth walking through slowly, because the three ways the inference failed have three different remedies.

The first failure was that the arithmetic was never done. Six steps at ninety-two per cent is sixty-one per cent, not ninety-two, and that much is available to anyone with a calculator before a line of code exists. It is the cheapest finding in this book and one of the most commonly missed.

The second is subtler and accounts for the remaining gap between sixty-one and the forty-three the pilot actually produced. Each step was measured on inputs drawn from a curated set of clean documents, which was the right thing to do for testing a step alone. In the assembled pipeline, step four does not receive a curated document. It receives step three's output, which is sometimes subtly degraded: a field extracted with a transposed digit, a classification that is defensible but not the one a human would have made. Every step after the first was therefore running on a different and worse input distribution than the one it had been validated against, and was, in the sense that matters, operating outside its domain, silently, with nothing raising an error.

The third failure is the most general. A probabilistic contract does not compose the way a deterministic one does. "Correct ninety-two per cent of the time" is not something a downstream element can build on, because it does not say which eight per cent, and the eight per cent is not randomly distributed with respect to what comes next. The documents that step three handled badly are disproportionately the ones step four will handle badly too.

Example 2.3 | CACE: the same finding a decade earlier

Setting. Everything argued in this section was reported for an earlier generation of machine-learning systems, before anyone was building agents, and it is worth reading because the mechanism is identical and the vocabulary is better.

The source. Sculley et al. [86], writing from experience with large production ML systems, name a principle they call CACE: *Changing Anything Changes Everything*.

What they observed. Machine-learning systems mix signals together and entangle them, so that isolating an improvement becomes impossible. Change the input distribution of one feature and the importance, weights and use of every other feature may shift. Adding a feature does this; so does removing one. Their summary sentence is the one to remember: *no inputs are ever really independent*. They note the principle extends past input signals to hyper-parameters, learning settings, sampling methods, convergence thresholds and data selection, which is to say to essentially every knob.

Why this is the same claim as non-compositionality. Compositionality says the properties of the whole follow from the properties of the parts. CACE says the parts have no properties independent of the arrangement they sit in, because changing the arrangement changes the parts. These are the same statement approached from opposite ends: one denies that you can derive the whole from the parts, the other denies that the parts hold still while you try.

What has changed since 2015, and what has not. What has changed is the locus of entanglement. In Sculley's systems it was features sharing a training objective. In agentic systems it is parts sharing a context window and parts sharing a model, which is the pair of couplings §2.2 identified. What has not changed is the engineering consequence: you cannot improve a part in isolation and expect the improvement to survive composition.

Their proposed mitigation, and its limit. They suggest isolating models and serving ensembles, which is the right instinct and which §2.4 qualifies sharply. An ensemble buys independence only to the degree its members fail independently, and six checks that call the same model share a hub. The mitigation and its limit are both in this chapter, ten years apart.

The reading to take away. When a practice from the previous generation of ML systems appears unchanged in agentic systems, that is evidence the mechanism is structural rather than an artifact of one technology. CACE has now survived a complete change of model architecture, training regime and interface. It is not going to be fixed by the next one either.

Put those three failures together and the general statement follows. AI systems are highly composable and stubbornly non-compositional: they are trivial to connect and nearly impossible to predict the result of connecting. The first half of that sentence is why the field moves so fast, and the second half is why so much of what it ships does not work.

Almost everything later in this book answers that one sentence. Part III exists because verification of elements does not transfer to the system, so validation has to happen at the composition. Part IV exists because reliability, safety and security have to be engineered into the arrangement rather than inherited from the parts. And Part II exists because the decomposition you choose settles which of these problems you inherit.

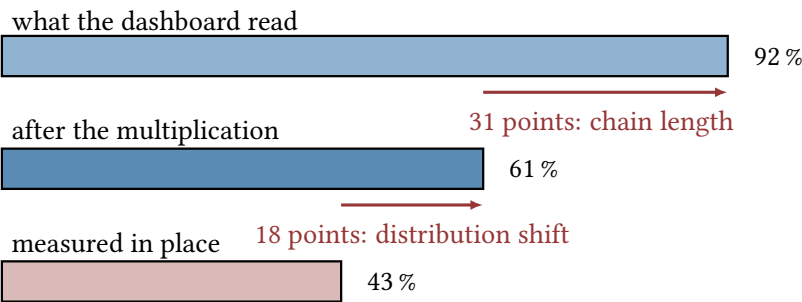
Example 2.4 | Attributing the pipeline’s forty-nine points

Setting. The customs pipeline of Example 1.2, whose arithmetic that example worked through. The three failures the section above separated are not equally large and do not have equally available remedies, and the point of attributing the gap is to see which of them a team could have closed before the pilot rather than after it.

The attribution.

	Predicted	Attributable to
Naive expectation from the dashboard	92 %	n/a
After the multiplication	61 %	chain length: 31 points
After the in-place correction	43 %	distribution shift: 18 points
Observed	43 %	n/a

Thirty-one points were free to discover: the multiplication needs no data the team did not have on the first day. Eighteen more required instrumentation the team could have built in a day, which is measuring each step on the inputs it receives *in place* rather than on the curated set it was developed against. Neither loss required a pilot to find.



Both were discoverable before the pilot: the first with arithmetic, the second with a day of instrumentation.

Attributing the forty-nine point gap. Neither loss required the pilot to discover.

What the third failure adds, which the first two do not. Chain length and distribution shift are both arithmetic once you have the right numbers. The third

failure is not. “Correct 92 % of the time” is not something a downstream element can build on, because it does not say *which* 8 %, and the 8 % is not randomly distributed with respect to what follows: a document that step three handled badly is disproportionately one that step four will handle badly too, because poor scan quality hurts every step. Errors are positively correlated along the chain, and no restatement of the contract fixes that. It has to be handled architecturally, by shortening the chain or inserting recovery, which is why Chapter 12 returns to it with a measurement for how correlated the failures actually are.

The reporting rule this yields. “Each step is 92 % accurate” is a true sentence that caused a project to fail. When the composite has not been measured, the honest report reads: *element accuracies are 91 to 94 per cent, measured in isolation on curated inputs; end-to-end performance has not been measured and cannot be derived from these figures.* That sentence is not longer than the false one, and it would have triggered a pilot a quarter earlier.

2.4 Three graphs and their disagreements

We have been drawing lines between boxes for several pages without asking what a line means. It is time to ask, because conflating the answers is the most common structural error in AI system design.

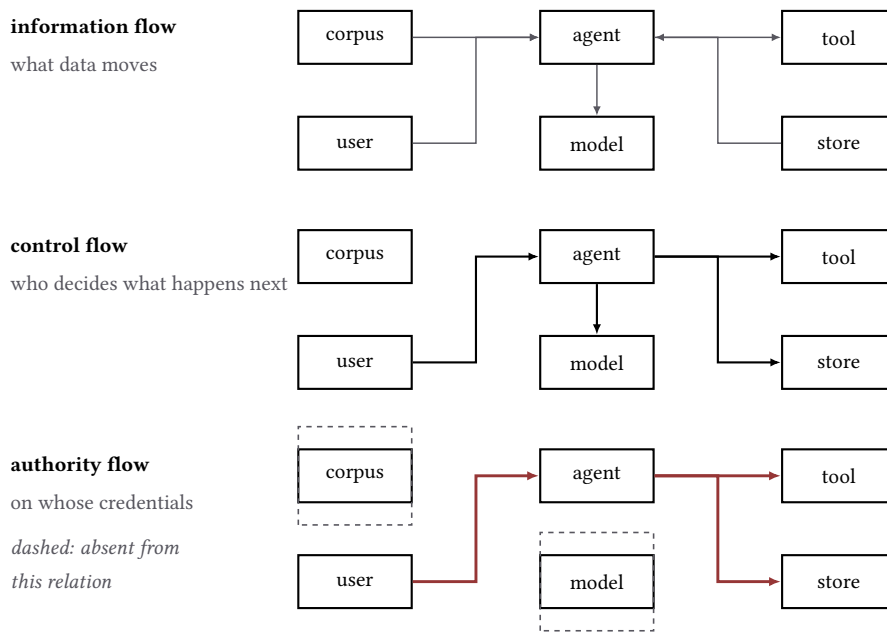
Consider a single line between two elements. It might mean that data moves from one to the other. It might mean that one decides what the other does next. It might mean that one acts using credentials that belong to the other. Those are three different relations; they hold over different pairs of elements; and drawing them with the same stroke hides exactly the discrepancies that matter.

Take them in turn on the support assistant. In the information graph the retrieval corpus is connected, because supplying text to the agent is the entire point of having it. In the control graph it is absent, because a document decides nothing. And in the authority graph it is absent more emphatically still, because a document has no credentials and cannot be given any.

Now read where the graphs disagree. The corpus appears in the first and in neither of the others. The agent appears in all three, and in the third it holds the database credential. So there is a path along which content carrying no authority whatever arrives at the element carrying all of it, through an interface that draws no distinction between text to be considered and text to be obeyed.

That divergence is the whole structure of prompt injection. Notice what has not been mentioned in describing it: no attacker, no jailbreak, no weakness in any model. It is a property of the arrangement, visible on paper before any adversary exists, and Chapter 14 does little more than draw the adversary onto this picture and work out what follows.

The security literature reached the same place from the opposite direction. Greshake and colleagues, giving indirect prompt injection its name in 2023, put the mechanism in one sentence: applications built on language models blur the line



The corpus sends information and holds no authority whatever, yet the element it sends that information to holds all of it. That one divergence is the whole structure of prompt injection.

Figure 2.5. Three relations over one set of elements. Information flow records what data moves, control flow records who decides what happens next, and authority flow records whose credentials an action executes under.

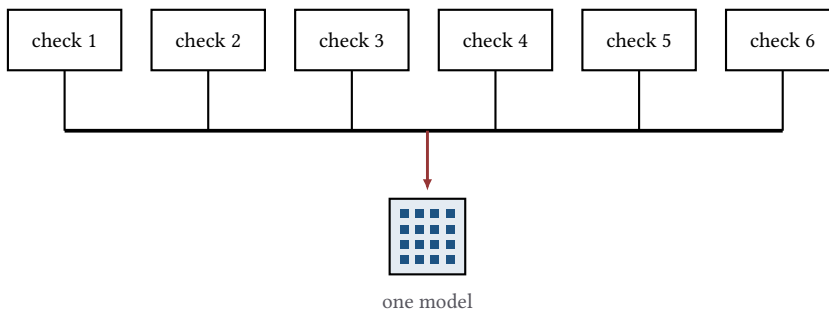
between data and instructions, so an adversary who can place text where the system will retrieve it can act through that system without ever touching its interface. Their taxonomy of consequences includes data theft, ecosystem contamination and propagation between systems.

What the three-graph reading adds is that you do not need their taxonomy to find the exposure in your own design. You need only notice that an information edge lands on a node whose authority its source lacks. The attack techniques will keep changing; that structural condition is what makes any of them possible, and it is checkable with a pencil.

It is also the clearest example in the book of a fault that a functional decomposition cannot show. In the functional view the corpus belongs to intake and the database to action, and they sit in different boxes, which feels like separation. The authority graph shows the separation to be an illusion, because the element between them holds privilege and treats everything in its context alike.

Once a relation is drawn as a graph, its shape starts to tell you things that inspecting elements never would, and the most consequential shape in AI systems is the hub: a node that many others depend on. Hubs are easy to create by accident, because whatever becomes a hub is usually whatever was most convenient to reuse.

six independent checks...



...sharing one dependency.

They cannot fail independently, so their errors do not cancel,
and one version change moves all six at once.

Figure 2.6. Six checks that the design document describes as independent, and the dependency graph that actually governs how they fail.

Figure 2.6 shows a pattern that recurs in nearly every system that tries to buy reliability through redundancy. A team worried about a model's errors adds a second model to check the first, then a rubric grader, then a consistency check across samples. Six checks, six authors, six evaluations, reported separately. The design document calls them independent, and in every sense the design document can see, they are. They all call the same model.

Two consequences follow. Because the checks share a dependency their errors are correlated, so a vote across them is not the redundancy it appears to be; if the model has a systematic blind spot, six checks drawing on it will share the blind spot, agree with each other, and produce a confident consensus that is wrong. And a single model version change moves all six at once, in ways no individual check's evaluation was built to notice, which is why model migration in Chapter 11 is never a local edit.

The principle generalises: shared dependencies defeat independence claims. Before asserting anywhere that parts fail independently, draw the dependency graph and look for what they share. A model, a corpus, a prompt template, a training distribution, a vendor. Any one of them is enough.

The same graph answers a question that matters in operations and is almost never asked during design: if this element fails, what else fails? In a sparsely connected system with clean boundaries a local fault stays local, which is the main practical benefit of a good decomposition. In a densely connected one it does not, and AI systems are dense in ways that are easy to underestimate, because context is shared, models are shared, and agents call agents.

Three structural mitigations follow directly, and all three return as concrete controls in Chapter 12. Reduce connectivity, especially edges that cross consequence levels, so that a fault in something trivial cannot reach something irreversible. Insert boundaries that do not propagate, which is what a bulkhead is: an edge deliberately

left out. And make some edges attenuating rather than transmitting, which is what a validator does when it stops an error instead of forwarding it. The customs pipeline’s eventual fix was the third, applied where the bad extractions were being created.

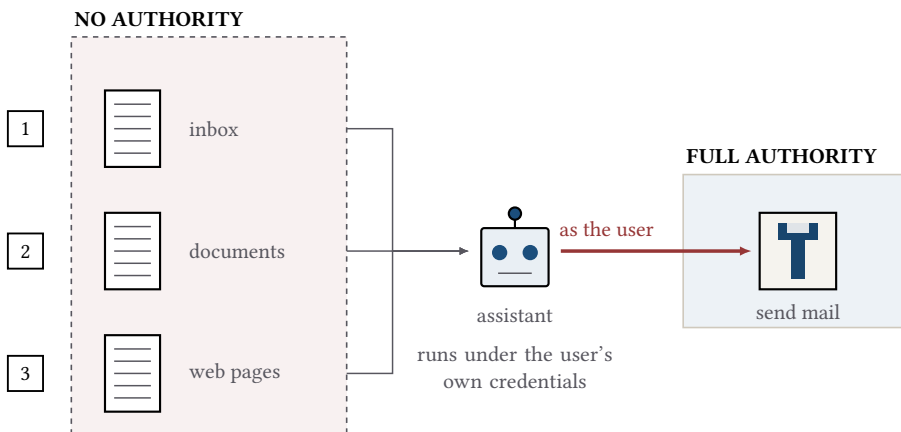
It is worth noticing that blast radius, which practitioners use informally and with some precision, is a statement about this graph: the set of nodes reachable from a compromised one. Making the notion structural makes it measurable, which is the first step towards putting a number in a requirement.

Example 2.5 | Finding the exfiltration paths in a personal assistant

Setting. An assistant integrated with a knowledge worker’s accounts. It can read their inbox, search their document store, fetch web pages, and send mail on their behalf. It runs under the user’s own credentials, which is what makes it useful and what makes this example possible.

The method. Draw the three graphs of §2.4 over the same six elements and read where they disagree. No threat model, no attacker, no list of known attacks. Just the three relations.

Element	Sends information	Exercises control	Holds authority
User	yes	yes	yes (delegates it)
Inbox	yes	no	no
Document store	yes	no	no
Web pages	yes	no	no
Assistant	yes	yes	yes (the user’s)
Mail sender	no	no	acts on the user’s



One control per path, each with its cost:

- | | |
|---|--|
| 1 | confirm any send whose content or recipient derives from unsent mail |
| 2 | mark retrieved text as data at the point of retrieval; refuse tool calls justified only by data-marked content |
| 3 | allow-list outbound recipients, so a successful injection cannot reach an attacker-chosen address |

Three information edges terminate on the one element that holds authority. Each is a path, and each has a control with a stated cost.

Reading the disagreement. Three rows send information and hold no authority: inbox, document store, web. All three send that information to the assistant, which holds all of it. So there are three information edges terminating on a node whose authority their sources lack, and each is a candidate path from content the user did not write to an action taken in the user's name.

Note what has not entered the analysis: no attacker, no known technique, no CVE. The exposure is structural and was visible the moment the integrations were listed.

Closing them, one at a time.

Inbox to send. Anyone who can email the user can place text in the assistant's context. Control: require explicit confirmation for any send whose content or recipient derives from unsent mail. Cost: one confirmation dialogue on a minority of sends.

Documents to send. Any document the user has ever been sent can carry instructions. Control: mark retrieved text as data at the point of retrieval and refuse tool calls justified only by data-marked content. Cost: a provenance field threaded through the context assembly.

Web to send. Any page the assistant fetches can carry instructions, and the assistant chooses which pages to fetch based on content it has already read. Control: an

allow-list on outbound recipients, so that even a successful injection cannot reach an attacker-chosen address. Cost: friction when the user legitimately mails somebody new.

What this produced. Three security requirements, derived before any threat modelling, each with a named control and an honest statement of its cost. When the threat literature is consulted afterwards, it adds technique and detail but few surprises: Greshake et al. [33] describe precisely this class under the name indirect prompt injection, and observe that applications built on language models blur the line between data and instructions, so that an adversary able to place text where the system will retrieve it can act through that system without ever touching its interface.

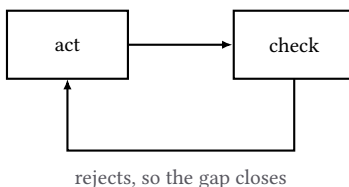
Why the structural reading is worth doing anyway. Attack techniques change every few months. The condition that makes any of them possible, an information edge landing on a node whose authority its source lacks, does not change, and it is checkable with a pencil in an afternoon by somebody who has read no security papers at all.

2.5 Cycles and feedback

Everything so far has been static. Boundaries, cuts, compositions and graphs are all pictures of a system at rest, and they tell you what is connected to what. The last structural idea concerns what a system does over time, and it enters as soon as a graph contains a cycle, because a cycle means that what the system does now changes what it will be asked to do next.

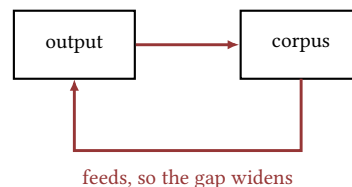
An element whose output returns to influence its own input creates a loop, and loops are where dynamics come from: behaviour that unfolds rather than being fixed at assembly. They come in two kinds. A balancing loop opposes change and seeks a setpoint, as an agent does when it retries until a check passes, or a reviewer does when rejecting bad output. A reinforcing loop amplifies, as when more usage produces more training data, which produces a better system, which produces more usage.

BALANCING, seeks a setpoint



an agent retrying until a check passes

REINFORCING, compounds



outputs re-entering the corpus they trained on

Figure 2.7. The two kinds of loop. A balancing loop opposes change and settles; a reinforcing loop amplifies and does not.

Two properties decide how a loop behaves, and both are engineering choices rather than facts about the domain. Gain is how strongly the loop responds to a discrepancy: a large correction after a small error is high gain. Delay is how long it takes before the effect of an action becomes visible to the thing that took it, which in an agent is often several steps and in an organisation is often several weeks.

From these comes the one piece of control intuition this book needs, and it can be read off a diagram without any mathematics: a balancing loop with high gain and long delay does not settle, it oscillates. An agent that cannot observe the effect of its last action for three steps will over-correct, then over-correct its over-correction, and an observer watching only the output will call the result erratic model behaviour. That description sends the engineer to the wrong layer, exactly as Chapter 1 warned. The failure is in the loop, and the remedies are loop remedies: reduce the gain by making corrections smaller, reduce the delay by getting feedback sooner, or add damping by requiring confirmation before a reversal. Chapter 5 applies all three to real harnesses.

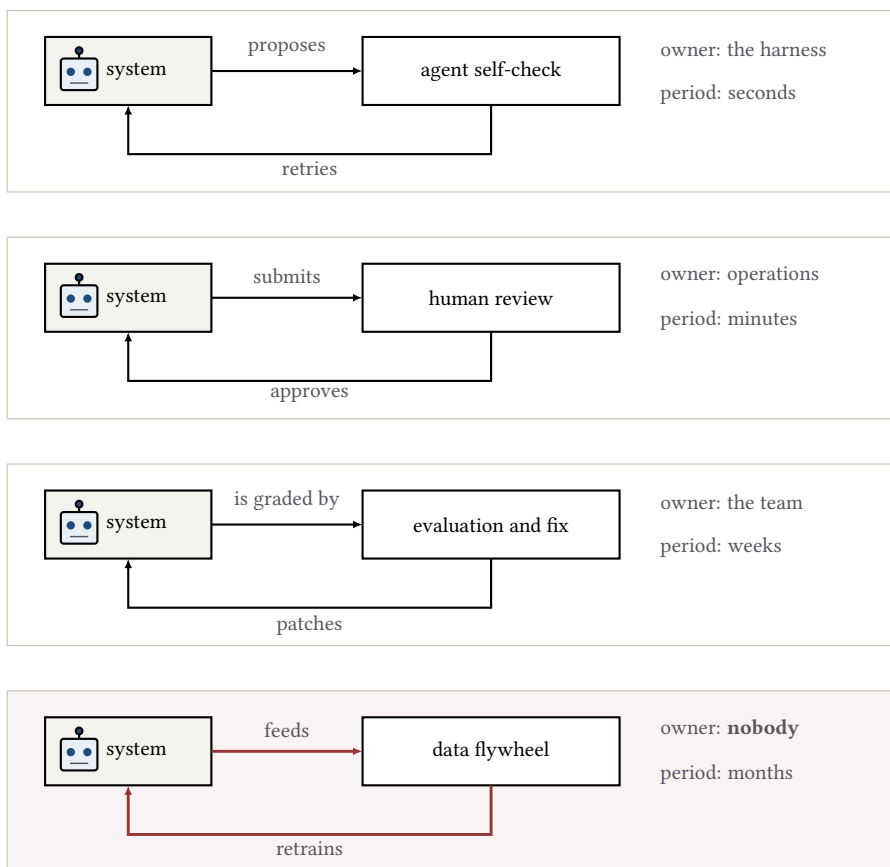
Most deployed AI systems contain at least six loops, and in our experience few teams can name them without prompting. Fewer still can say who owns each, which is the more serious gap.

Four of them are drawn in Figure 2.8. The agent's own correction loop closes in seconds and belongs to whoever writes the harness. The human review loop closes in minutes and belongs to operations. The evaluation and improvement loop closes in weeks and belongs to the team. The data flywheel, in which system output becomes training or retrieval input, closes in months and in most organisations belongs to nobody.

Two more are almost always present and almost never tracked. Users adapt to the system, changing the distribution of inputs it was verified against, over weeks, owned by nobody. And outputs get published, become training data, and return inside the next model version, over years, owned by no single organisation even in principle. This last loop has a documented failure mode. Shumailov et al. [88] show that training generative models on recursively generated data produces what they call model collapse: the tails of the original content distribution disappear, and the defect is irreversible. It is the clearest case in this book of a reinforcing loop that no participant owns, operating on a timescale longer than any product's roadmap, with a known and undesirable equilibrium.

The pattern across all of them is the lesson. The loops with owners are the fast ones, because a fast loop announces itself: somebody is watching the thing it affects. The unowned loops are the slow, reinforcing ones, which is the worst available combination, since slow means nobody will notice in time and reinforcing means it will not correct itself.

The insurance company lost four months to precisely this. Its corpus revision cycle was a loop with a period of one week and no owner on the assistant's side, and



The three loops with owners are the fast ones, because a fast loop announces itself. The unowned loop is the slow, reinforcing one, which is the worst available combination: slow means nobody notices in time, and reinforcing means it will not correct itself.

Figure 2.8. Four loops of a deployed system with their owners and periods. The pattern in the last two columns is the thing to notice.

by the time anyone traced the escalation rate back to it, the system had spent most of a year answering from a world its evaluation no longer described.

Closely related, and equally invisible to conventional monitoring, are the quantities that accumulate. Distinguishing a stock, which is a level, from a flow, which is a rate, prevents a class of error that is otherwise hard to see, because a stock can be filling steadily while every flow measurement looks normal. Context accumulates within a run. Memory accumulates across sessions. Evaluation coverage accumulates as tests are written. Technical debt accumulates whenever change outpaces cleanup, and Sculley et al. [86] argue that machine-learning systems accrue it faster than conventional software because entanglement makes every change wider than it looks. Corpus staleness accumulates whenever the world moves and the corpus does not, and that is the stock the insurance company was not watching.

One more deserves a name because it is tracked almost nowhere: unreviewed autonomy. Authority is granted to a system incrementally, each grant individually defensible, and almost no organisation has a process that removes a grant once it stops being needed. The level therefore rises monotonically, and the support assistant's administrative credential was one unit of it, sitting on the pile for four months.

All three incidents in Chapter 1 involved a stock nobody was measuring. That is not a coincidence. Stocks fail silently by construction, because a level crossing a threshold generates no event unless somebody has arranged for it to.

2.6 Structural constraints on behaviour

The chapter has been building towards a set of claims about what can and cannot be achieved by working on elements, and this section collects them.

Emergence is the appearance, at one level of organisation, of properties that do not exist at the level below. Safety is emergent, security is emergent, and reliability over a task is emergent. None of them is a property of a model, which is why asking whether a model is safe is a category error rather than a hard question. The answerable form asks whether a particular system, doing a particular job, in a particular setting, is safe enough for a stated use, and that question has an answer evidence can support.

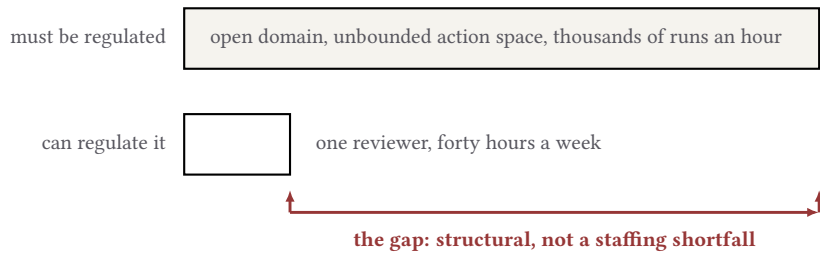
The complement of emergence is downward causation: constraints imposed from a higher level on what a lower level may do. A sandbox constrains the agent. A permission model constrains the tool call. An interlock constrains the action. In each case the constraint lives above the thing constrained and does not depend on its cooperation. This is the structure of the entire safety argument of this book, and it is why Chapter 13 derives constraints rather than seeking a better behaved component. A constraint enforced from above is enforced. A constraint written into a prompt is a request addressed to the element whose behaviour is in question, evaluated by that element, and overridable by anything else in its context. The support assistant's revised system prompt was a request. The scoped credential would have been a constraint.

Next, a limit on what oversight can achieve, and the sharpest instrument in the chapter. Ross Ashby's law of requisite variety [2] says that a regulator must command at least as much variety as the system it regulates, where variety counts distinguishable situations. Stated plainly: you cannot control what you cannot distinguish.

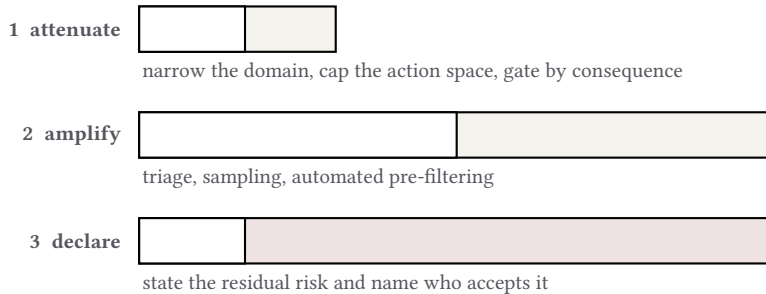
Apply it to the arrangement most organisations actually deploy, a human reviewer approving an agent's actions. The system's variety is very large: unbounded natural language inputs, an open action space, thousands of runs an hour. The reviewer's is small and fixed: one person, forty hours a week, with attention that measurably degrades somewhere around the twentieth near-identical approval.

Figure 2.9 draws this as length, because length makes clear what an arrow labelled "gap" conceals. The gap is not a shortfall to be closed by hiring, and it is not evidence

THE SITUATION



THE ONLY THREE HONEST RESPONSES



Attenuating shortens the upper bar; amplifying lengthens the lower one; declaring leaves both and records what remains. A design that does none of the three has an oversight *claim*.

Figure 2.9. Variety drawn as magnitude. The gap between what must be regulated and what can regulate it is a structural relation, not a staffing shortfall, and it admits exactly three honest responses.

that the reviewer is insufficiently diligent. It is a relation between two quantities, and it admits three responses and no others.

You can attenuate the system's variety by narrowing the operating domain, capping the action space, or routing by consequence so that only some actions need review at all. This is what good architectures do, and it is the only response that changes the shape of the problem rather than its size. You can amplify the regulator's variety through triage, sampling, better interfaces and automated pre-filtering, which is real and bounded, and which carries a subtlety: automated amplification introduces a new element that can itself fail, so the regulator's effective variety now depends on something that needs verifying in its own right. Or you can accept the gap and declare it, stating the residual risk and naming who accepts it. That is the least satisfying response and entirely legitimate. What is not legitimate is the fourth option, which is to do none of the three and call the result human oversight.

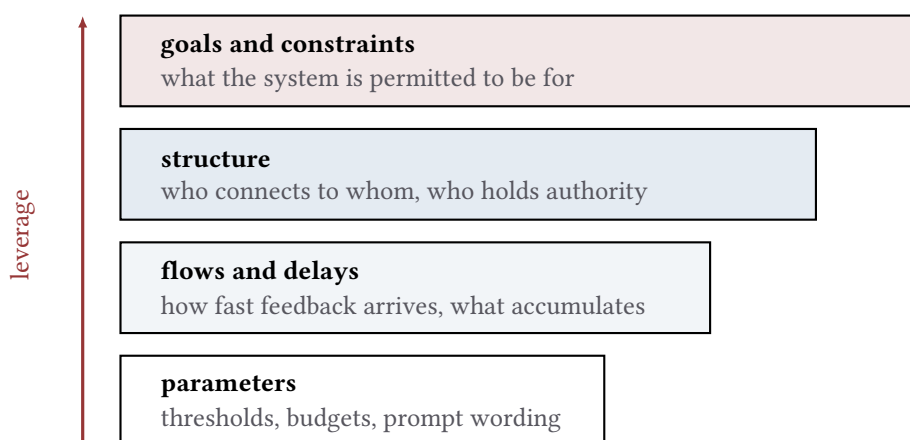
Two further limits are worth stating because they bound what any amount of engineering can deliver.

Optimising a system for expected conditions buys performance there and purchases brittleness elsewhere; the trade is relocated, never removed. A system tuned to its benchmark is a system tuned away from its tail, and because the operating

domain is open, it will meet that tail. This is also the honest answer to the question hanging over the whole book, which is whether better models dissolve the problem. Better models move the robust region, sometimes dramatically. They do not remove the fragile one, because the trade is conserved, and what changes is where the brittleness sits rather than whether there is any.

And there is a distinction between two words that ordinary speech treats as synonyms. A complicated system has many parts and predictable interactions, like a jet engine or a compiler, and can be understood by understanding its parts. A complex system has interactions producing behaviour not derivable from the parts, like an ecosystem or a market, and must be studied at its own level. AI systems are engineered like complicated systems and behave like complex ones, which is another way of stating Section 2.3's thesis and which explains a frustration that runs through this book: specification, decomposition and unit verification are all necessary here and all insufficient, which is more uncomfortable than their being wrong, because there is nothing to discard and no replacement to adopt.

Given all of this, where should an engineer push? Not all interventions are equal, and the differences are large enough to rank.



Most AI remediation happens on the bottom rung, because it is the cheapest and can be shipped without anyone else's agreement. "We updated the prompt" is a parameter change; "we removed the credential" is a structure change.

Figure 2.10. Where an intervention actually changes behaviour, ranked from weakest to strongest. Most AI remediation happens on the bottom rung.

At the bottom sit parameters: thresholds, budgets, retry counts, the wording of a prompt. They are easy to change, which is why they absorb most remediation effort, and they are weak, because they adjust the degree of a behaviour rather than its possibility. Above them sit flows and delays, which govern how fast feedback arrives and what accumulates while it does not. Above those sits structure: who is connected

to whom, and who holds what authority. At the top sit goals and constraints, which settle what the system is permitted to be for.

Chapter 1's post-mortem can now be described exactly. A stricter system prompt is a parameter change. A model upgrade is a parameter change, an expensive one. Scoping the credential is a structure change. Deciding that a support assistant drafts replies and never executes statements against production is a goals change. The first two alter how often the incident occurs; the last two alter whether it can.

Two questions close the chapter, and they are asked of every system in the rest of the book. Observability asks whether you can determine internal state from what the system emits, which in practice means whether you can reconstruct after the fact why a run did what it did. For most deployed systems the answer is no, and it is discovered during the first incident, at the worst moment, by somebody who needs it urgently. Controllability asks whether you can drive the system to a state you want: stop it mid-run, steer it, roll it back. Here too the answer is often no, for a reason no engineering removes, which is that the action has already been taken. The email has been sent, the statement has executed, the table is gone.

Both are design properties. A system does not become observable by accident, and instrumentation added after an incident cannot explain that incident.

Example 2.6 | Ranking seven fixes by leverage

Setting. The support-ticket agent of Chapter 1 has executed a destructive database statement, triggered by an instruction embedded in a customer ticket, using an administrative credential granted during integration. The team has seven candidate fixes on the table and budget for two.

The ranking instrument. Figure 2.10 orders interventions from weakest to strongest: parameters, then flows and delays, then structure, then goals and constraints. Place each fix on the ladder and ask what it changes.

Fix	Rung	What it changes	Effort
Reword the system prompt	parameter	how often text is read as instruction	hours
Upgrade the model	parameter	the same, by a larger factor	weeks
Filter for DROP	parameter	blocks one statement of several	hours
Add a confirmation gate	flows	inserts a delay in which a human can intervene	days
Scope the credential	structure	the destructive action becomes unavailable	2 weeks, another team
Split read and write agents	structure	untrusted input never shares a process with write authority	weeks
Forbid database writes entirely	goals	the capability leaves the concept of operations	a decision

What the team actually shipped. The top three rows. Together, perhaps a hundred-fold reduction in incidence, which on 400 tickets a day still leaves a recurrence within the lifetime of the system, and which depends on an adversary who does not adapt.

The observation worth carrying away. Look at the effort column against the rung column. The two parameter fixes take hours and can be shipped by the team alone. The two structure fixes take weeks and require another team's cooperation. The goals fix requires a product decision that somebody senior has to make and defend.

This is the ordinary shape of the choice, and it is not a story about laziness. *The fixes that change possibilities are almost always the ones that require somebody else's agreement, and the fixes that change rates are almost always the ones you can ship alone.* A team under incident pressure will ship what it controls, and will do so with entirely good intentions.

The cost comparison nobody made. Scoping the credential was requested in month one and blocked behind a two-week platform ticket, later closed as a duplicate. The incident cost forty minutes of lost writes, a customer notification, a regulatory conversation, and roughly a quarter of one engineer's attention. Set against two weeks of waiting in month one, the arithmetic is not close.

How to use this in a post-mortem. For every proposed fix, name its rung before estimating its effort, and record the rung in the post-mortem document. When the fix is a parameter and the failure was structural, the document should say in those words that the team has bought time rather than safety, and should record what the structural fix would have been and what blocked it. That record is what turns a single incident into a budget line in the next planning round.

2.7 The consolidated vocabulary

The terms introduced in this chapter are collected here with the chapter that first puts each to work, because a vocabulary is only worth learning if it is used, and each of these is used in anger later rather than merely defined. A reader who can state what every row means on a system they know has the equipment the rest of the book assumes.

Term	Meaning	First used in anger
Boundary	what is inside the system	Ch. 8, ConOps
Cohesion, coupling	the quality of a decomposition	Ch. 8, architecture
Composability	the parts fit together	Ch. 7, interoperability
Compositionality	the whole follows from the parts	Ch. 10, V&V
Authority flow graph	who acts on whose credentials	Ch. 14, trust boundaries
Hub	a shared dependency	Ch. 12, correlated failure
Balancing, reinforcing	a loop that regulates, or amplifies	Ch. 5, loop pathologies
Gain and delay	strength and lateness of feedback	Ch. 5, oscillation
Stock	what accumulates	Ch. 11, sustainment
Emergence	properties of the arrangement	Ch. 13, safety
Downward causation	constraint imposed from above	Ch. 13, safety constraints
Requisite variety	a regulator must match what it regulates	Ch. 6, oversight
Robust yet fragile	performance purchased with brittleness	Ch. 10, benchmarks
Leverage point	where an intervention actually matters	throughout

What this chapter established

- ▶ Decomposition buys context isolation, authority partition, independent verification and parallelism, and costs coordination, seams, brief fidelity and overhead. Both sides are real and the trade should be made explicitly.
- ▶ Composability is not compositionality. Elements can be assembled without the properties of the assembly following from properties of the parts, and that gap is where emergent failures live.
- ▶ A system has at least three graphs over the same elements: control, data and authority. They disagree, and the disagreements are where design defects hide.
- ▶ Reinforcing loops amplify and balancing loops oppose. The dangerous combination is a reinforcing loop that is slow and unowned, because slow means nobody notices and reinforcing means it will not correct itself.

- Requisite variety bounds what oversight can achieve: a regulator must command at least as much variety as what it regulates. You cannot control what you cannot distinguish.

Exercises

- 2.1 [D] Decompose one AI system three ways, functionally, by trust boundary, and by rate of change. State which cut you would build to and why. For each of the other two, name one fault that cut reveals and the chosen cut hides.
- 2.2 [D] Take two elements you have measured separately. Compose them. *Before* measuring the composite, write down your prediction of its success rate and your reasoning. Then measure it. Report the error and account for it in the terms of §2.3, which assumption of compositionality failed?
- 2.3 [D] Draw the information-flow, control-flow, and authority-flow graphs for a system you work on. Find one place where they disagree. Explain the consequence, and state what would have to change for them to agree.
- 2.4 [D] Identify every feedback loop in a system you know. For each: balancing or reinforcing, its approximate time constant, and its owner. Report the ones with no owner. If all of them have owners, look again, you have missed the content loop or the user-behavior loop.
- 2.5 [D] Apply requisite variety to a human-oversight arrangement in your organization. Estimate the variety on each side with explicit assumptions. State which of the three responses in §2.6 the design actually takes. If the answer is none, write the residual-risk statement it should have.
- 2.6 [A] A system has one shared model behind n nominally independent checks, each with false-pass rate q . A colleague claims the probability all n pass erroneously is q^n . (a) State the assumption required. (b) Using Figure 2.6, explain why it fails. (c) Give a qualitative bound on how much worse than q^n the truth could be, and say what measurement would settle it.
- 2.7 [L] Instrument one reinforcing loop in a running system. Estimate its gain and its delay. Predict from §2.5 whether it will oscillate. Run it and report whether your prediction held.
- 2.8 [D] Take the last three remediations your team applied to an AI system. Rank each by leverage (Figure 2.10). For any that were parameter-level fixes to structural problems, write the structure-level fix that was not applied, and estimate its cost.

Notes and further reading

On decomposition (§2.2). Near-decomposability is Simon [89], and the chapter on the architecture of complexity repays reading in the original; his account of why complex systems that survive are assembled from stable subassemblies is the clearest statement of

why decomposition is not merely convenient. The cohesion and coupling vocabulary is standard in software engineering and we use it without modification.

On composition and entanglement (§2.3). The distinction between composability and compositionality is standard in programming-language theory and formal methods; our contribution is only to insist on it for AI systems, where it is routinely elided. The strongest empirical statement of the underlying phenomenon remains Sculley et al. [86], whose CACE principle (*Changing Anything Changes Everything*) and observation that “no inputs are ever really independent” predate agentic systems by a decade and describe them accurately. Readers who find our treatment abstract should read that paper first.

On multi-agent composition. Cemri et al. [15] report that multi-agent systems frequently show minimal gains over single-agent baselines, and attribute the shortfall to organisational rather than component failures. This is the empirical counterpart to our claim that hierarchical composition does not pay for itself, and Chapter 6 develops the architectural consequences.

On feedback and the loops of §2.5. The balancing and reinforcing vocabulary, along with stocks, flows and the leverage-point ranking of §2.6, follows Meadows [63], whose treatment remains the most accessible. The leverage ranking is heuristic: it is useful for structuring an argument and should not be mistaken for a theorem. The content loop has a documented failure mode in Shumailov et al. [88], who show that recursive training on generated data causes the tails of the distribution to disappear irreversibly; we know of no comparable empirical treatment of the user-behaviour loop, which we regard as a gap.

On bounded attention as a modelling assumption (§2.6). The observation that a regulator’s variety is limited by what it attends to, rather than by what it could in principle distinguish, has a formal treatment in the security literature. Chen and Zhu [16] model interdependent risk management where each participant attends to only a sparse subset of its neighbours and optimises against the risk it can perceive, which is the same structure as an overseer who can read only part of what they are accountable for. We use requisite variety qualitatively in this chapter; readers who want the equilibrium analysis should start there.

On requisite variety (§2.6). The law is Ashby [2]. Its application to human oversight of AI systems is ours, and we state it as a structural constraint rather than an empirical finding. Readers should note that the three responses we enumerate are exhaustive only given the framing; a fourth response, raising the regulator’s variety by automating it, is really the second, and carries the difficulty that the automation is itself an element requiring verification.

On adversarial structure (§2.4). The three-graph reading is our own device, but the vulnerability it exposes is precisely that described by Greshake et al. [33], who observe that LLM-integrated applications blur the line between data and instructions. We would emphasise that the structural reading is available to an engineer with no security background, which the attack literature, by its nature, is not.

Open at the time of writing. How far non-compositionality is fundamental, as against an artifact of current interfaces, is genuinely open. Work on compositional certificates and assume-guarantee reasoning for learned components is active and unresolved, and the epilogue records what a solution would have to provide.

Deliberately omitted. Formal control theory, quantitative network science, and system-dynamics simulation. Each is a legitimate deepening of material this chapter treats qualitatively, and none is required for what follows.

Re-deriving the Systems Engineering Process

Learning Objectives

1. Name the standard SE technical processes and say what artifact each produces.
2. For each, state whether it survives intact for AI systems, requires rework, or has no adequate analogue, and give the reason.
3. Explain why the V-model's right-hand side cannot close, and what replaces it.
4. Describe a system in the L0-L6 reference architecture.
5. Use the layer $\times$ lifecycle matrix to find engineering concerns you have not addressed.

Why this chapter is here

Chapter 1 argued that AI systems break classical assumptions. Chapter 2 supplied the vocabulary to say how. This chapter does the audit: it takes the standard process set seriously, process by process, and marks what happens to each. This is the book's central claim, and the rest of the book is its elaboration. If you disagree with the audit, you will disagree with the book's structure, which is the correct place to have the argument.

3.1 The standard process set

Systems engineering is not a body of opinion. It is a body of practice, and the practice has been written down. ISO/IEC/IEEE 15288 [47] and the INCOSE handbook [41] define a set of technical processes that a project is expected to perform, each producing an artifact that gets reviewed and placed under configuration control rather than merely an activity that happens.

Stripped to essentials, the sequence runs as follows. Business or mission analysis establishes the problem and why it is worth solving. Stakeholder needs and requirements capture what the people who will live with the system actually need, in their own terms. System requirements definition converts those needs into statements an engineer can build and test against. Architecture definition fixes the elements

and their arrangement; design definition says how each element will be realised; implementation produces them; integration assembles them. Verification produces evidence that the system was built right, meaning to its specification. Transition puts it into operational use. Validation produces evidence that the right system was built, meaning that it serves the need. Operation, maintenance and disposal carry it through service and out the other end.

Readers with a systems background can skim that paragraph, since none of it will be new. Readers meeting the process set for the first time should read Appendix A before going further, because the audit that follows assumes each of those thirteen processes is familiar enough to be argued with. Everyone should pause on two points, because both are routinely lost in the translation between the two fields, and losing them is expensive.

The first is that verification and validation are different processes answering different questions. Verification asks whether the system conforms to its specification. Validation asks whether the specification was the right one. A system can pass verification completely and fail validation completely, and the insurance assistant of Chapter 1 did exactly that: it conformed to everything anyone had written down, and what had been written down described a corpus that no longer existed. The AI field calls both of these activities “eval”, which destroys a distinction that this chapter will argue is the most important one in the discipline.

The second is that each process produces a reviewed artifact. Not a dashboard, not a shared understanding, not a number in a slide. An artifact with an owner, a version and a review record. A great deal of what follows concerns artifacts that AI projects do not currently produce at all.

3.2 The audit

The question this chapter exists to answer is what happens to that process set when the system contains a component that is statistical, open domain, opaque, supplied by somebody else and exposed to an adversary through its ordinary data path.

The answer is neither “nothing” nor “everything”. Taking the thirteen processes one at a time, three survive essentially unchanged, nine need rework of a kind that is specific and describable, and one has no adequate analogue at all. That ratio is the argument of this book, and it is worth stating before the detail, because both of the convenient positions are wrong. Systems engineering does not already solve this, and AI does not need a new discipline built from nothing.

3.2.1 Processes that survive unchanged

Business and mission analysis is **intact**. The question of what problem is being solved, for whom, and at what value, is not affected by the technology used to solve it. If anything the process deserves more weight here than elsewhere, because AI projects are unusually prone to beginning from a capability rather than from a need,

and the discipline of writing down the need first is the cheapest available defence against that.

Transition is **intact**, with one addition. Readiness, operator training and the first week in service work as they always have. What changes is that transition recurs: every model migration is a partial re-transition, with its own readiness question and its own training implications, and Chapter 11 treats it as a scheduled event rather than an interruption.

Disposal is **intact**. Data disposition, user transition and the obligations that outlive shutdown are what they have always been, and they are neglected in AI projects at about the same rate as everywhere else.

3.2.2 Processes requiring rework

The reworks fall into four groups, and grouping them shows something the process-by-process list conceals: they are not nine independent problems but four, appearing in nine places.

Specification cannot be completed, so it must be bounded instead. Stakeholder needs and requirements is **rework**. Classical elicitation assumes stakeholders can describe what they need, and for AI systems they can describe what they want the system to do while being unable to describe what they need it never to do, because the failure modes are unfamiliar. Nobody at the support company would have thought to say that the assistant must not treat text inside a ticket as an instruction. The possibility had not occurred to them, and there is no elicitation technique that extracts a requirement a stakeholder cannot imagine. Needs therefore have to be partly constructed by the engineer, through structured hazard and misuse discovery, and then confirmed by the stakeholder rather than collected from them.

System requirements definition is **rework** for a related reason. The process survives; the form of a requirement does not. “The system shall classify the document correctly” is not verifiable when behaviour is a distribution, so requirements acquire quantifiers, populations and confidence levels: correct in at least ninety-five per cent of cases drawn from a stated domain, with ninety per cent confidence, on a held-out set of stated size. Three further changes follow. The operating domain has to be declared explicitly, since it cannot be enumerated, and behaviour outside it specified rather than left undefined. Negative requirements need a stated enforcement point, because “the system shall never execute a destructive statement” is discharged by an architectural control and not by a sentence in a prompt. And every requirement must name the evidence that will discharge it at the moment it is written, which is ordinary practice elsewhere and honoured mostly in the breach here.

You constrain a supplied element rather than designing it. Architecture definition is **rework**, and it becomes more important rather than less, because Chapter 2 established that an AI system’s dependability lives in its arrangement.

Two things change. The criteria for decomposition are no longer purely functional: trust boundary, consequence, verifiability and rate of change all have claims, and Section 2.3 showed that the security architecture is invisible in a functional cut. And the architecture now has to carry the autonomy decision explicitly, with the evidence that justifies it, because how much authority the system holds is an architectural property and not an operational setting.

Design definition is **rework** in a way that has no classical counterpart. Design classically specifies how an element will realise its requirements. You do not design a supplied model. You characterise it and you constrain it, and the design activity relocates entirely to what surrounds it: the prompt, the context strategy, the tool schemas, the loop, the authority envelope. The consequence is that prompts, policies and tool definitions are design artifacts, which makes them configuration items subject to change control rather than strings in a source file. Most organisations do not treat them that way, and Chapter 9 is about the cost of that.

Configuration extends to something you do not own. Implementation is **rework**. Much of the work is data and configuration rather than code, which shifts what has to be versioned and tested. But the genuine rework is that classical configuration management assumes you control your configuration items, and here the central item is a model under a supplier's control, on a supplier's release schedule, with a deprecation date you do not set. There is no clean classical precedent for a configuration item that changes without your consent and without any interface you can see changing. This is the most under-managed risk in deployed AI systems, and it is the reason the rate-of-change decomposition of Figure 2.2 earns its place.

Maintenance is **rework** for the same underlying reason, seen later in the lifecycle. Classical maintenance repairs degradation and fixes defects that were always latent. AI systems exhibit a different phenomenon: they decay while nobody touches them, because the world moves and the substrate moves. Prompt rot, corpus staleness, tool interface drift and model deprecation are all forms of it. Maintenance has to be scheduled against decay and not only against defects, which means somebody has to own a calendar rather than a queue.

Evidence about the parts does not transfer to the whole. Integration is **rework**, and it carries more weight here than in conventional practice. Chapter 2 gave the reason: the composite's properties genuinely do not follow from the parts', so integration is not assembly followed by a smoke test. It is the first point at which most of the system's real behaviour exists at all. It is made harder by interfaces that are natural language and therefore undeclared, with no compiler to catch a mismatch, which is why interface verification has to precede system verification rather than being folded into it.

Verification is **rework**, and this is where the customs pipeline earns its place in Chapter 1. The process survives and its methods change completely. Inspection is

unavailable, because the component is opaque. Analysis is severely limited, because you cannot derive composite behaviour from element behaviour. Test therefore carries nearly the whole burden, and test now needs sample sizes, confidence intervals and holdout discipline that conventional software testing does not. Most importantly, element verification does not discharge system verification. Classically it largely does, which is what makes the V-model economical. Here it does not, and a system that has only ever been verified part by part has not been verified. The pipeline's three green milestone reviews were all honest and none of them was evidence about the pipeline.

Operation is **rework** because it absorbs work that used to belong to development. Drift detection, continuous evaluation, guardrail monitoring and the routing of production failures back into the requirements and the evaluation set are all operational activities now, which means the operations function has become part of the assurance function. Chapter 11 works out what that implies for staffing and for who is on call.

3.2.3 The process with no classical analogue

Validation is **no analogue**, and this is the sharpest finding in the audit. It is also the one most likely to be resisted, because every reader has signed a validation report at some point and would like to believe the signature still means something.

Classical validation establishes that the right system was built, in its operational environment, against real stakeholder need. Then it is done. The evidence stands until the system changes or its environment changes, and both of those are under configuration control, which is what makes “done” meaningful. A validation report has a date on it and the date is not a problem.

For an AI system, validation evidence expires, and it expires for three independent reasons, any one of which would be sufficient. The model changes underneath the system on a supplier's schedule. The operating domain keeps arriving, bringing inputs the validation never saw and could not have enumerated. And the adversary adapts, which means a security validation describes a defender's position against yesterday's attacker.

The insurance company's validation was correct on the day it was produced and was still being cited eleven months later, by which time it described a system that no longer existed in a world that had moved on. Nobody did anything wrong. There was no process whose job it was to notice that a validation result had aged.

What replaces one-time validation is continuous assurance: a loop with detectors, re-verification triggers, a cadence and an owner, engineered deliberately rather than assembled after the first incident. No classical process produces this. That is why the mark is **no analogue** and not **rework**: there is nothing here to rework, because there is nothing that does the job.

3.2.4 Summary of the audit

Process	Verdict	Why
Mission analysis	intact	unaffected by the technology
Stakeholder needs	rework	stakeholders cannot state the failure modes
System requirements	rework	behaviour is distributional; the domain must be declared
Architecture	rework	new decomposition criteria; autonomy is architectural
Design	rework	you constrain a supplied element rather than design it
Implementation	rework	configuration management of an item you do not own
Integration	rework	undeclared interfaces; non-compositionality
Verification	rework	element verification does not discharge the system
Transition	intact	unchanged, but it recurs at every migration
Validation	no analogue	evidence expires, and nothing classical replaces it
Operation	rework	absorbs part of the assurance function
Maintenance	rework	decay without change
Disposal	intact	unaffected

Nine of thirteen need rework, one has no analogue, three are intact. The discipline is not obsolete. It is nine-thirteenths applicable and one-thirteenth missing, and the missing thirteenth is the one that decides when you are allowed to say a system is fit for use.

3.3 Closure failure in the V-model

The V-model is the picture most engineers carry of that process set, and it is worth seeing precisely where it fails, because the failure is structural rather than a matter of emphasis.

On the left, specification cannot be written exhaustively. This is not a matter of writing it badly or of insufficient effort. In an open domain the specification is incomplete by construction, which means you are writing a deliberately partial one, and the partiality has to be managed rather than ignored. Declaring an operating domain is exactly that management: it converts “we did not think of that” into “that is outside the domain, and here is the specified behaviour when it occurs”.

On the right, the arm cannot close, for the reason the audit gave. Validation evidence expires, so there is no state of the system in which the V is complete and the work is finished.

What replaces the V is not a different diagram so much as a different shape. The right-hand arm, instead of terminating in a signed validation report, feeds a loop

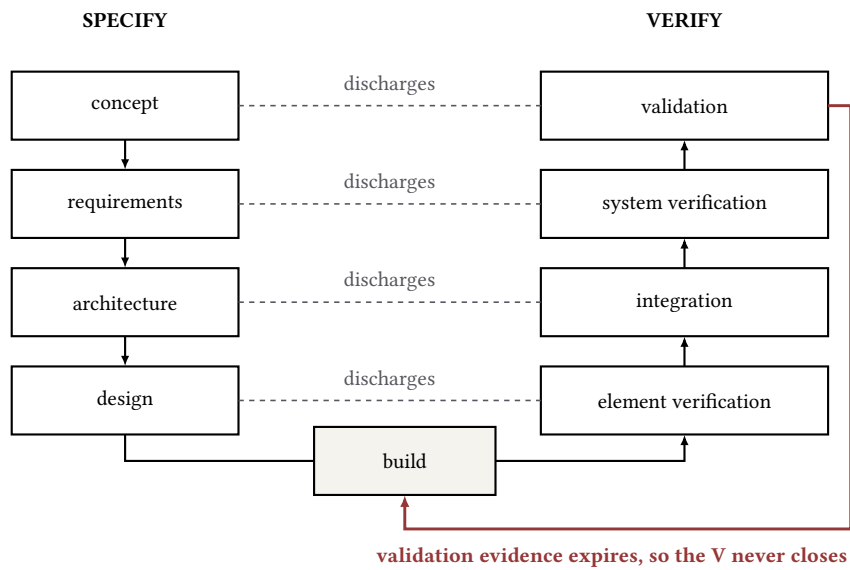


Figure 3.1. The V-model drawn as two columns. The dashed lines are its central promise: each verification activity on the right discharges the specification opposite it on the left. For AI systems the promise fails at both ends.

that continues for as long as the system is in service. Chapter 11 is about that loop, and the claim that assurance never closes is a structural consequence of this figure rather than a slogan.

One practical consequence is worth stating now, because it changes a decision that teams make every week. The release decision is not “verification passed”. It is a compound statement: the residual risk is acceptable for this use, and the assurance loop that will keep it acceptable exists, is staffed and has an owner. A system that satisfies the first clause and not the second is a system that was safe on Tuesday. Chapter 13 gives that decision its artifact and says who is entitled to sign it.

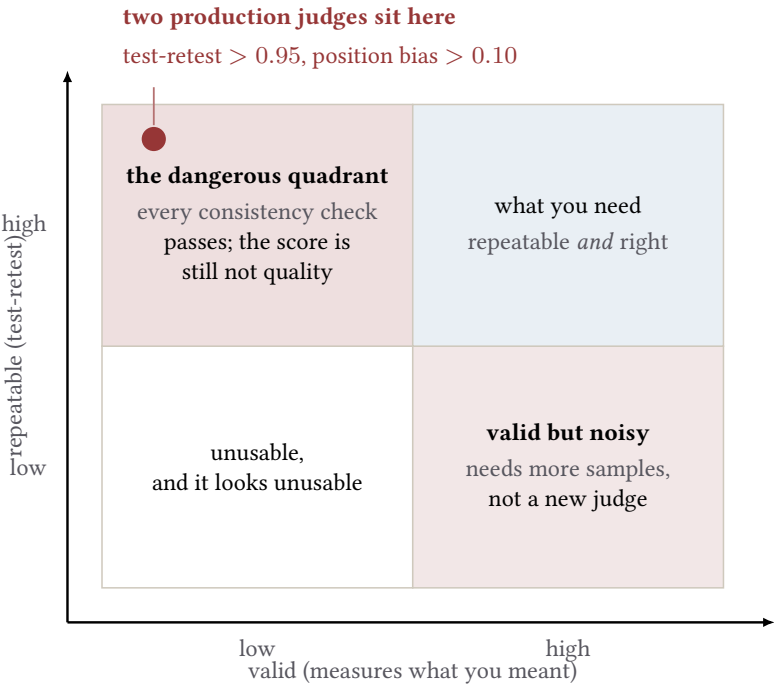
Example 3.1 | Reliability without validity in an automated judge

Setting. Evaluating an AI system at scale means grading outputs that have no single correct answer, so teams increasingly use a second model as the grader. The question this example asks is the one this chapter is about: does a judge’s evidence establish that the system was built right, or only that the grader is steady?

The headline finding. Norman et al. [69] evaluate 21 judges from nine providers across three benchmarks, on roughly 541,000 individual judgments. Two of the production judges they examine show *test-retest reliability above 0.95 alongside position bias above 0.10*.

Read that slowly. A judge with test-retest reliability above 0.95 returns the same verdict every time you ask it. Every consistency check you run will pass. It is, in the plain

sense of the word, reliable. And a position bias above 0.10 means its verdict changes when you reorder the candidates, which is to say that part of what it is measuring is presentation rather than quality.



Repeatability and validity are independent. Only one of them is cheap to measure, which is why teams measure it and stop.

The supporting results. Three further findings bear on how a team should use a judge.

Measure	Reported	Consequence for your pipeline
Kappa deflation, MT-Bench	33 to 41 pts	“85 % agreement” may be $\kappa \approx 0.45$; report κ , not raw agreement
Rank stability across benchmarks	up to 14 places	a judge chosen on one benchmark may be mid-table on your workload
Verbosity bias	< 0.011	a widely feared bias was small here; received wisdom also needs measuring

Why this belongs in a chapter about process. Consistency is a property of the instrument. Validity is a property of the relationship between the instrument and the thing you want to know. The audit of §3.2 marks verification **rework** and validation

no analogue for precisely this reason, and here the distinction appears inside a single tool: the judge can be verified and still not be valid.

Three checks, none expensive.

1. Validate the judge against human labels *on your own workload* before trusting it, and report Cohen's κ rather than raw agreement.
2. Test for position bias by re-running a sample with the candidate order reversed. A judge that changes its mind is telling you that its score is partly about presentation.
3. Re-validate whenever the judge model or version changes, because the rank instability above says the choice does not carry across workloads.

The general lesson. An instrument that is repeatable is not thereby correct, and repeatability is far easier to measure, which is why teams measure it and stop. Whenever a stable number reassures you, ask what would have to be true for it to mean what you are taking it to mean, and then go and check *that*.

3.4 Interfaces as the unit of engineering

Classical practice concentrates attention on interfaces because that is where defects concentrate, which is why interface control documents exist as a category. AI systems make the problem worse in a specific and underappreciated way: their interfaces are natural language, and natural language interfaces are undeclared by default.

Four interfaces recur throughout this book, and not one of them is a conventional interface control document. Each carries information that determines behaviour, each is edited routinely, and each is usually undeclared, unversioned and unowned.

The context interface is what the model is shown. In most systems it is unversioned, unreviewed and assembled at runtime by code scattered across several files, and Chapter 2 noted that because every part reads and writes it, it behaves as a global variable rather than as an interface.

The tool interface is the schema, the description and the error messages of each available function. Section 1.1 noted that its description is read by the model at runtime, so documentation and specification are one artifact. The consequence for *process* is the one this chapter cares about: an interface specification that any engineer may edit in a pull request about something else, with no review gate and no test covering its effect, is not under configuration control in any sense the discipline would recognise.

The delegation interface is what one agent passes to another. In practice it is usually a free-text string carrying no constraints, no provenance and no record of what has already been tried, which is why Chapter 6 treats the handoff as the place where information dies.

The oversight interface is what a human is shown in order to be accountable for what the system did. It is designed last, when it is designed at all, and Chapter 2’s requisite variety argument explains why designing it last is fatal.

Example 3.2 | Auditing the four interfaces of a working agent

Setting. Any deployed agent has four interfaces that classical practice would expect to find under configuration control, and that AI practice almost never puts there. This example is an audit you can run on your own system in an afternoon.

The audit. For each interface, record four facts.

Interface	What it carries	What to record
Context assembly	everything the model is shown	schema, version, owner, last review
Tool definitions	names, argument schemas, descriptions, error text	the same four
Delegation brief	what one agent passes to another	the same four
Oversight view	what a human is shown in order to approve	the same four

What teams typically find. The tool interface has a schema, because the runtime refuses to start without one. It rarely has a version, so nobody can say which description the model was reading last Tuesday. It rarely has an owner, so it is edited by whoever is nearest to the defect. The other three interfaces usually have none of the four.

Why the missing version is not a documentation quibble. Consider what a tool description is. It is a specification, written in natural language, that the model reads at runtime in order to decide whether and how to call the tool. Editing it changes system behaviour, immediately, in production, with no build step and no test gate.

In any other engineering discipline, an unversioned specification that anyone may edit and that changes runtime behaviour would be recorded as a defect in the configuration management plan. Here it is the normal case, and it usually lives in a docstring.

A concrete consequence. A team changed a tool description from “search the knowledge base” to “search the knowledge base for policy questions” in order to discourage a misuse. Tool-call volume on that tool fell by a third, including on cases the team wanted. Nobody connected the two for six weeks, because the change was a one-line docstring edit in a pull request about something else, and no test covered tool selection.

The rule. If an interface in your system is a string with no schema, no version and no owner, it is not an interface. It is a hope. Promote all four to configuration items: give each a schema where one is possible, a version always, an owner always, and a place in the change-control process. The delegation brief and the oversight view will resist,

because they have never been treated this way, and those two are where Chapters 6 and 13 find the most damage.

3.5 The layered reference architecture

The audit tells you what has to change in each process. It does not tell you where in a system any given concern lives, and without that a process list becomes a checklist that everyone agrees with and nobody can act on. The next two devices supply the missing coordinates.

LAYER	WHAT IT ADDS	THE FAILURE CLASS IT INTRODUCES	CHAPTER
L6	Ecosystem agents under other operators' authority	counterparties you cannot test, and may not be able to observe	Ch. 7
L5	Institution procurement, audit, accountability	accountability gaps, procurement risk, externalities	Ch. 6
L4	Human-AI oversight, handoff, escalation	automation bias, review fatigue, unclear handoff	Ch. 6
L3	Agent organisation delegation, coordination	coordination failure, lost delegation context	Ch. 6
L2	Agent loop, harness, authority envelope	non-termination, goal drift, compounding error	Ch. 5
L1	Context and tools state, memory, effect on the world	retrieval failure, staleness, tool misuse	Ch. 4
----- L1 is the first layer that can change the world -----			
L0	Model the supplied stochastic element	fabrication, refusal, capability shortfall	Ch. 4

Ochre tags mark the two layers you do not control: the model you are supplied, and the agents you meet.

Figure 3.2. The seven layers used throughout the book, with the chapter that treats each. Every layer adds capability, and inseparably a class of failure the layer below could not exhibit; the third column names it. The two ochre tags are the layers you do not control.

The layers are not an arbitrary slicing. Each one adds capability and, inseparably, adds a class of failure that the layer beneath could not exhibit. At L0 the model can fabricate, refuse or fall short of a capability. At L1 context and tools introduce retrieval failure, staleness and tool misuse, and something more important: this is the first layer that can change the world, so it is the first layer at which consequence exists. At L2 the agent closes a loop, which introduces non-termination, goal drift and compounding error. At L3 an organisation of agents introduces coordination

failure, lost delegation context and diffusion of responsibility. At L4 the human relationship introduces automation bias, review fatigue and unclear handoff. At L5 the institution introduces accountability gaps, procurement risk and externalities. And at L6 the ecosystem introduces counterparties you do not control, cannot test and may not be able to observe.

The engineering summary is the line at the foot of the figure, and it is worth memorising because it assigns four different bodies of practice. You procure L0. You engineer L1 through L3. You negotiate L4 and L5. And you cannot test L6.

3.6 Measures of effectiveness and measures of performance

Classical practice distinguishes measures of effectiveness, which say whether the system achieves the stakeholder's purpose, from measures of performance, which say how the system behaves technically. The distinction is worth defending here because the field has a strong pull toward the wrong one.

A benchmark score is a measure of performance, and frequently a poor one, because the benchmark's distribution is not the deployment's, which is the robust-yet-fragile trade of Chapter 2 arriving in a project status report. The measure of effectiveness is whether tickets got resolved, whether the clinician's documentation burden fell, whether losses from fraud declined.

The gap between the two is where AI projects die, and they die in a characteristic way: the measures of performance improve steadily while the measures of effectiveness do not move, and the project is defended with the former for two quarters longer than it should be. The insurance assistant is the pure case, with a performance measure that stayed at ninety-four per cent while the effectiveness measure, escalation rate, moved against it for four months.

When correctness is distributional, measures of effectiveness have to be stated distributionally too, and measured in operation rather than in a test harness. That makes them a Chapter 11 activity rather than a Chapter 10 one, which is an unfamiliar place for them to live.

Example 3.3 | A performance measure that diverged from its effectiveness measure

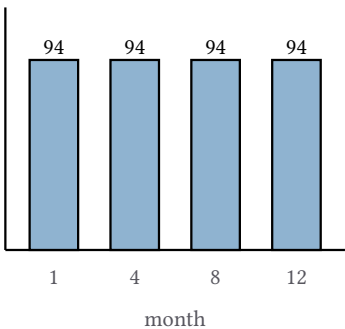
Setting. The insurance policy assistant of Chapter 1, reporting to a steering group monthly. Two numbers appeared on every slide.

The measure of performance. Accuracy on the internal evaluation set, six hundred question-and-answer pairs built by two domain experts over three weeks from real call transcripts.

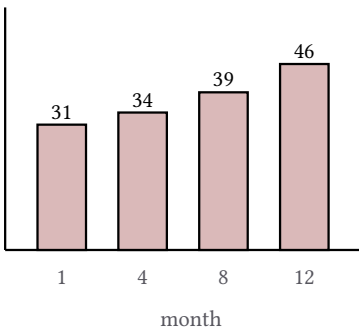
The measure of effectiveness. Escalation rate: calls per thousand that the assistant could not resolve and that went to a senior handler. This is what the project was funded to reduce.

Month	Eval accuracy	Escalations /1000	Explanation offered
1	94 %	31	baseline
4	94 %	34	product launch
8	94 %	39	seasonal mix
12	94 %	46	under investigation

measure of PERFORMANCE
evaluation accuracy



measure of EFFECTIVENESS
escalations per 1000 calls



The left-hand measure was structurally incapable of detecting the failure on the right: it ran against a frozen snapshot, so its value was independent of what the live corpus did.

Eleven months of divergence between the number that was reported and the outcome that was funded.

Why the first column could not move. The evaluation ran against a frozen snapshot of the corpus. Freezing it was the correct decision: an evaluation set whose ground truth moves underneath it is not an evaluation set, and the team were right to pin it. But the deployed system retrieved from a live index into which underwriters published weekly. The evaluation was therefore structurally incapable of detecting the failure mode that was occurring, whatever its value happened to be. It would have read 94 % on the morning the withdrawn clause was found, and it would read 94 % today.

The unstated assumption. For the 94 % to stand proxy for anything operational, somebody had to believe that the snapshot resembled the live corpus. That belief was reasonable in week one, was never written down, and stopped being true somewhere around week seven. Because it was never written down, it was never given an owner, never monitored, and never re-examined when the number it supported was quoted in month twelve.

What the steering group should have asked. Not “is 94 good” but three harder questions. What effectiveness measure does this number stand proxy for? What has to be true for it to stand proxy? And who is watching that assumption, on what schedule?

The rule that follows. Beside every performance number in a status report, record the effectiveness measure it represents and the assumption linking them, then place the *assumption* on a monitoring schedule. The number does not expire. The assumption does, and it expires quietly, which is why nothing in the reporting chain flagged eleven months of divergence.

The cheap detector that was missing. A weekly job comparing the document identifiers and modification dates in the evaluation snapshot against the live index, reporting the percentage that had diverged. Perhaps a day to write. It would have alarmed in week seven.

3.7 The layer and lifecycle matrix

Put the audit on one axis and the layer stack on the other, and you get the instrument this book navigates by. It is not a theory of anything. It is a grid whose purpose is to make absence visible, which is harder than it sounds, because a concern nobody has raised leaves no trace in any document.

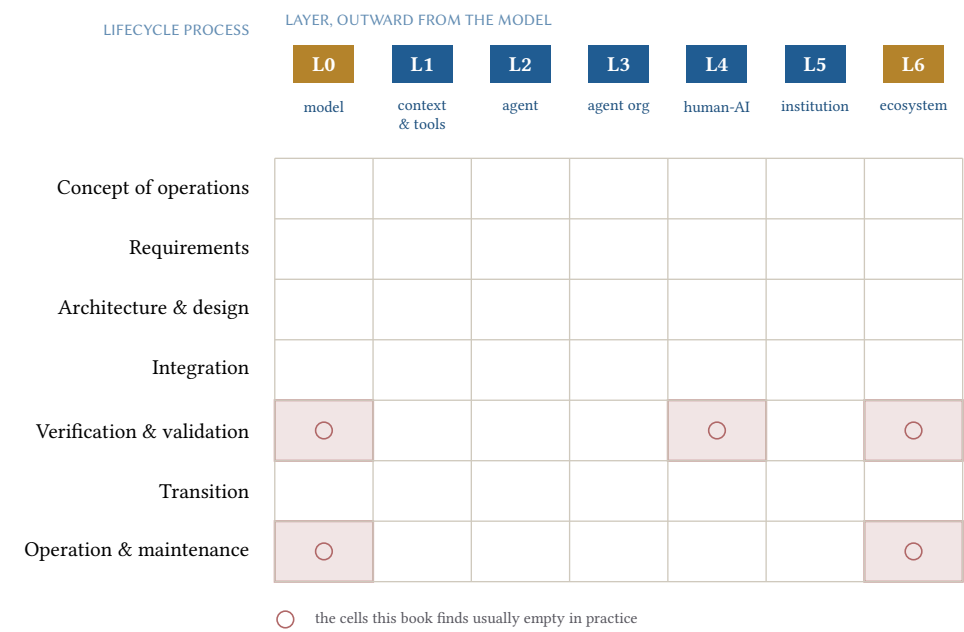


Figure 3.3. The audit on one axis, the layer stack on the other. Layers run left to right outward from the model, so the columns are the stack of Figure 3.2 laid on its side. Every cell is a real engineering question; the ringed cells are the ones this book finds usually empty in practice.

It is used in two ways. As a coverage checklist, you walk the cells and ask what artifact exists for each. “What is our concept of operations statement for the ecosystem layer?” is a question most teams have never been asked, and the honest answer is usually that there is none, which is itself a finding worth recording.

As a gap finder, the ringed cells are where this book expects to find nothing. Verification at the ecosystem layer, because you cannot test a counterparty's agent. Operation at the model layer, because few teams have any answer to what they do when the supplier changes it. Verification of the human layer, because almost nobody has ever tested whether their oversight arrangement works. These are not exotic corners. Every incident in Chapter 1 came out of one of them.

The matrix is reproduced as a worksheet in Appendix B, and Exercise 3.2 asks you to fill it in for a system you work on.

3.8 The three running systems

Three systems recur through the rest of the book. They are introduced here and developed end to end in Chapter 16, and each was chosen to put weight on a different part of the matrix.

A software engineering agent in a CI/CD pipeline has write authority over a repository, runs in a sandbox, passes through a human review gate, and has a first-pass success rate low enough to make its economics interesting. It stresses authority design at L1, loop termination at L2, and the unit economics of Chapter 15.

A clinical documentation and decision support assistant operates in a regulated setting, serves a professional user who remains accountable for the output, and carries hazards with physical consequence. It stresses oversight design at L4, the hazard analysis and assurance case of Chapter 13, and the requisite variety argument of §2.6.

An autonomous security operations agent is the adversarial case, in which the system both defends and is itself a target, and in which counterparties are hostile by assumption. It stresses the trust boundaries of Chapter 14, the ecosystem interoperation of Chapter 7, and the resilience of Chapter 12.

Vignettes from these three appear in every chapter that follows. They are the book's connective tissue, which is the reason there are three of them developed in depth rather than a dozen sketched.

What this chapter established

- ▶ Of the thirteen technical processes in the standard set, three survive essentially unchanged, nine need rework that is specific and describable, and one has no adequate analogue at all.
- ▶ The missing one is validation, because its evidence expires. This is the book's strongest claim and it is stated as a claim to be attacked.
- ▶ The reference architecture has seven layers, L0 to L6. L1 is the first layer that can change the world, and the two layers you do not control are the model you are supplied and the agents you meet.
- ▶ The layer by lifecycle matrix is a coverage instrument. Its empty cells are the finding, and each must be defended as either not applicable or not addressed.

- ▶ Three systems recur through the book and are developed end to end in the final chapter, each chosen to load a different part of the method.

Exercises

- 3.1 [D] Perform the §3.2 audit for a system you work on. For each of the thirteen processes, record the artifact you actually have, its owner, and when it was last reviewed. Mark each verdict from *your* evidence, not the book's. Defend, in one paragraph each, any process where your verdict differs.
- 3.2 [D] Populate the layer $\times$ lifecycle matrix for that system. Defend the empty cells: for each, state whether it is empty because the concern does not apply, or because nobody has addressed it. Report the count of each.
- 3.3 [D] Take five requirements from a real AI project. Rewrite each so that it can actually be verified: with a quantifier, a population, a confidence level, and a named verification method. Note which of the five turn out not to be requirements at all.
- 3.4 [D] Write the operating-domain definition for one of the three running systems, including three explicit out-of-domain conditions and the required behavior in each. State how you would detect, in operation, that the system has left its domain.
- 3.5 [A] The audit marks validation **no analogue** because its evidence expires. Estimate the *half-life* of validation evidence for a system you know, how long before you would no longer testify to a validation result without re-running it? State your assumptions. Compute what re-validating at that cadence costs per year, and compare it to the project's build cost.
- 3.6 [L] Map a real agent framework onto the L0-L6 architecture. Identify which layers it implements, which it leaves to you, and (the interesting category) which it *silently decides on your behalf*. For each silent decision, state whether you would have made it the same way.

Notes and further reading

On the process baseline (§1). The thirteen technical processes are those of Int [47]; Int [41] is the practitioner's companion and gives worked artifacts for each. Nothing in our summary of them is novel, and readers intending to apply the audit to a real programme should work from the standard rather than from our compression of it.

On the audit itself (§3.2). The verdicts are this book's argument rather than received wisdom, and we state them as claims to be attacked. The **no analogue** on validation is the strongest of them. A reader who believes classical validation does have an adequate analogue for AI systems should be able to name the process that manages expiring evidence, and we have not found one in the standards, in the AI assurance literature, or in practice. We would be glad to be shown otherwise.

On verification of stochastic systems (§3.2, verification). The most directly relevant recent evidence concerns automated grading. Norman et al. [69] evaluate 21 judge models

across roughly 541,000 judgments and find, among other results, that test-retest reliability above 0.95 can coexist with position bias above 0.10, and that agreement metrics overstate reliability substantially once corrected for chance. Their proposed minimum viable validation protocol is a reasonable starting point for any team relying on automated evaluation, and Example 3.1 works through the implications.

On why element verification does not discharge the system. The mechanism is the non-compositionality of Chapter 2, for which Sculley et al. [86] remains the clearest empirical statement. Teams that find our claim overstated should reproduce the exercise of Example 2.3 on their own pipeline before dismissing it.

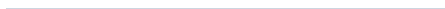
On safety and assurance (§3.2, and forward to Chapter 13). Leveson [58] is the foundational modern treatment, and her central claim, that safety is a control problem rather than a reliability problem, is the one we adopt. The assurance-case literature supplies what replaces a completion criterion once validation stops terminating; readers wanting depth before Chapter 13 should begin there.

On measures of effectiveness (§6). The MOE and MOP distinction is standard systems engineering practice and is not contested. Its application to AI is, in our experience, rarer than it should be, and Example 3.2 is included because the failure it describes is common and cheap to prevent.

Open at the time of writing. Whether continuous assurance can bear the regulatory weight that one-time validation bears in existing sectoral regimes is unresolved. Current AI regulation has not settled it, and we do not attempt to settle it either; Chapter 13 sets out what an answer would need to provide, and the epilogue records it as an open problem rather than a solved one.

Part II

Architecture



Characterising the Components

Learning Objectives

1. Write a usable datasheet for an element that resists classical specification, and say what it cannot promise.
2. Manage the configuration of a component you do not own and cannot freeze.
3. Treat context as an engineered resource with capacity, ordering, freshness and integrity properties.
4. Classify tools by consequence and grant authority proportionate to what an action can do.
5. Choose among prompting, retrieval, fine-tuning and distillation as a control decision rather than a preference.

What this chapter is not

It is not a tutorial on how language models work. We treat the model as a supplied element and describe only what an engineer must know in order to specify it, integrate it, verify it and sustain it. A reader who wants to understand attention should look elsewhere; a reader who wants to know what to write in an interface control document is in the right place.

4.1 The model as a supplied element

Part I established that an agentic system has four parts and that you procure one of them. This chapter is about the three you configure and the one you buy, and it begins with the one you buy, because almost every habit an engineer brings to a supplied component turns out to need adjusting.

Consider what you normally receive with a purchased part. A resistor comes with a datasheet giving tolerance, temperature coefficient and power rating, each a bound that holds across the stated operating range. A library comes with a version number, a changelog and a test suite. In both cases the supplier has told you what the part does, and told you in a form that lets you reason about what happens when you connect it to something else.

A foundation model comes with a model card, a price list, and a set of benchmark scores. None of those is a bound. A benchmark score is a measurement on a distribution that is not yours, taken at a moment that has passed, on a version that may already have been superseded.

4.1.1 The object of integration

The properties that matter for integration are a short list, and most of them are not on the supplier's page.

The **context limit** is stated, and is not the number you need. What you need is the length at which accuracy on *your* task begins to fall, which is invariably shorter and which §4.3 takes up.

Non-determinism is rarely stated at all. Sampling makes the output a draw rather than a value, and even with sampling parameters pinned, the provider's serving stack, batching and hardware can introduce variation. The engineering consequence is not that the system is unusable but that every claim about it is a claim about a distribution, and must carry a sample size.

Structured output and tool-calling behaviour determine whether the model can be integrated at all without a parser you maintain. Reliability here differs sharply between models and between versions of one model, and it is worth measuring rather than assuming.

Refusal behaviour is the boundary the supplier has drawn around what the model will do. It is a property of your supplier's policy rather than of your system, it changes without notice, and a refusal that arrives in production is an outage in every sense that matters to a user.

Latency and cost envelopes are stated per call and consumed per task. Chapter 15 works out why the ratio between those two numbers is the one that decides whether a product exists.

Known failure surfaces are what the supplier will not tell you, because they do not know them for your domain. You will discover them, and the only question is whether you discover them in evaluation or in production.

4.1.2 The supplier datasheet

Faced with a supplied part whose documentation does not support the reasoning you need, the classical move is to characterise it yourself and record the result. That record is worth constructing deliberately, and it has three registers rather than one.

Table 4.1 fills the three registers in for a typical integration.

The third column is the unusual one, and it is the reason this element is unlike any other supplied part. For a resistor, every honest entry is a number. Here some of the entries an engineer most wants are not numbers but refusals: there is no bound you can write for "this output will be correct", and writing one anyway is how a system acquires a requirement nobody can discharge.

Recording a refusal is not defeatism. It is what makes the surrounding architecture necessary and therefore fundable. A datasheet that says *this element cannot promise*

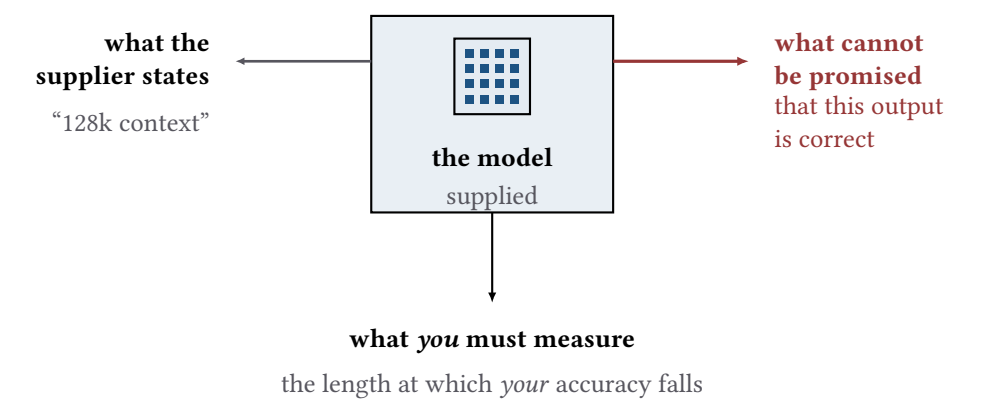


Figure 4.1. Three registers of claim about a supplied element. Only the downward arrow is work you must do yourself, and only the rightward one is a refusal.

Table 4.1. The component datasheet. The left column is a starting point, the middle column is the one that decides whether your system works, and the right column is what makes this element unlike any other supplied part.

The supplier states	You must measure	Cannot be promised
context window	effective context on <i>your</i> task	that any given output is correct
price per token	cost per <i>successful</i> task	that behaviour is stable across versions
latency percentiles	latency inside your loop	that two identical calls agree
training cut-off	refusal rate on your domain	that a stated limit is a usable limit
supported modalities	the failure modes you actually hit	that the next version behaves as this one

that two identical calls agree is a datasheet that justifies the validator, the retry policy and the sample-size discipline downstream of it.

4.1.3 Selection as a trade study

Choosing a model is an architectural decision with long consequences, and it is routinely made on a single axis. Capability is one criterion among six, and the others have a habit of dominating over the life of a system.

Cost and latency are obvious and usually measured wrongly, per call rather than per successful task. Hosting and data residency may be constraints rather than preferences, and in regulated settings they can eliminate most of the field before capability is discussed at all. Refusal behaviour determines how much of your domain is even addressable.

The criterion most often omitted is **exit cost**: what it would take to move to a different supplier. Prompts tuned against one model’s idiosyncrasies do not transfer

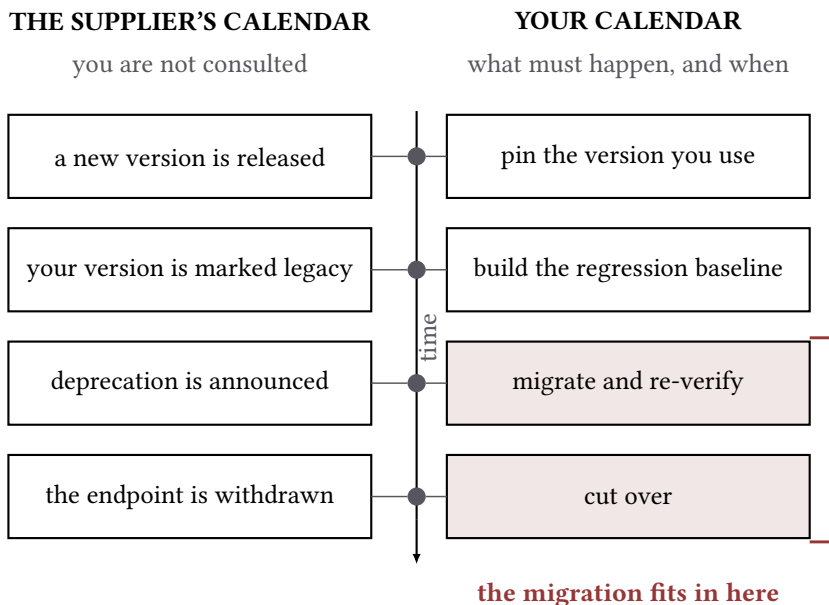
cleanly. Fine-tunes do not transfer at all. Evaluation sets transfer, which is one more reason to own them. A system whose exit cost is high has made a commercial commitment that nobody recorded as one, and Chapter 8 treats this as a trade study with the exit cost stated.

4.2 Configuration management of a supplied element

Classical configuration management rests on an assumption so basic that standards rarely state it: you control your configuration items. You decide when a component changes, you record the change, and you re-verify what the change touched.

The central element of an agentic system violates that assumption. The model is under a supplier's control, on a supplier's release schedule, with a deprecation date you do not set and were not consulted about. Chapter 3 marked this as the rework with no clean classical precedent, and it is, in our experience, the most under-managed risk in deployed AI systems.

The failure mode is not dramatic. It is a system that worked in March and works slightly less well in June, with no change in your repository to explain it.



If the regression baseline does not already exist when the deprecation lands, it gets built inside that window, under a deadline somebody else set, by a team that has other work.

Figure 4.2. Two calendars govern one system, and only one of them is yours. The work in the right-hand window is fixed; what varies is whether it was prepared for.

Four practices address this, and none is exotic.

Pin the version. Calling a floating alias means accepting silent substitution of a configuration item. The convenience is real and the trade is rarely made explicitly. Pinning converts an invisible risk into a visible task, which is usually the whole of what good configuration management does.

Hold a regression baseline. A set of cases whose behaviour you have recorded, that can be re-run against a candidate version, and whose diffs you will actually read. This is the artifact that makes migration an engineering activity rather than a leap. Note that it must be built before it is needed, because the window Figure 4.2 shows is not long enough to build one in.

Track the supplier's calendar as a project risk. Deprecation dates belong in the same register as any other externally imposed deadline, with an owner, and they should be reviewed when they move.

Treat migration as a planned event. Dual-running the old and new versions across the regression baseline, reading the diffs, and cutting over deliberately is a week of work when scheduled and a fortnight of unpleasantness when not.

Example 4.1 | The cost of an unpinned version

Setting. A document-classification service in production for eight months, calling a hosted model through a floating alias because that is the default in the vendor's quickstart and nobody revisited it. Accuracy on the internal evaluation set was 93 % at launch and was checked quarterly.

What happened. In the second quarter's check, accuracy read 89 %. The team's first hypothesis was data drift, which is reasonable and was wrong. Three weeks of investigation followed: sampling recent documents, comparing distributions, re-labelling a subsample to rule out annotation drift.

What had actually happened. The alias had moved to a new model version six weeks earlier. The vendor had announced it. The announcement went to an address that forwarded to a team mailing list nobody read.

The arithmetic of the loss. Three weeks of one engineer investigating, at a fully loaded cost the reader can supply, plus six weeks of degraded service affecting roughly 40,000 documents. Against that, the cost of the practices above: pinning is one line of configuration; the regression baseline for this system was 300 cases that took two days to assemble; watching the deprecation calendar is a recurring half-hour.

Why the investigation took three weeks. Not incompetence. The team had no way to *rule out* a model change, because they had no record of which version had served which request. Once telemetry included the version string, the same investigation would have taken an afternoon. Observability turns a hypothesis space into a lookup.

The general point. The four-point drop was not the expensive part. The expensive part was that the system gave no account of itself, so a question with a one-line answer consumed three weeks. Chapter 11 returns to this: instrumentation added after an incident cannot explain that incident.

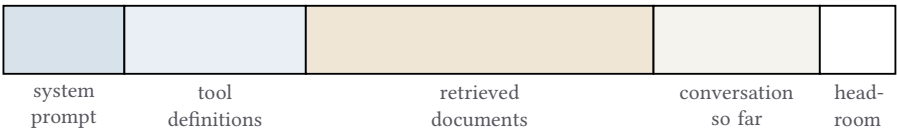
What to do this week. Find every model reference in your deployment configuration. For each, record the version actually serving traffic, whether it is pinned, and where a deprecation notice would arrive. Most teams doing this for the first time discover at least one floating alias and at least one notification address that nobody reads.

4.3 Context as an engineered resource

The context is everything the model is shown on a given call, and it is assembled by code you wrote. That makes it the first place where engineering decisions live, and the place where the largest gains are usually available.

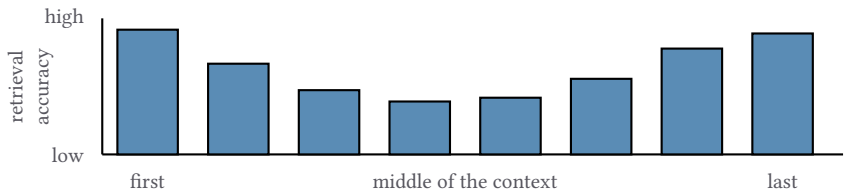
Section 1.1 observed that this is an unusual interface, because every part of the system writes into one accumulating channel that every other part reads, and §2.2 worked out what that does to a decomposition. What neither did is treat the channel as a resource with a budget, an allocation and a contention problem, which is what it is and what this section is about.

THE BUDGET IS FINITE, AND YOU ALLOCATE IT



Every element of the system writes into this one channel. When it overflows, something is dropped, and what gets dropped is a design decision whether or not anyone made it.

AND POSITION WITHIN IT MATTERS



Schematic, after Liu et al. Relevant information placed in the middle of a long context is used least reliably, and this holds for models explicitly built for long contexts.

Figure 4.3. Above: the budget is finite and its allocation is a design decision. Below: position within the budget is not neutral, and material placed in the middle of a long context is used least reliably.

4.3.1 Capacity and edge behaviour

The budget is finite. When it overflows, something is dropped, and what gets dropped is a design decision whether or not anyone made it. The default in most frameworks is to drop the oldest conversation turns, which is defensible for chat and wrong for an agent, because the oldest turns frequently contain the constraints the task was given.

That failure mode is not hypothetical. Recall from Example 1.1 that the dominant reported failures on long-horizon computer-use work include losing track of constraints and missing information that arrived part-way through the task. Both are context-management failures, and both are addressable in code you control.

4.3.2 Positional effects

A budget suggests that what matters is whether material fits. It does not. Where material sits within the context also affects whether it is used.

Example 4.2 | Position effects in a long context

Setting. Liu et al. [61] examine how language models use long input contexts, across multi-document question answering and key-value retrieval, using models including long-context variants explicitly marketed for this case.

The manipulation. They hold the content of the context fixed and vary only the *position* of the relevant document within it. Everything the model needs is present in every condition; the only difference is where it sits.

The result. Performance is highest when the relevant information is at the beginning or the end of the context, and degrades substantially when it falls in the middle. The effect holds for models built for long contexts, so it is not simply a matter of the window being too small.

Why an engineer should care about a result like this. Because it converts a property most teams treat as a capacity constraint into a placement decision. If you are assembling a context from twenty retrieved passages and the ranker's best result is placed twelfth, you have put your strongest evidence in the least reliable position. Nobody decided that; the ranker's output order was passed through unchanged.

Three changes that follow directly. Place the highest-ranked evidence first, or first and last, rather than in rank order through the middle. Cap the number of passages rather than filling the window because it is available, since dilution has a cost that free capacity does not offset. And when you measure retrieval quality, measure it at the position the passage will actually occupy, not in isolation.

A caution. This is a finding about particular models at a particular time, and the magnitude of the effect varies by model and has moved as architectures have changed. Treat the specific numbers as dated and the structural lesson as durable: *context is not a bag, and order is a parameter you are setting whether or not you are choosing it.*

4.3.3 Prompts and policies as configuration items

A system prompt states the element's role, its constraints and its refusal boundaries. It determines behaviour, it is edited routinely, and in most organisations it is neither versioned nor reviewed nor traceable to any requirement.

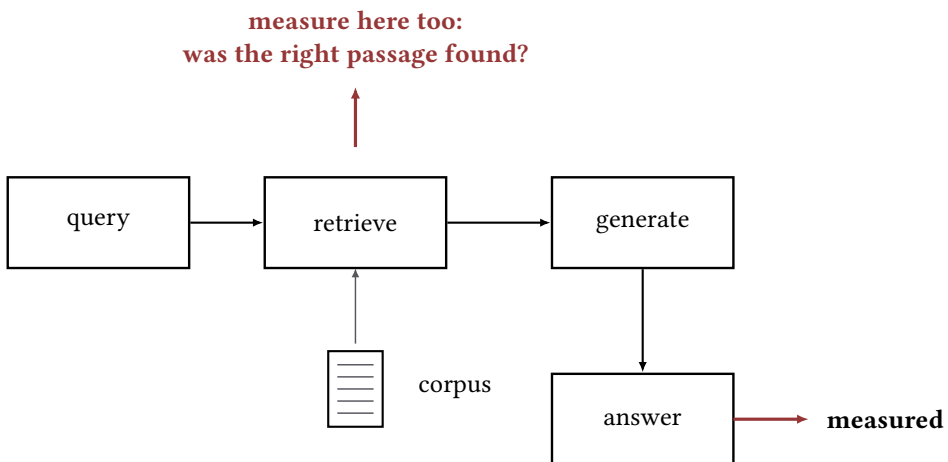
Chapter 3 argued that design artifacts for AI systems are prompts, policies and tool definitions, which makes them configuration items subject to change control.

In practice this means four things: a prompt lives in version control rather than a console; a change to it is reviewed by somebody other than its author; a change triggers the regression baseline; and each substantive instruction in it can be traced to a requirement, so that when somebody proposes deleting a line there is a record of why it was added.

The last is worth insisting on. System prompts accumulate. Every incident adds a sentence, nobody removes one, and after two years the prompt contains instructions addressing failure modes that no longer exist, written by people who have left, in tension with one another. That is a configuration item that has never been maintained, and its length is itself a cost, since it consumes budget and dilutes attention.

4.3.4 Retrieval as a subsystem

Where a system retrieves from a corpus, the retrieval is an element with its own requirements, its own failure modes and its own tests, and it should be verified independently of generation.



A system that measures only the answer cannot tell a retrieval failure from a generation failure, and the two have different remedies.

Figure 4.4. Retrieval has a measurement point of its own, and a system that uses only the one on the right cannot tell two very different failures apart.

The reason is diagnostic. When an answer is wrong there are two candidate explanations: the right passage was not retrieved, or it was retrieved and not used. These have entirely different remedies, and a system that only measures end-to-end quality cannot distinguish them. Scoring retrieval alone, against a set of queries with known relevant documents, separates the two and usually reveals that a substantial fraction of end-to-end failures were never generation failures at all.

Freshness belongs here too, and Chapter 1’s insurance assistant is the cautionary case: a corpus with no owner, no freshness requirement and no divergence check between the evaluated snapshot and the live index.

4.3.5 Grounding: citation against evidence

A system that answers with citations feels accountable. It is worth being precise about what the citation establishes.

A citation establishes that a passage was retrieved and presented alongside the answer. It does not establish that the passage supports the claim, that the claim is contained in the passage rather than elaborated beyond it, or that the passage is current. Each of those is a separate property requiring a separate check, and users reading a citation will assume all three.

Where the stakes justify it, the check is mechanical: verify that the cited passage entails the claim, by a second model or a rule, and fail closed when it does not. Where the stakes do not justify it, the honest move is to be careful about how much assurance the citation is allowed to imply in the interface.

4.4 Tools and authority

Tools are where an AI system stops producing text and starts changing the world, and the design decisions here are the ones most likely to appear in an incident report.

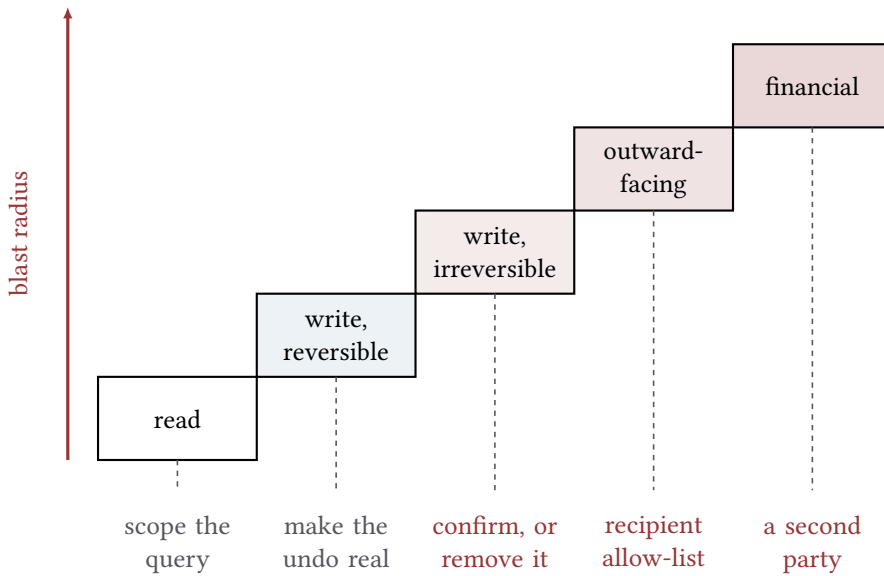
4.4.1 Classification by consequence

Tool catalogues are usually organised the way the code is organised: database tools, messaging tools, file tools. That grouping is convenient for the people who maintain them and useless for the question that matters, which is what happens if this is called wrongly.

Table 4.2. Tools ordered by consequence. The column that decides the control is *undo*, and it does not correlate with the subsystem a tool belongs to.

Consequence	Example	Undo?	Proportionate control
read	look up an account	n/a	scope the query; log it
write, reversible	update a draft	yes	log it, and make the undo real
write, irreversible	delete a record	no	human confirmation, or remove the capability
outward-facing	send an email	no	confirmation, plus a recipient allow-list
financial	issue a refund	no	confirmation, a value cap, a second party

Ordering by consequence, as in Table 4.2, produces a different catalogue and a different set of controls. Two tools that both write belong on different rungs if one can be reversed and the other cannot. A tool that sends email sits above one that



the control has to climb the same stairs

Figure 4.5. The consequence staircase. Blast radius rises from left to right, and the control has to rise with it.

writes to an internal store, not because the technology is riskier but because the action leaves the system’s boundary and cannot be recalled.

The support assistant of Chapter 1 had one tool, and a consequence classification would have put it on the third rung, where the proportionate control is human confirmation or removal of the capability. The tool was catalogued as a database tool, on the second rung by technology and the third by consequence, and nobody had drawn the distinction.

4.4.2 The tool description as a specification

A tool is declared with a name, an argument schema and a natural-language description of what it does and when to use it. That description is read by the model at runtime and determines whether and how the tool is called.

It follows that documentation and specification are the same artifact here, which has no counterpart in conventional practice. Editing a docstring changes system behaviour, immediately, with no build step. Chapter 3’s Example 3.1 records a team that changed a tool description from “search the knowledge base” to “search the knowledge base for policy questions” and lost a third of that tool’s call volume for six weeks before anyone connected the two.

Error messages deserve the same treatment. A tool’s error text is not a log line for a human; it is an input to the next deliberation step. An error that says 400 Bad Request tells the agent nothing it can act on. An error that says which field

was invalid and what form was expected converts a dead end into a retry that can succeed.

4.4.3 Catalogue sprawl

Every capability added to an agent is added to its context, and the catalogue grows monotonically because nobody is responsible for removing tools.

Example 4.3 | Catalogue size and tool-selection accuracy

Setting. A natural assumption in agent design is that more available tools can only help: if the agent does not need one, it will not call it. Repantis et al. [80] test this across registries ranging from twenty to several thousand tools.

The methodological point first. Measuring selection accuracy against tool count has a trap. Showing fewer tools makes random selection likelier to succeed, so a shorter list can look better while contributing nothing. They therefore use a chance-corrected measure, asking whether success at a given list length beats what random selection would achieve at that length. An engineer running this experiment in-house should correct the same way.

The results. An adaptive, query-specific shortlist averaging about *seven* tools achieved coverage of 90.3 % against 90.8 % for a fixed list of fifty: essentially the same coverage from a seventh of the context. Downstream, selection accuracy with a frontier model was 93.1 % with the shorter adaptive list against 87.1 % with the longer one.

Read that last number again. Showing the agent *fewer* tools made it choose *better*. The extra forty-three tools were not neutral; they were actively harmful, because schemas in context compete for attention and near-duplicates become genuinely ambiguous.

The mechanism, and why it is familiar. This is the dilution of §4.3 appearing at a different point in the budget. Tool definitions occupy the same finite channel as retrieved documents and conversation history, and they degrade the same way.

What to do. Retrieve tools per query rather than presenting the catalogue. Measure selection accuracy as the catalogue grows, and correct for chance. Put a removal process in place, because without one the catalogue only grows. And treat a request to add a tool as a change to a configuration item, which it is.

4.4.4 Authority, gates, and blast radius

Three controls sit behind the consequence ladder, and they are not interchangeable.

Least privilege means the credential a tool holds is scoped to what the tool needs, and nothing more. This is the control that would have prevented Chapter 1's incident, and it is the one that was requested and deferred. Note that it operates on possibility rather than probability, which is why the book keeps returning to it.

Confirmation gates insert a human between a decision and its effect. They are effective and they are not free: each one costs latency and attention, and a system

that asks for confirmation on everything trains its operators to approve without reading, which converts a control into a formality. Gates belong on the rungs where undo is unavailable, and the consequence ladder is how you decide which those are.

Sandboxing limits what an action can reach rather than which actions are permitted. It is the control that fails least gracefully when it fails, as §1.2.4 records: an evaluation environment whose isolation depended on a proxy, and whose proxy was the component that gave way.

Beneath all three sits the structural fact this book returns to repeatedly. An element that consumes untrusted content while holding authority is a confused deputy [35], and no amount of instruction to that element resolves it, because the instruction is evaluated by the element whose judgement is in question. Chapter 14 treats this properly; here it is enough to say that the tool layer is where the authority half of that pairing is granted, usually in a configuration file, often in an afternoon.

4.4.5 Third-party tools as supplied elements

Where tools are provided by others, everything in §4.1 applies again, with less recourse. A third-party tool server may change its schema, alter its behaviour, or fail in ways you cannot instrument. You inherit its availability and its security posture, and you usually have no supplier agreement of the kind that would ordinarily govern a dependency this consequential. Chapter 7 takes this up as the ecosystem layer.

4.5 Adaptation as a control decision

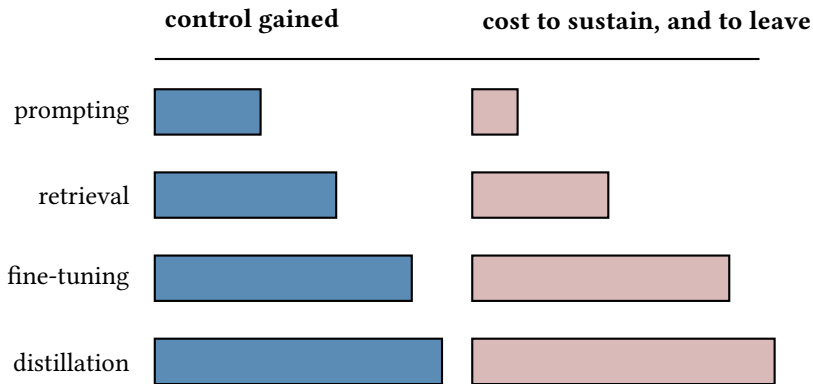
The last decision in this chapter is how to make a general element do a particular job. Four mechanisms are available, and they are usually discussed as a capability ladder. They are better understood as points on a curve trading control against cost to sustain.

Prompting is immediate, costs nothing to change, and holds least. It is the right default and the wrong place to put a property you must guarantee, for the reason §2.12 gives: an instruction to the element is not a constraint on it.

Retrieval supplies knowledge at run time rather than baking it in. It keeps the corpus as a separate, ownable, updatable artifact, which is usually the right structure for anything that changes. It adds a subsystem with its own failure modes, as §4.3 describes.

Fine-tuning changes the element's behaviour rather than its instructions, which buys consistency and costs flexibility. It is an engineering commitment in a specific sense: it freezes a behaviour, it creates a new configuration item with its own lineage and its own re-verification burden, and it raises exit cost sharply, because a fine-tune does not transfer to another supplier.

Distillation trades capability for cost and latency by training a smaller model on a larger one's behaviour. It is the strongest move available for unit economics, and it locks in the teacher's failure modes along with its strengths.



Schematic, not measured: the ordering is the claim, not the lengths. Every step down the list buys control over behaviour and is paid for in maintenance and in the cost of moving to another supplier. Start at the top and move down only when the behaviour is stable enough to be worth freezing.

Figure 4.6. The four mechanisms on the two axes that decide between them. Moving right buys control and is paid for in maintenance and in the cost of leaving.

The decision procedure we would offer is short. Start with prompting, because it is reversible. Move knowledge that changes into retrieval, because a corpus can be owned and updated. Reach for fine-tuning when the behaviour is stable, the volume justifies the maintenance, and you have accepted the exit cost. Reach for distillation when the economics require it and the capability ceiling is understood. And at every step, ask what the choice costs to sustain over two years, not what it costs to adopt this week.

What this chapter established

- ▶ The model is a supplied element. You cannot inspect it, you did not build it, and its behaviour can be withdrawn or changed by its supplier on a schedule that is not yours.
- ▶ A pinned API version is not stationarity. Behaviour can move under a fixed version identifier, so change detection is an operational requirement rather than a precaution.
- ▶ Context is an engineered resource with a finite budget, and every element of the system writes into one accumulating channel that every other element reads. Two components that share a context are coupled whether or not a line on the diagram joins them.

- ▶ Position within the context is not neutral: material placed in the middle of a long context is used least reliably, and this holds for models built for long contexts.
- ▶ Tools are where authority is granted, usually in a configuration file and often in an afternoon. The tool layer is the point at which a system acquires the ability to be wrong in ways that matter.

Exercises

- 4.1 [D] Write the datasheet of Table 4.1 for a model your organisation depends on. Fill all three columns. For the middle column, state how you measured each entry or mark it as not yet measured. Report how many entries in the middle column were unmeasured before you started.
- 4.2 [D] Write the configuration management plan for that model: pinning policy, regression baseline, where deprecation notices arrive and who reads them, and the migration procedure. Then check the notification address actually reaches a person.
- 4.3 [L] Take a working agent and measure the effect of context position. Hold the content fixed and move the critical passage from first, to middle, to last. Report the spread, and say what your context assembly currently does with ranker output.
- 4.4 [L] Measure tool-selection accuracy as a function of catalogue size on your own system, correcting for chance as Repantis et al. [80] do. Find the point at which accuracy begins to fall, and compare it with the number of tools you currently present.
- 4.5 [D] Classify every tool in an existing agent using Table 4.2. For each, record the credential it holds, whether the action can be undone, and the control currently in place. Name the rows where the control does not match the rung.
- 4.6 [A] A tool is called n times a day with an error rate p per call. For the third rung of Table 4.2, where undo is unavailable, derive the expected time to first uncontrolled incident as a function of n and p , and evaluate it for $n = 400$ and $p = 10^{-4}$. Comment on what this implies for controls that reduce p rather than removing the possibility.

Notes and further reading

On characterising a supplied element (§4.1). The three-register datasheet is our own device. The underlying practice, characterising a part yourself when the supplier's documentation does not support the reasoning you need, is ordinary engineering and is treated in any systems text; Int [41] covers supplied-item management in the conventional case, which is the useful contrast.

On configuration management of a vendor model (§4.2). We know of no standard that addresses a configuration item under a third party's release control in the way this situation requires, which is why Chapter 3 marks the surrounding process **rework**. Sculley et al. [86] anticipate the maintenance burden in the earlier ML setting, and their entanglement argument explains why a version change is rarely local in effect.

On context position (§4.3). Liu et al. [61] is the standard reference for the position effect, and Example 4.2 follows their manipulation. Readers should note that the magnitude varies by model and has moved with successive architectures; the structural lesson, that ordering is a parameter you are setting whether or not you choose it, is the durable part. We are not aware of a comparably careful treatment of how position interacts with tool definitions rather than retrieved documents, which seems to us an open and practically important question.

On tool catalogues (§4.4). Repantis et al. [80] provide the chance-corrected treatment we follow, and their methodological point deserves emphasis independently of their results: uncorrected comparisons across list lengths are not interpretable, because random-selection success rises as the list shortens. Their finding that shorter adaptive lists improve selection accuracy is the counter-intuitive result worth carrying.

On authority and the confused deputy (§4.4). Hardy [35] remains the clearest statement of the underlying problem. Its agentic instance is Greshake et al. [33], and the July 2026 incident of §1.2.4 [38] is the largest documented case at the time of writing. Chapter 14 is where these are analysed rather than cited.

Open at the time of writing. There is no settled practice for regression-testing a natural-language interface across model versions. Teams currently assemble ad hoc suites, and the diffs require human reading, which does not scale with either the number of cases or the frequency of vendor releases. We regard this as the most valuable unsolved tooling problem in the area covered by this chapter.

End of sample

This sample contains Part I complete, which is the argument the rest of the book rests on, together with the first chapter of Part II. The contents pages list the whole book; the page numbers there are the full book's, so a reader can see where everything sits.

What the sample does not include:

Part II: Architecture

Chapters 5 to 7: the agent as a system element, organisations of agents and people, and interoperability across operator boundaries.

Part III: The Adapted Lifecycle

Chapters 8 to 11: concept and requirements, implementation and integration, verification and validation, and transition through to sustainment.

Part IV: Dependability

Chapters 12 to 14: reliability and resilience, hazard analysis and assurance, and security.

Parts V and VI

Chapter 15 on economics and the engineering organisation, and Chapter 16, which carries three systems end to end.

The epilogue

Nine problems the book can state and not solve.

Five appendices

A systems engineering primer for readers who have not met the discipline, a worksheet, the quantitative background with every formula the book relies on, a glossary of 89 terms, and a capstone exercise.

The bibliography that follows is the whole book's, not only this sample's, so that a reader can see what the argument is built on.

Bibliography

- [1] Special review of procurement procedures related to UTMDACC oncology expert advisor project. Internal audit special review, University of Texas System Administration, System Audit Office, November 2016. Examines procurement and contracting for the project; expressly does not assess the scientific or clinical merit of the technology.
- [2] W. Ross Ashby. *An Introduction to Cybernetics*. Chapman and Hall, 1956. The law of requisite variety.
- [3] Algirdas Avizzenis. The N-version approach to fault-tolerant software. *IEEE Transactions on Software Engineering*, 11(12):1491–1501, 1985.
- [4] Algirdas Avizzenis, Jean-Claude Laprie, Brian Randell, and Carl Landwehr. Basic concepts and taxonomy of dependable and secure computing. *IEEE Transactions on Dependable and Secure Computing*, 1(1):11–33, 2004.
- [5] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022. URL <https://arxiv.org/abs/2212.08073>.
- [6] Lisanne Bainbridge. Ironies of automation. *Automatica*, 19(6):775–779, 1983.
- [7] Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy, editors. *Site Reliability Engineering: How Google Runs Production Systems*. O’Reilly Media, 2016.
- [8] Arnar Birgisson, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrabie, and Mark Lentczner. Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. In *Network and Distributed System Security Symposium (NDSS)*, 2014.
- [9] Samuel R. Bowman and George E. Dahl. What will it take to fix benchmarking in natural language understanding? In *Proceedings of NAACL-HLT*, 2021.
- [10] British Columbia Civil Resolution Tribunal. *Moffatt v. air canada*, 2024 BCCRT 149. Decision of the Civil Resolution Tribunal, British Columbia, 2024.
- [11] California Department of Motor Vehicles. Order of suspension: GM cruise LLC autonomous vehicle deployment and driverless testing permits, 2023.

- [12] Dallas Card, Peter Henderson, Urvashi Khandelwal, Robin Jia, Kyle Mahowald, and Dan Jurafsky. With little power comes great responsibility. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.
- [13] Nicholas Carlini, Florian Tramèr, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Úlfar Erlingsson, Alina Oprea, and Colin Raffel. Extracting training data from large language models. In *Proceedings of the 30th USENIX Security Symposium*, 2021.
- [14] Nicholas Carlini, Daniel Paleka, Krishnamurthy Dj Dvijotham, Thomas Steinke, Jonathan Hayase, A. Feder Cooper, Katherine Lee, Matthew Jagielski, Milad Nasr, Arthur Conmy, Eric Wallace, David Rolnick, and Florian Tramèr. Stealing part of a production language model. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2024.
- [15] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, et al. Why do multi-agent LLM systems fail? *arXiv preprint arXiv:2503.13657*, 2025. URL <https://arxiv.org/abs/2503.13657>. Presented at NeurIPS 2025.
- [16] Juntao Chen and Quanyan Zhu. Interdependent strategic security risk management with bounded rationality in the Internet of Things. *IEEE Transactions on Information Forensics and Security*, 14(11):2958–2971, 2019.
- [17] Juntao Chen and Quanyan Zhu. *A Game- and Decision-Theoretic Approach to Resilient Interdependent Network Analysis and Design*. SpringerBriefs in Control, Automation and Robotics. Springer, 2020.
- [18] Melvin E. Conway. How do committees invent? *Datamation*, 14(5):28–31, 1968.
- [19] Richard I. Cook. How complex systems fail. Cognitive Technologies Laboratory, University of Chicago, 1998.
- [20] CrowdStrike. Channel file 291 incident: External technical root cause analysis. CrowdStrike technical report, 2024.
- [21] Edoardo DeBenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Proceedings of the NeurIPS Datasets and Benchmarks Track*, 2024.
- [22] Sidney Dekker. *The Field Guide to Understanding ‘Human Error’*. Ashgate, 3rd edition, 2014.

- [23] Sidney W. A. Dekker and David D. Woods. MABA-MABA or abracadabra? progress on human–automation co-ordination. *Cognition, Technology & Work*, 4(4):240–244, 2002.
- [24] Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. The hitchhiker’s guide to testing statistical significance in natural language processing. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2018.
- [25] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. *arXiv preprint arXiv:2305.14325*, 2023. URL <https://arxiv.org/abs/2305.14325>. Published at ICML 2024.
- [26] Madeleine Clare Elish. Moral crumple zones: Cautionary tales in human-robot interaction. *Engaging Science, Technology, and Society*, 5:40–60, 2019.
- [27] *New European Interoperability Framework: Promoting Seamless Services and Data Flows for European Public Administrations*. European Commission, 2017.
- [28] *Regulation (EU) 2024/1689 Laying Down Harmonised Rules on Artificial Intelligence*. European Parliament and Council of the European Union, 2024.
- [29] Paul M. Fitts. *Human Engineering for an Effective Air-Navigation and Traffic-Control System*. National Research Council, Washington, DC, 1951.
- [30] João Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4):1–37, 2014.
- [31] Deep Ganguli, Liane Lovitt, Jackson Kernion, Amanda Askell, Yuntao Bai, Saurav Kadavath, et al. Red teaming language models to reduce harms: Methods, scaling behaviors, and lessons learned. *arXiv preprint arXiv:2209.07858*, 2022. URL <https://arxiv.org/abs/2209.07858>.
- [32] Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021.
- [33] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023. URL <https://arxiv.org/abs/2302.12173>. arXiv:2302.12173.
- [34] Dick Hardt. *The OAuth 2.0 Authorization Framework*. Internet Engineering Task Force, 2012. URL <https://www.rfc-editor.org/rfc/rfc6749>. RFC 6749.

- [35] Norm Hardy. The confused deputy: (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review*, 22(4):36–38, 1988.
- [36] Erik Hollnagel, David D. Woods, and Nancy Leveson, editors. *Resilience Engineering: Concepts and Precepts*. Ashgate, 2006.
- [37] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2310.01798>. arXiv:2310.01798.
- [38] Hugging Face. Security incident disclosure — july 2026. Hugging Face blog, 2026. URL <https://huggingface.co/blog/security-incident-july-2026>.
- [39] Hugging Face. Anatomy of a frontier lab agent intrusion: A technical timeline of the july 2026 incident. Hugging Face blog, 2026. URL <https://huggingface.co/blog/agent-intrusion-technical-timeline>.
- [40] Jez Humble and David Farley. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, 2010.
- [41] *INCOSE Systems Engineering Handbook*. International Council on Systems Engineering, 5th edition, 2023.
- [42] *IEC 61508:2010, Functional Safety of Electrical, Electronic and Programmable Electronic Safety-Related Systems*. International Electrotechnical Commission, 2010.
- [43] *ISO/IEC 9646-1:1994, Information Technology — Open Systems Interconnection — Conformance Testing Methodology and Framework — Part 1: General Concepts*. International Organization for Standardization, 1994.
- [44] *ISO 26262:2018, Road Vehicles — Functional Safety*. International Organization for Standardization, 2018.
- [45] *ISO/IEC/IEEE 29148:2018, Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*. International Organization for Standardization, 2018.
- [46] *ISO 21448:2022, Road Vehicles — Safety of the Intended Functionality*. International Organization for Standardization, 2022.
- [47] *ISO/IEC/IEEE 15288: Systems and Software Engineering — System Life Cycle Processes*. International Organization for Standardization, 2023.
- [48] *ISO/IEC 23894:2023, Information Technology — Artificial Intelligence — Guidance on Risk Management*. International Organization for Standardization, 2023.

- [49] *ISO/IEC 42001:2023, Information Technology — Artificial Intelligence — Management System*. International Organization for Standardization, 2023.
- [50] Michael Jackson. The world and the machine. In *Proceedings of the 17th International Conference on Software Engineering (ICSE)*, pages 283–292, 1995.
- [51] Santosh Janardhan. More details about the october 4 outage. Meta Engineering blog, 2021.
- [52] Michael B. Jones, John Bradley, and Nat Sakimura. *JSON Web Token (JWT)*. Internet Engineering Task Force, 2015. URL <https://www.rfc-editor.org/rfc/rfc7519>. RFC 7519.
- [53] Michael B. Jones, Anthony Nadalin, Brian Campbell, John Bradley, and Chuck Mortimore. *OAuth 2.0 Token Exchange*. Internet Engineering Task Force, 2020. URL <https://www.rfc-editor.org/rfc/rfc8693>. RFC 8693.
- [54] Ralph L. Keeney and Howard Raiffa. *Decisions with Multiple Objectives: Preferences and Value Tradeoffs*. Wiley, 1976.
- [55] Tim Kelly and Rob Weaver. The goal structuring notation: A safety argument notation. In *Proceedings of the Dependable Systems and Networks Workshop on Assurance Cases*, 2004.
- [56] Gary Klein, David D. Woods, Jeffrey M. Bradshaw, Robert R. Hoffman, and Paul J. Feltovich. Ten challenges for making automation a “team player” in joint human-agent activity. *IEEE Intelligent Systems*, 19(6):91–95, 2004.
- [57] John C. Knight and Nancy G. Leveson. An experimental evaluation of the assumption of independence in multiversion programming. *IEEE Transactions on Software Engineering*, 12(1):96–109, 1986.
- [58] Nancy G. Leveson. *Engineering a Safer World: Systems Thinking Applied to Safety*. MIT Press, 2011.
- [59] Nancy G. Leveson and John P. Thomas. *STPA Handbook*. Massachusetts Institute of Technology, 2018.
- [60] Tao Li, Ya-Ting Yang, Yunian Pan, and Quanyan Zhu. From texts to shields: Convergence of large language models and cybersecurity. *arXiv preprint arXiv:2505.00841*, 2025. URL <https://arxiv.org/abs/2505.00841>.
- [61] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. URL <https://aclanthology.org/2024.tacl-1.9/>.

- [62] Mars Climate Orbiter Mishap Investigation Board. Mars climate orbiter mishap investigation board phase i report. Technical report, National Aeronautics and Space Administration, 1999.
- [63] Donella H. Meadows. *Thinking in Systems: A Primer*. Chelsea Green, 2008. Stocks, flows and leverage points.
- [64] Mark S. Miller. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. PhD thesis, Johns Hopkins University, 2006.
- [65] Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT*)*, pages 220–229, 2019.
- [66] *ATLAS: Adversarial Threat Landscape for Artificial Intelligence Systems*. MITRE Corporation, 2024.
- [67] *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. National Institute of Standards and Technology, 2023. NIST AI 100-1.
- [68] National Transportation Safety Board. Collision between vehicle controlled by developmental automated driving system and pedestrian, tempe, arizona, march 18, 2018. Technical Report Highway Accident Report NTSB/HAR-19/03, NTSB, 2019.
- [69] Justin D. Norman, Michael U. Rivera, and D. Alex Hughes. Reliability without validity: A systematic, large-scale evaluation of LLM-as-a-judge models across agreement, consistency, and bias. *arXiv preprint arXiv:2606.19544*, 2026. URL <https://arxiv.org/abs/2606.19544>.
- [70] Michael Nygard. Documenting architecture decisions. Blog post, 2011. URL <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>.
- [71] Michael T. Nygard. *Release It! Design and Deploy Production-Ready Software*. Pragmatic Bookshelf, 2nd edition, 2018.
- [72] Ofqual. Awarding GCSE, AS and A level qualifications in summer 2020: Interim report. Technical report, Office of Qualifications and Examinations Regulation, 2020.
- [73] *OWASP Top 10 for Large Language Model Applications*. Open Worldwide Application Security Project, 2025.
- [74] Raja Parasuraman and Victor Riley. Humans and automation: Use, misuse, disuse, abuse. *Human Factors*, 39(2):230–253, 1997.

- [75] Raja Parasuraman, Thomas B. Sheridan, and Christopher D. Wickens. A model for types and levels of human interaction with automation. *IEEE Transactions on Systems, Man, and Cybernetics—Part A*, 30(3):286–297, 2000.
- [76] Jeffrey Pawlick and Quanyan Zhu. *Game Theory for Cyber Deception: From Theory to Applications*. Static & Dynamic Game Theory: Foundations & Applications. Birkhäuser, 2021.
- [77] Quinn Emanuel Urquhart & Sullivan. Report to the boards of directors of cruise LLC, GM cruise holdings LLC and general motors co. regarding the october 2, 2023 accident in san francisco. Independent investigation report, 2024.
- [78] Inioluwa Deborah Raji, Emily M. Bender, Amandalynne Paullada, Emily Denton, and Alex Hanna. AI and the everything in the whole wide world benchmark. In *Proceedings of the NeurIPS Datasets and Benchmarks Track*, 2021.
- [79] Stefan Rass, Stefan Schauer, Sandra König, and Quanyan Zhu. *Cyber-Security in Critical Infrastructures: A Game-Theoretic Approach*. Advanced Sciences and Technologies for Security Applications. Springer, 2020.
- [80] Leonidas Repantis, Aditya Gawde, Karan Singh, and Ii Blackwell. How many tools should an LLM agent see? a chance-corrected answer. *arXiv preprint arXiv:2605.24660*, 2026. URL <https://arxiv.org/abs/2605.24660>.
- [81] *DO-178C: Software Considerations in Airborne Systems and Equipment Certification*. RTCA, 2011.
- [82] *J3016: Taxonomy and Definitions for Terms Related to Driving Automation Systems for On-Road Motor Vehicles*. SAE International, 2021. Source of the operational design domain and the levels of driving automation.
- [83] Oscar Sainz, Jon Ander Campos, Iker García-Ferrero, Julen Etxaniz, Oier Lopez de Lacalle, and Eneko Agirre. NLP evaluation in trouble: On the need to measure LLM data contamination for each benchmark. In *Findings of the Association for Computational Linguistics: EMNLP*, 2023.
- [84] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.
- [85] Nithya Sambasivan, Shivani Kapania, Hannah Highfill, Diana Akrong, Praveen Paritosh, and Lora M. Aroyo. “everyone wants to do the model work, not the data work”: Data cascades in high-stakes AI. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*, 2021.
- [86] D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François

- Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015. URL <https://papers.nips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems>.
- [87] Adam Shostack. *Threat Modeling: Designing for Security*. Wiley, 2014.
- [88] Ilya Shumailov, Zakhar Shumaylov, Yiren Zhao, Nicolas Papernot, Ross Anderson, and Yarin Gal. AI models collapse when trained on recursively generated data. *Nature*, 631:755–759, 2024. URL <https://www.nature.com/articles/s41586-024-07566-y>.
- [89] Herbert A. Simon. *The Sciences of the Artificial*. MIT Press, 3rd edition, 1996. The architecture of complexity and near-decomposability.
- [90] Linda J. Skitka, Kathleen L. Mosier, and Mark Burdick. Does automation bias decision-making? *International Journal of Human-Computer Studies*, 51(5): 991–1006, 1999.
- [91] Martin Thomson and David Schinazi. *Maintaining Robust Protocols*. Internet Architecture Board, 2023. URL <https://www.rfc-editor.org/rfc/rfc9413>. RFC 9413.
- [92] Andreas Tolk and James A. Muguira. The levels of conceptual interoperability model. In *Proceedings of the Fall Simulation Interoperability Workshop*, 2003.
- [93] United States Environmental Protection Agency. Notice of violation to volkswagen AG, audi AG and volkswagen group of america. Clean Air Act notice of violation, 2015.
- [94] Naman Vats and Oleg Golev. The scaffold effect in coding agents: Harness choice as a hidden variable in coding-agent evaluation. *arXiv preprint arXiv:2607.22585*, 2026. URL <https://arxiv.org/abs/2607.22585>.
- [95] Simon Willison. The lethal trifecta for AI agents: Private data, untrusted content, and external communication. simonwillison.net, 2025.
- [96] Andrew Wong, Erkin Otles, John P. Donnelly, Andrew Krumm, Jeffrey McCullough, Olivia DeTroyer-Cooley, et al. External validation of a widely implemented proprietary sepsis prediction model in hospitalized patients. *JAMA Internal Medicine*, 181(8):1065–1070, 2021.
- [97] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024. URL <https://arxiv.org/abs/2406.12045>.

- [98] Mengqi Yuan, Zilong Zhou, Xinzhuang Xiong, et al. OSWorld 2.0: Benchmarking computer use agents on long-horizon real-world tasks. *arXiv preprint arXiv:2606.29537*, 2026. URL <https://arxiv.org/abs/2606.29537>.
- [99] Pamela Zave and Michael Jackson. Four dark corners of requirements engineering. *ACM Transactions on Software Engineering and Methodology*, 6(1): 1–30, 1997.
- [100] Rui Zhang and Quanyan Zhu. FlipIn: A game-theoretic cyber insurance framework for incentive-compatible cyber risk management of Internet of Things. *IEEE Transactions on Information Forensics and Security*, 15:2026–2041, 2020.
- [101] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Proceedings of the NeurIPS Datasets and Benchmarks Track*, 2023.
- [102] Quanyan Zhu. Game theory meets LLM and agentic AI: Reimagining cybersecurity for the age of intelligent threats. *arXiv preprint arXiv:2507.10621*, 2025. URL <https://arxiv.org/abs/2507.10621>.
- [103] Quanyan Zhu and Zhiheng Xu. *Cross-Layer Design for Secure and Resilient Cyber-Physical Systems: A Decision and Game Theoretic Approach*, volume 81 of *Advances in Information Security*. Springer, 2020.
- [104] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023. URL <https://arxiv.org/abs/2307.15043>.